

Pixel-level Reconstruction Large Language Model

1. 基础数学理论

1.1. 微积分前置基础约定

1.1.1. 第一组：导数、偏导数、全导数（变化率：微小增量的比值）

1.1.1.1. 导数（一元函数专属）

1.1.1.2. 偏导数（多元函数，固定其余自变量）

1.1.1.3. 全导数（多元复合函数，所有自变量都随同一个参数变化）

1.1.1.4. 核心区别

1.1.2. 第二组：微分、偏微分、全微分（微小增量：线性主部）

1.1.2.1. 微分（一元函数）

1.1.2.2. 偏微分（多元函数，单个自变量微小变化）

1.1.2.3. 全微分（多元函数，所有自变量同时微小变化）

1.1.3. 第三组：梯度（多元函数 $\rightarrow$ 向量，深度学习核心）

1.1.3.1. 定义

1.1.3.2. 梯度与全微分的绑定关系（矩阵求导/迹体系核心）

1.1.3.3. 矩阵梯度

1.1.4. 极简对比表

1.1.5. 易混易错

1.2. 线性代数前置基础约定

1.3. 概率论前置基础约定

1.3.1. 概率与分布

1.3.2. 数字特征

1.3.3. 信息论视角（现代概率的延伸）

1.3.4. 条件概率

2. 从单个标量理解神经网络Backward的工作原理

2.1. 求解一个未知数

2.2. 通过误差修正盲猜假设

2.3. 补充知识：导数

2.4. 利用损失函数的导数寻找极小值

2.5. 思考

2.6. python

3. 微分传递的过程中，梯度是怎么“自然浮现”的？

3.1. 步骤 1：传递微分（链式法则本体）

3.2. 步骤 2：从微分中“读出”梯度对比全微分的标准形式：

4. 从行向量/列向量的点积理解逐元素运算的求导逻辑：

- 4.1. 行向量/列向量的点积
- 4.2. 补充知识：标量对向量的求导
- 4.3. 神经网络中，从标量loss把梯度回传矩阵W
- 4.4. 思考
- 4.5. python
- 5. 从“单样本-多输出”来理解多个神经元如何协作
 - 5.1. 单样本-多输出意味着什么？
 - 5.2. 均方误差 (MSE)
 - 5.3. 反向求导
 - 5.4. 思考
 - 5.5. 从全微分角度再理解一次
 - 5.6. python
- 6. 依靠“向量-Jacobian乘积”，绕过Jacobian Matrix!
 - 6.1. 繁杂的Jacobian Matrix
 - 6.2. 现代神经网络反向传播的做法
- 7. 从权重的损失景观引发的思考
 - 7.1. “一个样本-一个神经元”模型
 - 7.2. 构建欧几里得空间
 - 7.3. 解空间的退化 (Degeneracy)
 - 7.4. “两个样本-一个神经元”模型
 - 7.4.1. 再引入一个数据样本
 - 7.4.2. 碗状谷
 - 7.5. 过拟合与泛化
 - 7.5.1. 单样本例子：极端的过拟合
 - 7.5.2. 双样本例子：通过增加数据抑制过拟合
 - 7.6. 由解空间的退化引发的思考（如下内容特指隐藏层）
 - 7.6.1. 样本数量决定了什么？
 - 7.6.1.1. 样本数是决定模型性能的“天花板”
 - 7.6.1.2. 样本数决定泛化能力（过拟合 vs 欠拟合）
 - 7.6.2. 样本特征数量决定了什么？
 - 7.6.2.1. 决定了每个神经元的“感受野”规模（突触数量）
 - 7.6.2.2. 决定“维度灾难”的严重程度
 - 7.6.2.3. 决定了网络自动处理输入噪声的难度
 - 7.6.2.4. 决定了模型从“原始输入”到“精炼特征”的压缩比
 - 7.6.3. 神经元权重数量决定了什么？
 - 7.6.3.1. 决定了神经元的“感受野”大小和粒度
 - 7.6.3.2. 决定了该神经元决策边界的维度
 - 7.6.3.3. 决定了该神经元被冗余或无关特征“误导”的风险
 - 7.6.4. 神经元数量决定了什么？

- 7.6.4.1. 决定了模型的“特征检测器”丰富度（表示宽度）
 - 7.6.4.2. 决定了模型的容量与过拟合风险（表示能力的上限）
 - 7.6.4.3. 决定了网络的鲁棒性与冗余度
- 7.6.5. 小结
- 7.7. 收拾行李再出发
- 8. 打破线性诅咒！
 - 8.1. 从线性变换到仿射变换
 - 8.2. 激活函数：Sigmoid
 - 8.2.1. 来自非线性相关的难题
 - 8.2.2. 让直线弯曲的奥秘
 - 8.2.2.1. 初识Sigmoid
 - 8.2.2.2. 近似逼近
 - 8.3. 为什么要引入非线性相关？
 - 8.3.1. 经典XOR问题
 - 8.3.2. 多层线性层等价于单层线性层
 - 8.3.3. 更多激活函数
 - 8.3.4. 激活函数的抉择
- 9. 从理论到实践---Dirty your hands
 - 9.1. 设备
 - 9.2. IDE
 - 9.3. 深度学习框架
 - 9.4. 环境配置
- 10. 构建BigramMLP——MLP原理与前向传播
 - 10.1. 大模型如何学会生成名字？
 - 10.2. 导入与处理数据
 - 10.2.1. 导入数据
 - 10.2.2. 构建数据集和目标集
 - 10.2.3. 映射
 - 10.3. 基于计数的Bigram概率矩阵
 - 10.3.1. 计数
 - 10.3.2. 归一化
 - 10.4. 独热编码（One-Hot）
 - 10.5. 前向传播——线性变换+非线性激活
 - 10.5.1. 线性变换（加权求和 + 偏置）
 - 10.5.2. 非线性激活
 - 10.5.3. 潜在危机——梯度消失
 - 10.6. 前向传播——从隐藏层到输出层
 - 10.6.1. 线性变换
 - 10.6.2. Softmax

10.6.3. 交叉熵

10.6.3.1. 真实概率 $p(x)$

10.6.3.2. 预测概率 $q(x)$ ——置信度

10.6.3.3. 为什么会有个 $\log()$ 呢?

10.6.3.4. 为什么 $p(x)$ 和 $\log q(x)$ 之间是乘法运算?

10.6.4. 像素级推导交叉熵的计算

10.6.5. 最后一击——求loss

11. 构建BigramMLP——反向传播

11.1. 反向传播的另一种视角

11.1.1. 《生命法典》

11.1.2. 肩负反向传播的天命人

11.2. 广播机制

11.2.1. 概述

11.2.2. 隐式广播

11.2.3. 手动广播

11.3. 反向传播 (Backpropagation)

11.3.1. 梯度之源

11.3.2. $L \rightarrow \text{sum_loss}$

11.3.3. $\text{sum_loss} \rightarrow \log_probs$

11.3.4. $\log_probs \rightarrow probs$

11.3.5. $probs \rightarrow counts \quad \& \quad counts \rightarrow \text{sum_inv}$

11.3.6. $counts \rightarrow \text{sum_inv} \rightarrow counts_sum$

11.3.7. $counts_sum \rightarrow counts$

11.3.8. $counts \rightarrow logits2$

11.3.9. $\mathbf{logits2} \rightarrow \mathbf{h} \quad \& \quad \mathbf{W2} \quad \& \quad \mathbf{h} \rightarrow \mathbf{logits1}$

11.3.10. $\mathbf{h} \rightarrow \mathbf{logits1}$

11.3.11. $\mathbf{logits1} \rightarrow \mathbf{W1} \quad \& \quad \mathbf{b1}$

11.3.12. 反向传播完整代码

12. 构建BigramMLP——梯度更新与批次训练

12.1. 通过梯度更新

12.2. loss下降

12.3. 逐批次训练

12.4. 再优化

12.4.1. batchsize

12.4.2. learning rate

12.4.3. 终极尝试

12.5. 权重指纹

13. 构建BigramMLP——生成名字

13.1. 创建 model.pt

- 13.2. 创建 generate.py
- 13.3. 生成!
- 14. 关于BigramMLP的补充
 - 14.1. 性能优化
 - 14.1.1. 设备优化
 - 14.1.2. 代码优化
 - 14.2. 为什么显存越大越好?
 - 14.2.1. 深度学习框架 (PyTorch) 底层计算图与内存管理机制
 - 14.2.2. 为什么需要给loss加一个.item()?
 - 14.2.3. 为什么我们需要大显存?
 - 14.3. Xavier 初始化
 - 14.3.1. 高斯采样: .randn()
 - 14.3.2. 概率论中“大量独立同分布变量之和”的方差
 - 14.3.3. “大量独立同分布变量之和”的方差在输入层中的体现
 - 14.3.4. 解决方法
 - 14.4. 学习率优化器
 - 14.4.1. 损失的变化率
 - 14.4.2. 寻找合适的函数
 - 14.4.3. 代码处理
 - 14.5. 代码中的数据类型
- 15. L2正则化 (L2 Regularization)
 - 15.1. L2正则化起源与过程
 - 15.1.1. 起源
 - 15.1.2. 过程
 - 15.2. 为什么需要正则化?
- 16. 结语

碎碎念-写在前边

“我似乎只像是一个在海边玩耍的男孩，不时找一颗平滑的卵石，或比较美丽的贝壳来取悦自己，而真理的大海则横陈在我面前，一无发现。”

——牛顿

我的科学启蒙是一本杂志，一本没有封面的杂志，那是我第一次海边拾贝。

即使在多年以后的今天，我还依然清晰地记得，那是一个下过雨的上午。在北方农村，下过雨的天气异常凉爽，土质地面很快将积水吸干，我在路上发现了那本杂志。它躺在地面上，封面不知何缘故，已经不见了。我捡起来，泥巴仅附着在朝下的几张纸上，我抖掉上边的泥巴，庆幸这可怜的家伙没有被雨水打碎。

那本书里具体什么内容，我已忘却大部分。但我印象深刻的是，其中有极大篇幅的宇宙天体图片，那些奇幻瑰丽的宇宙图景，神奇的宇宙现象令我着迷。我一整个上午就蹲在原地，翻看那本杂志。只是当时

不以为然，多年以后才意识到，我对科学最早的记忆来自于那本没有封面的杂志。那是我第一次想捡起一片贝壳。

虽然着迷于这些科学，可我数学并不好，理科成绩也一般般。我总是会在课下思考那些奇怪的东西，我甚至在高三最紧张的那段日子里，花了一晚上去思考，化学方程式里的加号和数学里的加号有什么区别？后来阴差阳错，选了化学这个专业，大抵是自己对物理和数学的不自信吧。

大学四年，读研三年，似乎要消磨尽这种热情，以至于在读研最后一年，拼了命的考公。几番挣扎，总是差点意思，便也渐渐接受了自己不是公务员这块料。

毕业的最后半年，是2022年末。ChatGPT-3.0火爆全球，铺天盖地的报道令我兴奋，似乎那种热情又被重新点燃一般，那片贝壳似乎若隐若现地再次出现。我当晚在朋友圈写下长文，惊叹这项技术，并对未来充满想象。作为化学人，我意识到有机反应这种结构化的数据，似乎也可以被训练为大模型。我对这个想法并无确信，只是莫名感到潜力巨大。于是毅然决定进入化工行业工作，寻找机会，期间学习一下相关基础理论。

开始工作的半年，常驻上海出差。得益于上海国际性展会的规模与频次，令我闲暇之余得以经常逛展。那年下半年，去各化工、自动化和人工智能展会看前沿技术。惊叹于实验室自动化的发展速度，后来具身智能的火爆令我更加确信，大模型+自动化一定是化工行业的未来。

从上海出差回到江苏，在自己的租房里搭了第一个学习大模型的本地机。开始啃花书、李沐的动手学深度学习，奈何自己的基本功太差，于是开始补微积分和概率论。如同久饿之人看到食物，想一下吃成胖子，在庞大的数学体系前，我啃了一个月，依然觉得自己像个门外汉，加上我喜欢刨根问底，在一个数学问题上喜欢深究，难免落入数学分析的层层地狱，等到发现啃不动，才知道自己走偏了方向。

那段时间，一边在数学的世界里游荡，认为数学基础打好了才能接触大模型相关理论，一边面对市面上众多优秀的教材，如同一只布利丹驴，把时间浪费在选择上，似乎从未踏上前进的旅途。

直到2025年3月，偶然刷到Karpathy的一个视频：深入探索像ChatGPT这样的大语言模型。Karpathy是OpenAI的联合创始人之一，他对于大模型的解读深入且有趣，对我产生了极大的震撼。我去翻他的其他视频，“**Neural Networks: Zero to Hero**”这个系列视频，影响我至深至远，这篇文档也是基于Karpathy这个视频的整体逻辑来的，其中补充了相关的数学基础知识，从很大程度上来说，这篇文档是这个视频的注脚，也是我的“观后感”。

Neural Networks: Zero to Hero系列视频，质量极高。但对于一个非科班出身，没写过几行代码的我来说，无异于天书，第一个视频时长2个小时，最基本的内容，我来回刷了四五遍，一遍遍复现代码，尝试去理解每一行的作用及原理。

第一个视频还好，到了第二个视频，关于矩阵乘法，One-Hot编码，我始终无法建立持久准确的空间想象，这对我来说是不可接受的，为了理解这些，我尝试再去学习线性代数。期间再次出差，无意中读到一位网友推荐的科普书：《why machines learn》，这本书对我理解那些算法也提供了莫大的帮助。就这样，断断续续，东一榔头西一棒头的学着。

下半年被朋友骗走所有积蓄，意志消沉了很久，几乎就要精神崩溃，学习进度也几乎为零。后边自己打官司，东奔西走，渐渐觉得命运使然，开始信命。Deepseek说我这叫官印相生，须得以甲木正印，以智慧化解困境，我不明白具体什么意思，但那段时间开始决心走出阴霾，重新捡起那片贝壳。

于是有了这篇文档。

时间一直在继续，思考一直在继续，那本启蒙杂志的影响也一直在继续。

在思考中，我才能对未来保持积极的想象。

我常觉得自己太笨，无法参透这世间的所有真理，无法企及“朝闻道，夕死可矣”的高度，常因身处浩然宇宙而觉人类之渺小，又常感叹于人类顶尖思考者的伟大灵感；但又庆幸自己是个笨蛋，只懂一根筋

地做自己喜欢的事。

这篇笔记，赠予无数个追求真理的小小人类。

谨以此文，献给我坚韧且智慧的母亲。

1. 基础数学理论

朋友们，构建大模型冒险之旅出发在即，出发前请原谅我在第一章不免落俗地为你介绍一些枯燥地数学知识。

神经网络不是一天炼成的，无数科学先驱在这条路上披荆斩棘，才有了今天 ChatGPT/Gemini/Deepseek 这样的浩然工程。

现代大模型的成型之路，就是这些先驱不断解决问题的路，而他们手中唯一的剑，就是数学这把无上利刃。他们可能未曾读过《剑来》，也不懂东方剑客的诗意，但我想借用书中一句话，为数学这把利剑开刃：唯有一剑，可搬山，可倒海，可斩神，可杀仙，可开天，可裂地！

这不是本次冒险的臃肿背包，而是我们神挡杀神，佛挡杀佛的唯一武器。

即使在本篇文章结束之后，我希望你依然可以带着这把剑，笑着回味这次冒险时淡淡说出：一身转战三千里，一剑曾当百万师。

冒险开始。

话说，一名训练有素的剑客，应该对手中的武器了如指掌，做到人剑合一，接下来，让我为你简单介绍一下你手中的这把剑：

微积分是剑刃：在反向传播中，只能靠它劈开重重隐藏层，直抵矩阵之心。

线性代数是剑身：所有数据都将浇筑成张量形状，张量在千锤百炼中锻造变换与成型，线性代数令其刚柔并济，承托万千数据，统筹多维空间，为利刃筑牢根本。

概率论是剑谱：数据如何听命于执剑人？初始化、批归一化、采样与分布..... 一招一式无不严苛遵守此剑谱，循律而行，顺势而为，剑随心动，正是概率论的无上心法。

1.1. 微积分前置基础约定

先**统一符号、划分层级**，区分 **导数/偏导数/全导数 / 微分/偏微分/全微分 / 梯度** 七大概念，同时理清关联与混用误区。

设多元函数： $u = f(x_1, x_2, \dots, x_n)$

自变量： $x_1, x_2, \dots, x_n$

因变量： u （标量）

微分符号： d （整体微分）、 ∂ （偏微分/偏导）

场景：先从**一元函数**入手，再延伸**多元函数**，最后到**向量/矩阵、梯度**（对接神经网络）

1.1.1. 第一组：导数、偏导数、全导数（变化率：微小增量的比值）

核心：描述 **自变量变化** 与 **因变量变化的比值**（斜率or变化率），是微小增量的比值。

1.1.1.1. 导数（一元函数专属）

仅一个自变量 $y = f(x)$

定义：

$$f'(x) = \frac{dy}{dx} = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

含义： x 变化时， y 的瞬时变化率；

符号： $\frac{dy}{dx}$, $f'(x)$ ；

范围： **只存在于一元函数。**

1.1.1.2. 偏导数（多元函数，固定其余自变量）

多元函数 $u = f(x, y)$ ， 只让一个自变量变动， 其余全部固定不变。

对 x 的偏导数：

$$\frac{\partial u}{\partial x} = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x, y) - f(x, y)}{\Delta x}$$

对 y 的偏导数：

$$\frac{\partial u}{\partial y} = \lim_{\Delta y \rightarrow 0} \frac{f(x, y + \Delta y) - f(x, y)}{\Delta y}$$

符号：必带 ∂ ；

关键规则：求 $\frac{\partial u}{\partial x}$ 时，把 y 当常数；

例子（神经网络常用）：

$$L(w_{11}, w_{12}) = x_1 w_{11} + x_2 w_{12} , \quad \frac{\partial L}{\partial w_{11}} = x_1, \quad \frac{\partial L}{\partial w_{12}} = x_2$$

1.1.1.3. 全导数（多元复合函数，所有自变量都随同一个参数变化）

场景：多元函数 $u = f(x, y)$ ， 且 x, y 本身都是单参数 t 的函数：

$$x = x(t), \quad y = y(t)$$

此时 u 最终是一元函数 $u(t)$ ， 它对 t 的导数叫全导数， 符号用 d ：

$$\frac{du}{dt} = \frac{\partial u}{\partial x} \frac{dx}{dt} + \frac{\partial u}{\partial y} \frac{dy}{dt}$$

1.1.1.4. 核心区别

偏导：只动一个自变量，其余固定；

全导数：所有中间变量全都随同一个参数联动变化；

符号：用 d 而非 ∂ 。

1.1.2. 第二组：微分、偏微分、全微分（微小增量：线性主部）

核心：描述自变量发生微小变化 dx, dy 时，因变量对应的近似微小变化量，是增量表达式，不是变化率。

导数or偏导 = 变化率（微小增量的比值）

微分or全微分 = 微小增量（线性近似量）

1.1.2.1. 微分（一元函数）

一元 $y = f(x)$ ，由导数定义变形：

$$\Delta y \approx dy = f'(x) dx$$

dx ：自变量 x 的微小增量；

dy ： y 的微分（ Δy 的线性主部，近似微小变化）；

关系： $dy = \frac{dy}{dx} \cdot dx$

导数是微分的系数。

1.1.2.2. 偏微分（多元函数，单个自变量微小变化）

多元 $u = f(x, y)$ ，仅让 x 变、 y 固定：

$$\partial u = \frac{\partial u}{\partial x} dx$$

仅让 y 变、 x 固定：

$$\partial u = \frac{\partial u}{\partial y} dy$$

含义：单个自变量微小变动带来的因变量增量；

符号： ∂u ，日常单独使用较少，一般并入全微分。

1.1.2.3. 全微分（多元函数，所有自变量同时微小变化）

多元函数最常用形式，所有自变量一起发生微小增量：

$$du = \frac{\partial u}{\partial x} dx + \frac{\partial u}{\partial y} dy$$

推广到 n 元：

$$du = \sum_{i=1}^n \frac{\partial u}{\partial x_i} dx_i$$

结合神经网络例子（核心对标）

损失 $L = L(y_{\text{pre1}}, y_{\text{pre2}})$

全微分：

$$dL = \frac{\partial L}{\partial y_{\text{pre1}}} dy_{\text{pre1}} + \frac{\partial L}{\partial y_{\text{pre2}}} dy_{\text{pre2}}$$

dL : 损失的全微分 (损失整体微小变化)

$\frac{\partial L}{\partial y_{pre1}}$: 偏导数 (变化率)

dy_{pre1} : 中间变量的微分

1.1.3. 第三组：梯度 (多元函数 $\rightarrow$ 向量, 深度学习核心)

1.1.3.1. 定义

对多元标量函数 $u = f(x_1, x_2, \dots, x_n)$, 梯度是把所有一阶偏导数按顺序拼成的向量 (本质也是个比值) :

$$\nabla u = \left(\frac{\partial u}{\partial x_1}, \frac{\partial u}{\partial x_2}, \dots, \frac{\partial u}{\partial x_n} \right)$$

本质: 偏导数构成的向量, 不是单个数;

几何意义: 函数值增长最快的方向;

符号: ∇ (nabla)

1.1.3.2. 梯度与全微分的绑定关系 (矩阵求导/迹体系核心)

全微分可以写成向量内积形式:

$$du = \nabla u \cdot (dx_1, dx_2, \dots, dx_n)^\top$$

神经网络实例

设:

$$\nabla_{\mathbf{Y}_{pre}} L = \left(\frac{\partial L}{\partial y_{pre1}}, \frac{\partial L}{\partial y_{pre2}} \right)$$

微分向量:

$$d\mathbf{Y}_{pre} = (dy_{pre1}, dy_{pre2})^\top$$

则全微分:

$$dL = \nabla_{\mathbf{Y}_{pre}} L \cdot d\mathbf{Y}_{pre}$$

1.1.3.3. 矩阵梯度

当自变量是矩阵 $\mathbf{W}$ 时:

把矩阵所有元素看成一组自变量;

$\nabla_{\mathbf{W}} L$ 就是和 $\mathbf{W}$ 同形状的梯度矩阵, 每个位置元素 = L 对对应矩阵元素的偏导数。

1.1.4. 极简对比表

名称	适用场景	符号	本质	核心特征
导数	一元函数 $y = f(x)$	$\frac{dy}{dx}$	一元函数变化率 (数)	只用 d
偏导数	多元函数, 单变量变动	$\frac{\partial u}{\partial x}$	单方向变化率 (数)	必用 ∂
全导数	多元复合函数, 全变量随参数变	$\frac{du}{dt}$	复合路径总变化率 (数)	链式法则, 用 d
微分	一元函数微小增量	dy	一元函数微小变化量	$dy = y' dx$
偏微分	多元函数, 单变量微小变动	∂u	单变量带来的微小增量	极少单独用
全微分	多元函数, 全变量同时变动	du	总微小增量 (线性组合)	偏导×微分 求和
梯度	多元标量函数	∇u	全体偏导数拼成的向量/矩阵	深度学习核心

1.1.5. 易混易错

- d 和 ∂ 怎么选?
只一个自变量：一律用 d （导数、微分）
多个自变量、固定其余只动一个：一律用 ∂ （偏导、偏微分）
多元函数所有自变量一起动：整体增量用 d （全微分）
• 导数 $\neq$ 微分
导数： $\frac{dy}{dx}$ 是比值、斜率；
微分： dy 是增量；
关系： $dy = \text{导数} \cdot dx$ 。
• 偏导数 $\neq$ 全导数
偏导：人为固定其他变量；
全导数：变量之间存在函数依赖，全部联动变化。
• 梯度 $\neq$ 偏导数
偏导数：单个标量；
梯度：一堆偏导数组成的向量/矩阵。

线性代数前置基础约定

1.2.

因为线性代数是贯穿整个文档的基础，是我们从浅入深的脉络，所以，很多知识我们会在章节中的知识补充中进行学习。

去强化你的兵器吧！

1.3. 概率论前置基础约定

1.3.1. 概率与分布

这是描述随机变量底层规律的基石。

- Ω : **样本空间**：随机试验 E 所有**基本结果（样本点）**构成的全集集合；单个基本结果记作 ω （样本点）， $\omega \in \Omega$ 。 $\Omega = \{\omega_1, \omega_2, \dots, \omega_n, \dots\}$ 。

掷一颗骰子的样本空间是： $\Omega = \{\square, \square, \square, \square, \square, \square\}$ 。

- X : **随机变量本身**，它像一个“黑盒子”，是一个**函数/规则**，结果不确定。可以这样理解： $X = \text{“掷一颗骰子的点数”}$ 。它是一个概念，一个等待被执行的过程。

随机变量 X 其实是一个**函数**，它所做的就是从样本空间 Ω 到实数轴 R 的一个映射：

$X: \Omega \rightarrow R$ 。它给每一个可能的结果，按你定义的规则，**分配一个独一无二的实数**。 X 取值的意义是人为定义的，它只提供一种区分/信息。

- x : **随机变量的一个具体取值**。它是这个黑盒子打开之后，真实发生的一个**数字**。比如，掷出“3点”，这个3就是 x 。3 不是变量，它是确定的。
- $P(A)$: **概率**，事件 A 发生的可能性。
- $P(A | B)$: **条件概率**，就是上一轮我们讨论的核心，表示在 B 发生下 A 的概率。
- $F(x) = P(X \leq x)$: **累积分布函数**。它给出随机变量 X 取值不超过 x 的概率，是完整描述随机变量统计规律的最根本函数。无论是离散还是连续变量，都有这个函数。
- $f(x)$ 或 $p(x)$: **概率密度/质量函数**。

- **质量函数** $p(x)$ 用于离散变量，直接给出 $X = x$ 的概率： $P(X = x)$ 。比如抛骰子， $P(X = 3) = 1/6$ 。

- **密度函数** $f(x)$ 用于连续变量，它的关键作用是积分后得到概率：

$$P(a \leq X \leq b) = \int_a^b f(x) dx. \text{。需要注意，密度值 } f(x) \text{ 本身可以大于1，并且}$$
$$P(X = x) = 0.$$

1.3.2. 数字特征

它们把复杂的分布浓缩成一个或几个关键数值，描述其核心特性。最核心的是“期望”和“方差”。

- $E[X]$: **期望**。理论上的均值，就是概率加权下的“重心”。
- 离散： $E[X] = \sum x \cdot p(x)$

- 连续: $E[X] = \int x \cdot f(x)dx$ 。 $f(x)dx$ 来自于极限理论: $f(x)$ 是表“高”, dx 是“底”, $f(x)dx$ 意为曲线下面积, 即概率。
- $E[g(X)]$: **函数的期望**。算的不是 X 本身, 而是其函数 $g(X)$ 的均值。例如 $E[X^2]$ 就是把上面公式里的 x 换成 x^2 再求和/积分。这在计算方差时至关重要。
- $Var(X)$ 或 $D[X]$: **方差**。衡量随机变量在期望周围的离散程度。定义式是 $Var(X) = E[(X - E[X])^2]$, 但计算时几乎都用简化公式:

$$Var(X) = E[X^2] - (E[X])^2$$
- σ 或 $SD[X]$: **标准差**。方差的算术平方根, **量纲与原始数据一致**, 做描述时更直观。
- $Cov(X, Y)$: **协方差**。刻画两个变量线性相关的方向与强度。

$$Cov(X, Y) = E[(X - E[X])(Y - E[Y])] = E[XY] - E[X]E[Y]$$
- ρ 或 $Corr(X, Y)$: **相关系数**。就是对协方差做标准化, **消除量纲影响**, 值永远在 $[-1, 1]$ 之间。公式为: $\rho = \frac{Cov(X, Y)}{\sqrt{Var(X)Var(Y)}}$ 。

1.3.3. 信息论视角 (现代概率的延伸)

这在机器学习和现代统计中极其常见。

- $H(X)$: **熵**。度量随机变量的不确定性大小。比如, 一个均匀骰子的熵就比一个灌铅骰子高。公式为 $H(X) = -\sum p(x)\log p(x)$ 。
- $D_{KL}(P\|Q)$: **KL 散度** (相对熵)。用来衡量两个概率分布 P 和 Q 之间的差异。它不对称, 即 $D_{KL}(P\|Q) \neq D_{KL}(Q\|P)$ 。机器学习里的交叉熵损失函数, 本质上就是 KL 散度加上一个常数。

1.3.4. 条件概率

条件概率是概率论中的一个核心概念, 它描述的是在已知某个事件已经发生的条件下, 另一个事件发生的概率。

设 A 和 B 是两个事件, 且 $P(B) > 0$, 则在事件 B 已发生的条件下, 事件 A 发生的**条件概率**记为 $P(A|B)$, 定义为:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

其中:

- $P(A \cap B)$ 是事件 A 和事件 B **同时发生**的概率。
- $P(B)$ 是事件 B 发生的概率。

直观上, 这个公式表示我们将样本空间从原来的整个空间缩小到了事件 B 的范围内, 然后计算在这个新空间中 A 所占的比例。

大语言模型可以定性为一个巨大的条件概率。以“我爱你”为例, 给出“我爱”的条件, 生成“你”的概率是:

$$P(\text{你}|\text{我爱}) = \frac{P(\text{你} \cap \text{我爱})}{P(\text{我爱})}$$

2. 从单个标量理解神经网络Backward的工作原理

剑意沛然，那让我们带着这把利剑，杀进新手村，面对这次冒险的第一个敌人。

2.1. 求解一个未知数

我们给定一个数据 $X1$ 和 $W1$ ，可以满足： $X1 * W1 = Y1$ （ $Y1$ 就是我们的目标值），假设我们不知道 $W1$ 应该是多少，那该如何找到这个 $W1$ ？

2.2. 通过误差修正盲猜假设

- 现在，我们随机假设一个 $W1$ ，我们就令 $W1 = 2$ 吧（在神经网络中， W 为权重，是通过 `torch.randn` 随机生成的）；第一次计算 $X1 * W1 = OUT = 2$ ，离 5 差得远呢，需要调整 $W1$ ；关键是，如何调整？
- 我们发现 OUT 和 $Y1$ 之间存在着误差，我们将这个误差想办法调整为 0，不就可以了嘛！这里就需要定义一个“误差函数”，更专业一点，我们叫他损失函数 `loss`。
- `loss` 在这里是误差的平方来计算的，为什么这样设计？其实还有更多损失函数可以用，这里我们为了理解简单，用它足够了，后边我们还会用到神经网络中常用的另一个损失函数。

2.3. 补充知识：导数

- 这里我们再回顾一下：导数的定义什么？变化率意味着什么？如何用数学表示？其物理意义又代表什么？
- 我这里有一个函数 $y = x^2$ ，想必，学过高中数学的朋友可以轻而易举的告诉我 y 关于 x 的导数，但这里我还是想再回顾一下：
- dx 里的 d 是什么？是取一个极小的数的意思， dx 就是在 x 的某一点，或左或右，取了一个极小值，当 $x + dx$ 的时候， y 于是跟也变化了一个极小值，我们叫它 dy 。
- 这个时候，对应的结果就变成了 $y + dy$ 。根据展开式，我们知道 $y + dy = x^2 + 2x \cdot dx + dx^2$ 。因为原本的 $y = x^2$ ，我们在等式两侧把它们抵消掉，就得到了 $dy = 2x \cdot dx + (dx)^2$ 。由于 dx 本身是一个极小值，其平方更小，在这里我们忽略不计，于是就得到了 $2dx \cdot x = dy$
- 再两侧同时除 dx ，就得到了 $\frac{dy}{dx} = 2x$ 。这就是导数的由来，它决定了导数的变化方向。这里，我们把 $\frac{d}{dx}$ 当作一个算符，这个算符用来求导数， $\frac{d}{dx}y$ 就是 y 对 x 求导！

2.4. 利用损失函数的导数寻找极小值

- 好了，导数的定义说完了，有人要疑惑，为什么突然讲到导数了？还记得我们说的这句话吗：“我们将这个误差想办法调整为 0，不就可以了嘛！”

- 精髓来了！我们有一个 $loss$ 函数，我们找这个函数的最小值，其实就是找到了 $W1$ ！我们把这个找 $W1$ 的过程建模为一个数学问题了！

上述“求解一个未知数”的问题就等价于求loss的最小值问题。

- 你还记得如何求一个标准一元二次函数的最小值吗？当一个凹函数上（局部）某个点的导数为 0 的时候，这个点就是（局部）最小值了！
- 你可以画一个标准的一元二次函数，想象你在函数的某一点，正快速滑向函数底部，是不是你离底部越远，越陡峭，越靠近底部，越平坦！这种陡峭/平坦的感受，就是由导数决定的！
- 我们带上数学的眼睛看这个过程：

你在 $y = x^2$ 的 $x = 2$ 处，此处的导数是 4，你想要前往的点是导数为 0 的点，于是你要逆着导数方向走，你才能走到导数为 0 的点；

你在 $y = x^2$ 的 $x = -2$ 处，此处的导数是 -4，你想要前往的点是导数为 0 的点，于是你还是要逆着导数方向走，你才能走到导数为 0 的点；

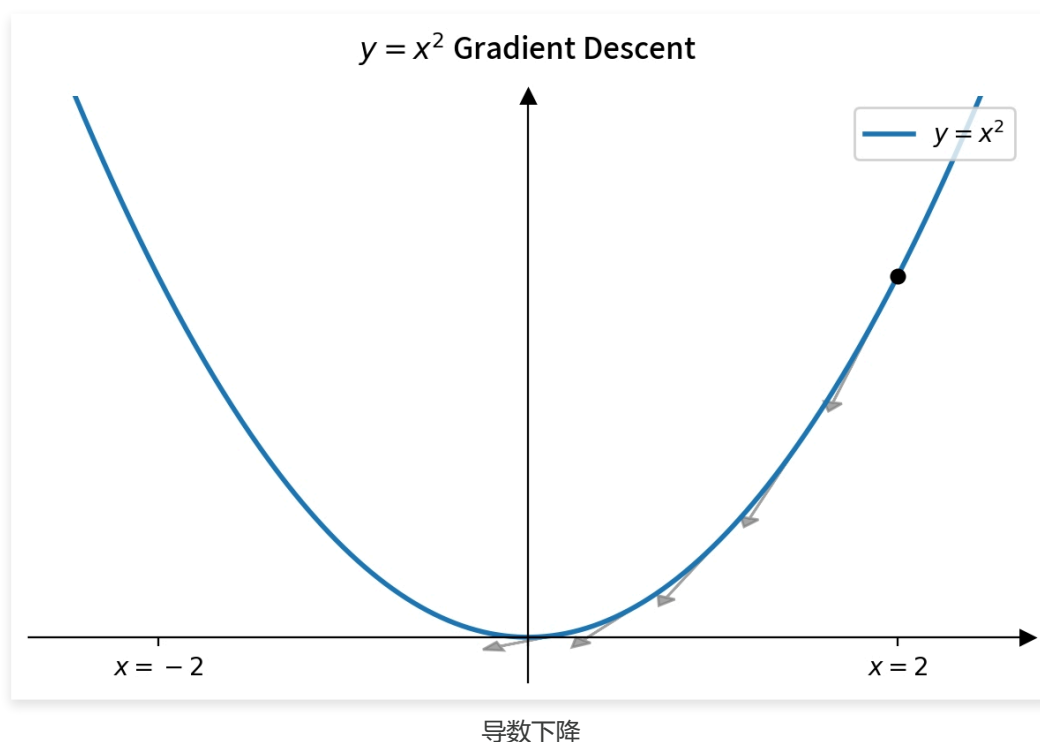
发现了吗？只要我们逆着导数方向走，我们就能走到导数为 0 的点！这就是为什么”

$W_{new} = W_{old} - \Delta W$ “有个减号的原因。

好，在 $y = x^2$ 的 $x = 2$ 处，此处的导数是 4，你迫不及待地调整自己的位置，你直接在 $x = 2$ 这个位置逆着导数走了一整个 4！你来到了 $x = -2$ 处，你再次迫不及待地往反方向走，诡异的事情发生了

你在 $x = 2$ 和 $x = -2$ 两个点来回横跳！永远下不去了，这是你在神经网络中学到的第一个人生哲学：永远不要尝试一口吃成胖子！我们不要着急奔着那个目标就大步甩开地走，边走边调整才是智慧。

- 你现在明白了，你把位置调整地大小改为：导数*步长，更专业一点，这个步长叫做学习率，十分形象，你正在学着如何走向目的地！
- 于是，你一步一步地走，终于在几十步之后，你来到了那个导数为 0 的点附近，这个地方可能永远不是导数为 0 的点，但最先进的计算机也无法区分这个点距离导数为 0 的点有多近了，我们就当作目的地已到了。



- 这个时候，你会发现， $loss$ 稳定在 0.0000 ， $W1$ 稳定在某个常数。而这个常数大约是 5。

2.5. 思考

- 如果你在想：这么麻烦，我还不如直接用 $5 \div 1$ 呢，直接拿到结果，而且还准！
- 朋友们！结果是一样的，只能说殊途同归。你用两种方法，拿到同样的结果，不是意味着，这里边还有你未曾发现的奥秘吗？
- 我们通过一个简单的实验，掌握了神经网络中最重要的技术！反向传播，反向传播就是不断求导数，根据导数更新参数的过程！
- 在后边，我们不再局限在标量计算中，而是使用大量矩阵/张量，对于一个张量，这里的导数就换了名字：梯度！
- 彩蛋1：为什么要 $dW_1 = None$ ，每轮梯度要清零？
- 彩蛋2：在上述例子中， $x = 2$ 时，为了调整到函数的最小值，我们用2减去了 $lr \cdot$ 此处的导数！用数值减去自己的变化率，这在数学上并不严谨，这是神经网络里一种启发式算法。

$$W_{new} = W_{old} - \Delta W = W_{old} - lr * \nabla_W loss$$

- 彩蛋3：为什么是局部最小值？

2.6. python

```

1  X1 = 1
2  W1 = 2
3  y1 = 5
4  dloss = 1.0
5  learning_rate = 0.1
6
7  for i in range(20):
8      out = X1 * W1
9      loss = (out - y1) ** 2
10     dout = 2 * (out - y1) * dloss
11     dW1 = X1 * dout
12     W1 = W1 - learning_rate * dW1
13     print(f"i:{i+1},out:{out:.3f},loss:{loss:.3f},W1:{W1:.3f},dW1:{dW1:.3f}")
14     dW1 = None

```

3. 微分传递的过程中，梯度是怎么“自然浮现”的？

经此一战，你手中的利剑越发坚韧。擦拭一下手中的兵器，为其增加几个配件吧。

我们用简单的线性层例子，完整走一遍微分传递：

设 $L = L(\mathbf{Y}_{pre})$, $\mathbf{Y}_{pre} = \mathbf{XW}$

3.1. 步骤 1：传递微分（链式法则本体）

- 第一步：Loss 对中间变量的微分

$$dL = \nabla_{\mathbf{Y}_{pre}} L \cdot d\mathbf{Y}_{pre}$$

这里的 $\nabla_{\mathbf{Y}_{pre}} L$ ，是 $loss$ 对中间变量的梯度向量。

- 第二步：中间变量对权重的微分

$$d\mathbf{Y}_{pre} = \mathbf{X} \cdot d\mathbf{W}$$

- 第三步：微分传递，合并得到 $loss$ 对 $\mathbf{W}$ 的微分

$$dL = \nabla_{\mathbf{Y}_{pre}} L \cdot \mathbf{X} \cdot d\mathbf{W}$$

- 因为 $\mathbf{X}$ 为常数的情况下，必须满足如下：

$$dL = \frac{dL}{d\mathbf{W}} \cdot d\mathbf{W},$$

- 所以：

$$\nabla_{\mathbf{Y}_{pre}} L \cdot \mathbf{X} = \frac{dL}{d\mathbf{W}} = \nabla_{\mathbf{W}} L$$

3.2. 步骤 2：从微分中 “读出” 梯度对比全微分标准形式：

$$dL = \text{Tr}((\nabla_{\mathbf{W}} L)^{\top} d\mathbf{W})$$

我们发现：

$\nabla_{\mathbf{W}} L = \mathbf{X}^{\top} \cdot \nabla_{\mathbf{Y}_{pre}} L$ 关键逻辑：微分传递的过程，其实就是在不断 “凑” 全微分标准形式，凑完之后，微分前面的系数矩阵，就是梯度。

加油包：

遇到很多公式没看懂，也不妨碍我们往下进行，这些概念并不阻碍我们后续的理解。

我记得雷军在一档访谈节目中提到，他本科学习的时候，遇到不会的就硬着头皮往后看，来回多看几遍，就突然顿悟了。这大概就是：读书百遍，其意自现吧！

如果累了不妨停下来欣赏一下沿途的风光。

每个人在各自人生阶段的任务不同，不要纠结于当下慢了多少。

路虽远，行则将至，只要不放弃前进，终有一天我们会到达成功的彼岸。

4. 从 行向量/列向量的点积 理解 逐元素运算的求导逻辑：

如果前面的路还算畅通，欢迎你来到向量的世界。

行向量/列向量的点积是张量运算的基本单元，通关后，你会理解张量中那些复杂且巨大的运算，其本质就是一个个向量计算之后重组的过程，理解了逐元素（Element-wise）运算的求导逻辑，张量微积分也就不在话下了！

别害怕，拿起手中的剑，先跟它过过招~

4.1. 行向量/列向量的点积

朋友们，之所以先进入向量世界，是因为太早进入张量，对我们来说还有点吓人，索性就从一个标量扩展到几个标量开始，这比较好理解。

其实，这几个标量组合起来，就是一个简单的低阶张量运算了！

点积

- 计算方法

两个向量对应位置元素相乘再求和，结果是一个标量（数）。

- 代数： $\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n$

- 几何： $\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta$

- 在神经网络中的物理意义

点积是神经元线性运算的核心，它测量的是输入与权重之间的“相似度”或“响应强度”。

- a. 特征匹配检测器：当输入向量与某个神经元的权重向量方向一致时，点积最大。这就像一个“模板匹配”，让神经元能检测到特定模式。

- b. 线性变换的基础：一个全连接层的运算 $y = Wx + b$ ，本质就是输入 x 与权重矩阵 W 的每一行分别做点积，完成空间的线性映射。

- c. 注意力分数：在Transformer的自注意力机制中， $Q \cdot K^T$ 计算的就是Query和Key之间的点积相似度，以此来决定该“注意”哪些信息。

- 我们有：数据向量 $\mathbf{X} = [X_1, X_2, X_3]$ 和权重向量 $\mathbf{W} = \begin{bmatrix} W_1 \\ W_2 \\ W_3 \end{bmatrix}$ ， $Y_{pre} = \mathbf{X} \cdot \mathbf{W}$ ，根据

向量点积运算规则： $Y_{pre} = X_1 * W_1 + X_2 * W_2 + X_3 * W_3$ 。

- 如同上个问题，我们先定一个小目标：给定固定的输入数据向量 $\mathbf{X}$ 和标量目标值 Y_{target} （后边都用 Y 来代替），找到一组权重 $\mathbf{W}$ ，使预测值 Y_{pre} 尽可能接近目标值 Y 。

- 这个问题等价于寻找 $loss$ 的最小值，其中： $loss = (Y_{pre} - Y)^2$ 。

- 还是老套路，先随机设 $\mathbf{W}$ ， $\mathbf{X}$ ，暂且就让他们：

$W_1 = 4.0; W_2 = 5.0; W_3 = 6.0, X_1 = 1.0; X_2 = 2.0; X_3 = 3.0$ 吧。

4.2. 补充知识：标量对向量的求导

- 每次计算，都得到一个结果 $Y_{pre} = X_1 \cdot W_1 + X_2 \cdot W_2 + X_3 \cdot W_3$ ，稍有点不同的是：我们反向求导有点难度了。

- 因为我们这里 Y_{pre} 是关于 (W_1, W_2, W_3) 的函数： $Y_{pre}(W_1, W_2, W_3)$ 。

- 这里求导，就是在求 Y_{pre} 对 (W_1, W_2, W_3) 的偏微分，朋友们别害怕，去稍微看一下定义，没那么吓人。

- 我们求 $\frac{\partial Y_{pre}}{\partial W_1}$ 的时候，把 W_2, W_3 当作常数就好了（当然，因为是在给定

$\mathbf{X} = [X_1, X_2, X_3]$ 的情况下，所以 X_1, X_2, X_3 也是常数）

- 如果你会求 $f(x) = ax$ 的导数，这里就难不倒你。

$Y_{pre} = X_1 * W_1 + X_2 * W_2 + X_3 * W_3$ ， $\frac{\partial Y_{pre}}{\partial W_1} = X_1$ ，以此类推，

$$\frac{\partial Y_{pre}}{\partial W_2} = X_2, \quad \frac{\partial Y_{pre}}{\partial W_3} = X_3。$$

- 我们得到 $\frac{\partial Y_{\text{pre}}}{\partial \mathbf{W}}$ (梯度) : $\frac{\partial Y_{\text{pre}}}{\partial \mathbf{W}} = \begin{bmatrix} \frac{\partial Y_{\text{pre}}}{\partial W_1} \\ \frac{\partial Y_{\text{pre}}}{\partial W_2} \\ \frac{\partial Y_{\text{pre}}}{\partial W_3} \end{bmatrix} = \begin{bmatrix} X_1 \\ X_2 \\ X_3 \end{bmatrix}$, 或者我们这样写更准确:

$$\nabla_{\mathbf{W}} Y_{\text{pre}} = \begin{bmatrix} X_1 \\ X_2 \\ X_3 \end{bmatrix} \quad (\text{这个记号 } \nabla_W Y_{\text{pre}} \text{ 读作" } Y_{\text{pre}} \text{ 对 } W \text{ 的梯度"}).$$

- 彩蛋: 为什么 $\frac{\partial Y_{\text{pre}}}{\partial \mathbf{W}}$ 恰好是 $\mathbf{X}$ 的转置: $\mathbf{X}^T$? 工程约定还是数学法则的支配?

全微分

- 对于一个函数 $z = f(x, y)$ ，它的全微分 dz 确实就是各个偏导数乘上对应自变量的微小变化量，再求和：
$$dz = \frac{\partial f}{\partial x} dx + \frac{\partial f}{\partial y} dy$$
- 这表示的是：当所有自变量都发生微小变化时，函数值总共改变了多少。

张量求导：为什么看起来“不求和”？

- 当我们对向量或矩阵求导时，公式其实完全一样，只是“加号”被写进了矩阵运算的求和里。
- 刚才的例子 $Y_{\text{pre}} = X_1 * W_1 + X_2 * W_2 + X_3 * W_3$ 为例，它可以简写为 $Y_{\text{pre}} = \mathbf{X} \mathbf{W}$ 。
- 它的全微分同样是“偏导数乘以自变量的微分，然后求和”：
$$dY_{\text{pre}} = \frac{\partial Y_{\text{pre}}}{\partial W_1} dW_1 + \frac{\partial Y_{\text{pre}}}{\partial W_2} dW_2 + \frac{\partial Y_{\text{pre}}}{\partial W_3} dW_3$$
- 这个式子展开后就是 $dY_{\text{pre}} = X_1 \cdot dW_1 + X_2 \cdot dW_2 + X_3 \cdot dW_3$ 。
- 而我们为了把这一堆求和写得更紧凑，引入了向量/矩阵乘法这个工具。上面的求和正好可以写成向量点积的形式：
$$dY_{\text{pre}} = \begin{bmatrix} X_1 & X_2 & X_3 \end{bmatrix} \begin{bmatrix} dW_1 \\ dW_2 \\ dW_3 \end{bmatrix}$$
- 而那个包含了所有偏导数的行向量 X_1, X_2, X_3 ，就是梯度 $\nabla_W Y_{\text{pre}}$ 的转置。
- 所以，求和并没有消失，它只是被隐藏在了点积运算里。

全微分 vs. 梯度

- dY_{pre} (全微分)：是一个标量数值。它表示当所有自变量都发生一个具体的、微小的变化 (即 $dW_1, dW_2, \dots$) 时，函数值 Y 总共变化了多少。
- $\frac{dY_{\text{pre}}}{d\mathbf{W}}$ (梯度)：是一个向量值函数。它本身不依赖于自变量的变化量，而是由当前的位置 (即 $\mathbf{W}$ 的当前值) 决定的。它给出了 Y 在各个方向上的瞬时变化率。

用一元函数类比：

一元函数 $y = f(x)$

- 导数 $\frac{dy}{dx}$ ：是在某一点的切线斜率，是一个函数或数值。例如，在 $x = 1$ 处，斜率是5。
- 微分 dy ：是当 x 变化了 dx 时， y 近似变化的数值。计算公式就是 $dy = \frac{dy}{dx} \cdot dx$ 。
- 对应到我们的例子：
 - $\frac{dY}{d\mathbf{W}}$ 就相当于斜率 $\frac{dy}{dx}$ ，它告诉你变化的“速率”和“方向”。
 - dY 就相当于 dy ，它告诉你这次具体变动了多少。

4.3. 神经网络中，从标量loss把梯度回传矩阵W

- 在这里，我们的重点不仅仅是求得最后这个 $\nabla_{\mathbf{W}} \text{loss}$ ，而是想看明白，在每次循环中，是如何通过链式法则把梯度分配到下边的！

第一步，我们先固定好我们的目标：求 $\nabla_{\mathbf{W}} loss$ 。

链式法则：

$$f(x) = g(h(x)), \quad \frac{df(x)}{dx} = \frac{df(x)}{dg(x)} \cdot \frac{dg(x)}{dh(x)} \cdot \frac{dh(x)}{dx}$$

根据链式法则， $\nabla_{\mathbf{W}} loss = \frac{\partial loss}{\partial \mathbf{W}} = \begin{bmatrix} \frac{\partial loss}{\partial W_1} \\ \frac{\partial loss}{\partial W_2} \\ \frac{\partial loss}{\partial W_3} \end{bmatrix} = \frac{\partial loss}{\partial Y_{pre}} \cdot \frac{\partial Y_{pre}}{\partial \mathbf{W}}$ ($\nabla_{\mathbf{W}} loss$ 与 $\mathbf{W}$ 同形)

第二步，我们先求 $\frac{dLoss}{dY_{pre}}$ ：

$$loss = (Y_{pre} - Y)^2, \quad \frac{\partial loss}{\partial Y_{pre}} = 2 * (Y_{pre} - Y)$$

第三步，再求 $\frac{\partial Y_{pre}}{\partial \mathbf{W}}$ ：

$$Y_{pre} = X_1 \cdot W_1 + X_2 \cdot W_2 + X_3 \cdot W_3, \quad \frac{\partial Y_{pre}}{\partial \mathbf{W}} = \begin{bmatrix} \frac{\partial Y_{pre}}{\partial W_1} \\ \frac{\partial Y_{pre}}{\partial W_2} \\ \frac{\partial Y_{pre}}{\partial W_3} \end{bmatrix} = \begin{bmatrix} X_1 \\ X_2 \\ X_3 \end{bmatrix}$$

第四步，联立，得到 $loss$ 对 $\mathbf{W}$ 的微分：

$$\nabla_{\mathbf{W}} loss = \frac{\partial loss}{\partial \mathbf{W}} = \begin{bmatrix} \frac{\partial loss}{\partial W_1} \\ \frac{\partial loss}{\partial W_2} \\ \frac{\partial loss}{\partial W_3} \end{bmatrix} = \frac{\partial loss}{\partial Y_{pre}} \cdot \frac{\partial Y_{pre}}{\partial \mathbf{W}} = 2 * (Y_{pre} - Y) \cdot \begin{bmatrix} X_1 \\ X_2 \\ X_3 \end{bmatrix} = 2 * (Y_{pre} - Y) \cdot \mathbf{X}^T = \begin{bmatrix} 2 \cdot (Y_{pre} - Y) \cdot x_1 \\ 2 \cdot (Y_{pre} - Y) \cdot x_2 \\ 2 \cdot (Y_{pre} - Y) \cdot x_3 \end{bmatrix}$$

• 我们看到

$$\frac{\partial loss}{\partial W_1} = X_1 \cdot 2 \cdot (Y_{pre} - Y); \quad \frac{\partial loss}{\partial W_2} = X_2 \cdot 2 \cdot (Y_{pre} - Y); \quad \frac{\partial loss}{\partial W_3} = X_3 \cdot 2 \cdot (Y_{pre} - Y)$$

， $loss$ 对 W_1, W_2, W_3 他们的导数分别由两部分决定，一部分是 X_i (X 不仅仅是输入数据，它更像是一个“信号放大器”)，一部分是 $\frac{\partial loss}{\partial Y_{pre}}$ ， $\frac{\partial loss}{\partial Y_{pre}}$ 是上游产生的“修正因子” or “误差梯度”，由 X_i 去分配给 W_i 。

• 这意味着什么？这个过程就是在描述：当上游传来一个梯度，下游改如何根据这个梯度去修改自身的梯度。

上游产生的 $loss$ 发现， W_3 应该肩负起更大的责任，因为他的系数是 X_3 ，但是 X_3 在初始设定中是3.0，于是给了 W_3 最大的梯度变化，让它大步向前。

于是 W_3 在这个变化中最为激烈（震荡幅度最大）； W_1 在这个变化中最为安静。

• 彩蛋：你是否能把第三步中最后的一个向量改写为两个向量内积的形式？

$$\begin{bmatrix} 2 \cdot (Y_{pre} - Y) \cdot x_1 \\ 2 \cdot (Y_{pre} - Y) \cdot x_2 \\ 2 \cdot (Y_{pre} - Y) \cdot x_3 \end{bmatrix} = ?$$

历史沿用导致的符号混乱：

$\nabla_{Y_{\text{pre}}} \text{loss}$ 是 loss 对 Y_{pre} 的梯度，可以写作： $\frac{d\text{loss}}{dY_{\text{pre}}}$ 。

神经网络中或代码书写中，默认把 loss 对所有 node （子节点）的梯度简写为 $d\text{node}$ （在数学上这是全微分，而非导数或梯度），但纯数学推导情况下我们会单独说明。

$d\text{node}$ 实质上是导数/梯度： $\frac{d\text{loss}}{d\text{node}}$ ，你看到后要知道它背后的意义，不要和全微分混淆。

4.4. 思考

- 所以，我们粗浅的窥到反向传播的一角：权重的调整会自动找到一条最快的路径，让该大的数据往大了走，该小的往小了走。
- 彩蛋：梯度分配的“马太效应”，梯度分配受 X 的大小影响。如果我们的输入特征没有经过归一化（比如 X_1 是0.1而 X_3 是 1000）那么 W_3 的梯度会比 W_1 大得多。
- W_3 疯狂更新， W_1 像在原地踏步，几乎学不到东西。这也就是为什么在训练模型前，我们一定要做 Feature Scaling（特征缩放/归一化）的底层数学逻辑。

4.5. python

```
1  X1 = 1.0
2  X2 = 2.0
3  X3 = 3.0
4  W1 = 4.0
5  W2 = 5.0
6  W3 = 6.0
7  y1 = 15
8  dloss = 1.0
9  learning_rate = 0.05
10
11  for i in range(10):
12      out = X1 * W1 + X2 * W2 + X3 * W3
13      loss = (out - y1) ** 2
14      dout = 2 * (out - y1) * dloss
15      dW1 = X1 * dout
16      dW2 = X2 * dout
17      dW3 = X3 * dout
18      W1 = W1 - learning_rate * dW1
19      W2 = W2 - learning_rate * dW2
20      W3 = W3 - learning_rate * dW3
21      print(f"i:{i+1},dout:{dout:.3f},loss:{loss:.3f},W1:{W1:.3f},dW1:{dW1:.3f},W2:{W2:.3f},dW2:
22          {dW2:.3f},W3:{W3:.3f},dW3:{dW3:.3f}")
23
24      dW1 = None
25      dW2 = None
26      dW3 = None
```

5. 从“单样本-多输出”来理解多个神经元如何协作

激动人心的时刻就要到了，我们前边使用一套“四步”丝滑小连招，成功通关。

这一章，我们要升级难度了：这里的权重不再是单列的列向量，而是多个列向量组合而成。

某种意义上，这种权重已经和现代大模型中的权重无异，它们只存在规模上的差距。这足以让我们感到兴奋！你打败了眼前这个小小的 3×2 矩阵，就意味着，你不会害怕将来遇到的任意形状的矩阵了！

剑在手！跟我走！

5.1. 单样本-多输出意味着什么？

- 对于数据矩阵： $\mathbf{X} = [x_1, x_2, x_3]$ ，我们有：

$$\mathbf{X} = \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \leftarrow \text{样本1}$$

$\uparrow \quad \uparrow \quad \uparrow$
特征1 特征2 特征3

- 对于权重矩阵： $\mathbf{W} = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \end{bmatrix}$ ，我们有：

$$\mathbf{W} = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \end{bmatrix} \begin{matrix} \leftarrow \text{权重1} \\ \leftarrow \text{权重2} \\ \leftarrow \text{权重3} \end{matrix}$$

$\uparrow \quad \uparrow$
神经元1 神经元2

- 对于输出矩阵： $\mathbf{Y}_{\text{pre}} = \mathbf{X} \otimes \mathbf{W}$ ，我们有：

$$\mathbf{Y}_{\text{pre}} = \begin{bmatrix} y_{\text{pre1}} & y_{\text{pre2}} \end{bmatrix} \leftarrow \text{预测样本1}$$

$\uparrow \quad \uparrow$
类别1 类别2

- 对于目标值： $\mathbf{Y}$ ，我们有：

$$\mathbf{Y} = [y_1 \quad y_2]$$

- 其中 $y_{\text{pre1}} = x_1 w_{11} + x_2 w_{21} + x_3 w_{31}$ ； $y_{\text{pre2}} = x_1 w_{12} + x_2 w_{22} + x_3 w_{32}$
- 这意味着，在处理方式上：我们的数据 $\mathbf{X}$ 需要分别与两个神经元（神经元1，神经元2）向量做内积，这并不复杂，只是第二节的扩展而已。
- 这还意味着，从结果上看：之前我们一直游荡在回归（用一个变量预测另一个变量的线性方程）中，这是我们第一次进入了分类的领域！

分类与回归：

1. 分类与回归定义上的绝对界限

举例：预测房价（100.5万）、明天气温（23.6°C）、销售额（15000元）。

输出特性：数值之间具有大小和距离关系。30°C比25°C高5°C，这个差值有意义

举例：判断邮件是“垃圾”还是“正常”、识别图片是“猫”、“狗”还是“鸟”。

输出特性：类别之间没有数值大小关系。“猫”（可能被编码为0）和“狗”（编码为1）之间，不能说“狗比猫大1”，这里的0和1只是代号。

- 回归：预测连续数值，数值间有大小和距离意义（如房价）。
- 分类：预测离散标签，标签间无数值大小关系（如猫/狗）。

2. 神经网络中界限模糊的核心原因

这是最关键的一点。现代的神经网络分类器，除了最后一层，其内部99%的计算和回归网络别无二致。以经典的图像分类网络为例：

特征提取：卷积层等结构从像素中提取特征，这个过程是通用的。

全连接层：倒数第二层输出一个连续值的向量，比如 [-1.2, 3.5, 0.8]。此时，这还是一组纯粹的、连续的回归值。

输出层：用 Softmax 函数把这组连续值“压缩”成和为1的[0.02, 0.95, 0.03]。

- 本质统一：分类网络的实质是对“属于某类的概率”这一连续量进行回归。输出层之前的所有计算，与回归网络无异。
 - 结构通用：只需更换最后的输出层和损失函数，同一网络主体可在分类与回归任务间轻易切换。
 - 任务混合：现实中存在大量连续与离散交错的场景（如有序回归、物体检测同时需要分类和回归），天然地模糊了界限。
- 问题随之而来： $loss$ 是个标量，而 $\mathbf{Y}_{pre}$ 却不再是一个标量，我们无法令 $(\mathbf{Y}_{pre} - \mathbf{Y})^2$ 得到一个标量。
 - 接下来，我们要加入一个新的步骤，让上述 $(\mathbf{Y}_{pre} - \mathbf{Y})^2$ 转化为标量！

5.2. 均方误差 (MSE)

- 令 $\mathbf{OUT} = \mathbf{Y}_{pre} - \mathbf{Y}$ ，我们不仅将 $\mathbf{OUT}$ 中每一个的元素开平方，而且还取均值得到 $loss$ （我们用 L 来代替）：

$$L = \frac{(y_{pre1} - y_1)^2 + (y_{pre2} - y_2)^2}{2}$$

因为 L 是关于 $y_{pre1} - y_1$ ， $y_{pre2} - y_2$ 的函数，所以， L 为极小值时， $y_{pre1} - y_1$ ， $y_{pre2} - y_2$ 也各自到达各自的极小值。

5.3. 反向求导

同样是四步连招！

第一步，我们先固定好我们的目标，是求 $\nabla_{\mathbf{W}} L$ 。拆解这个目标我们发现：求 $\nabla_{\mathbf{W}} L$ ，就是求 L 对 $\mathbf{W}$ 中每一个元素的导数（你是否在这里已经隐约感觉到了 $\nabla_{\mathbf{W}} L$ 与 $\mathbf{W}$ 同形）。

$$\nabla_{\mathbf{W}} L = \begin{bmatrix} \frac{\partial L}{\partial w_{11}} & \frac{\partial L}{\partial w_{12}} \\ \frac{\partial L}{\partial w_{21}} & \frac{\partial L}{\partial w_{22}} \\ \frac{\partial L}{\partial w_{31}} & \frac{\partial L}{\partial w_{32}} \end{bmatrix}$$

• 我们利用链式法则： $\nabla_{\mathbf{W}} L = \frac{\partial L}{\partial \mathbf{Y}_{pre}} \cdot \frac{\partial \mathbf{Y}_{pre}}{\partial \mathbf{W}}$ ，顺着 L 一步步往回推：

第二步是求 $\nabla_{\mathbf{Y}_{pre}} L$ ， $\nabla_{\mathbf{Y}_{pre}} L = \begin{bmatrix} \frac{\partial L}{\partial y_{pre1}} & \frac{\partial L}{\partial y_{pre2}} \end{bmatrix}$ （误差梯度）。也就是分别求 $\frac{\partial L}{\partial y_{pre1}}$ 和 $\frac{\partial L}{\partial y_{pre2}}$ ，

因为

$$\frac{(y_{pre1} - y_1)^2}{2} = \frac{y_{pre1}^2 + y_1^2 - 2y_{pre1}y_1}{2} ; \quad \frac{(y_{pre2} - y_2)^2}{2} = \frac{y_{pre2}^2 + y_2^2 - 2y_{pre2}y_2}{2}$$

所以

$$\frac{\partial L}{\partial y_{pre1}} = y_{pre1} - y_1 \text{ 和 } \frac{\partial L}{\partial y_{pre2}} = y_{pre2} - y_2$$

张量形式：

$$\frac{\partial L}{\partial \mathbf{Y}_{pre}} = \nabla_{\mathbf{Y}_{pre}} L = \begin{bmatrix} \frac{\partial L}{\partial y_{pre1}} & \frac{\partial L}{\partial y_{pre2}} \end{bmatrix} = [y_{pre1} - y_1 \quad y_{pre2} - y_2]$$

我们有： $y_{pre1} = x_1 w_{11} + x_2 w_{21} + x_3 w_{31}$ ； $y_{pre2} = x_1 w_{12} + x_2 w_{22} + x_3 w_{32}$ 。

第三步是求 $\nabla_{\mathbf{W}} \mathbf{Y}_{pre}$ 。求 $\frac{\partial \mathbf{Y}_{pre}}{\partial \mathbf{W}}$ ，这里需要了解一下Jacobian矩阵（张量对张量的梯度矩阵）：

函数输出（分子）保持原状，此为分子布局： $\mathbf{Y}_{pre} = [y_{pre1} \quad y_{pre2}]$

向量化分母变量，把 $\mathbf{W}$ 拉伸为列向量： $\text{vec}(\mathbf{W}) = \begin{bmatrix} w_{11} \\ w_{21} \\ w_{31} \\ w_{12} \\ w_{22} \\ w_{32} \end{bmatrix}$

求 y_{pre1} 对 $\text{vec}(\mathbf{W})$ 的偏导（即Jacobian第一行，梯度在“分子布局”约定下默认写成行向量）：

$$\frac{\partial y_{pre1}}{\partial \text{vec}(\mathbf{W})} = [x_1 \quad x_2 \quad x_3 \quad 0 \quad 0 \quad 0]$$

求 y_{pre2} 对 $\text{vec}(\mathbf{W})$ 的偏导（即Jacobian第二行）：

$$\frac{\partial y_{pre2}}{\partial \text{vec}(\mathbf{W})} = [0 \quad 0 \quad 0 \quad x_1 \quad x_2 \quad x_3]$$

Jacobian矩阵就是 $\mathbf{Y}_{pre}$ 中的每一个元素对 $\mathbf{W}$ 中每一个元素求偏微分。在分子布局中， $\mathbf{Y}_{pre}$ 中每个元素产生的偏导向量组合起来得到最后结果：

$$\frac{\partial \mathbf{Y}_{pre}}{\partial \mathbf{W}} = \begin{bmatrix} x_1 & x_2 & x_3 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_1 & x_2 & x_3 \end{bmatrix}$$

第四步是计算 $\frac{\partial L}{\partial \mathbf{Y}_{pre}} \cdot \frac{\partial \mathbf{Y}_{pre}}{\partial \mathbf{W}}$ ：

$$\frac{\partial L}{\partial \mathbf{Y}_{pre}} \cdot \frac{\partial \mathbf{Y}_{pre}}{\partial \mathbf{W}} = [y_{pre1} - y_1 \quad y_{pre2} - y_2] \cdot \begin{bmatrix} x_1 & x_2 & x_3 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_1 & x_2 & x_3 \end{bmatrix}$$

$$= \begin{bmatrix} x_1(y_{pre1} - y_1) & x_2(y_{pre1} - y_1) & x_3(y_{pre1} - y_1) & x_1(y_{pre2} - y_2) & x_2(y_{pre2} - y_2) & x_3(y_{pre2} - y_2) \end{bmatrix}$$

我们计算得到的 1×6 梯度向量，在将其重新填充回与权重 $\mathbf{W}$ 一致的 3×2 矩阵形状后，得到了：

$$\frac{\partial L}{\partial \mathbf{Y}_{pre}} \cdot \frac{\partial \mathbf{Y}_{pre}}{\partial \mathbf{W}} = \begin{bmatrix} x_1(y_{pre1} - y_1) & x_1(y_{pre2} - y_2) \\ x_2(y_{pre1} - y_1) & x_2(y_{pre2} - y_2) \\ x_3(y_{pre1} - y_1) & x_3(y_{pre2} - y_2) \end{bmatrix}$$

经过整理，我们发现：

$$\frac{\partial L}{\partial \mathbf{Y}_{pre}} \cdot \frac{\partial \mathbf{Y}_{pre}}{\partial \mathbf{W}} = \begin{bmatrix} x_1(y_{pre1} - y_1) & x_1(y_{pre2} - y_2) \\ x_2(y_{pre1} - y_1) & x_2(y_{pre2} - y_2) \\ x_3(y_{pre1} - y_1) & x_3(y_{pre2} - y_2) \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \cdot [y_{pre1} - y_1 \quad y_{pre2} - y_2] =$$

$$\mathbf{X}^T \cdot \frac{\partial L}{\partial \mathbf{Y}_{pre}} = \mathbf{X}^T \cdot \nabla_{\mathbf{Y}_{pre}} L$$

看这里，能否回答上一个彩蛋里的问题了？

雅可比矩阵是指： $\mathbf{J} = \frac{\partial \mathbf{Y}_{pre}}{\partial \mathbf{W}}$

- 它描述的是：输出 $\mathbf{Y}_{pre}$ 对参数 $\mathbf{W}$ 的局部线性映射关系。
- 因为 $\mathbf{Y}_{pre} = \mathbf{XW}$ ，它的雅可比矩阵本身是一个高维张量，直接显式写出会非常大，在反向传播中通常不会直接构造。

VJP (Vector-Jacobian Product, 向量-雅可比积) 是指：

$$\mathbf{v} \cdot \mathbf{J} = \frac{\partial L}{\partial \mathbf{Y}_{pre}} \cdot \frac{\partial \mathbf{Y}_{pre}}{\partial \mathbf{W}} = \mathbf{X}^T \cdot \nabla_{\mathbf{Y}_{pre}} L$$

- 这里的 $\mathbf{v} = \frac{\partial L}{\partial \mathbf{Y}_{pre}}$ 是上游传来的梯度向量。
- VJP的本质是「上游梯度向量」和「雅可比矩阵」的点积，对应反向传播中“把梯度传回到参数”的操作。
- 你这里的例子中，因为 $\mathbf{Y}_{pre} = \mathbf{XW}$ ，所以它的转置雅可比矩阵作用在向量上的结果，恰好可以用矩阵乘法 $\mathbf{X}^T \cdot \mathbf{v}$ 高效实现，而不需要构造庞大的雅可比矩阵本身。

5.4. 思考

- VJP 的本质是「上游梯度向量」和「雅可比矩阵」的点积，对应反向传播中“把梯度传回到参数”的操作。
- 这里我们可以理解为，用转置的雅可比矩阵来分配上游梯度。

5.5. 从全微分角度再理解一次

- 第一步：写出顶层 L 的全微分

因为标量 L 是关于 y_{pre1} 和 y_{pre2} 的函数，根据全微分定义，它的总变动量 dL 为：

$$dL = \frac{\partial L}{\partial y_{pre1}} dy_{pre1} + \frac{\partial L}{\partial y_{pre2}} dy_{pre2}$$

写成向量形式，就是**梯度（行向量）点积微分（列向量）**：

$$dL = \nabla_{\mathbf{Y}_{pre}} L \cdot d\mathbf{Y}_{pre} = \nabla_{\mathbf{Y}_{pre}} L \cdot \begin{bmatrix} dy_{pre1} \\ dy_{pre2} \end{bmatrix} = \begin{bmatrix} \frac{\partial L}{\partial y_{pre1}} & \frac{\partial L}{\partial y_{pre2}} \end{bmatrix} \cdot \begin{bmatrix} dy_{pre1} \\ dy_{pre2} \end{bmatrix}$$

- 第二步：写出底层 y 的全微分

由于 $y_{pre1} = x_1 w_{11} + x_2 w_{21} + x_3 w_{31}$ ，输入 $\mathbf{X}$ 是常数，只有权重 $\mathbf{W}$ 在变。所以 y_{pre1} 的微小变动只能由 w 的微分引起：

$$dy_{pre1} = \frac{\partial L}{\partial w_{11}} \cdot dw_{11} + \frac{\partial L}{\partial w_{21}} \cdot dw_{21} + \frac{\partial L}{\partial w_{31}} \cdot dw_{31} = x_1 \cdot dw_{11} + x_2 \cdot dw_{21} + x_3 \cdot dw_{31}$$

同理：

$$dy_{pre2} = \frac{\partial L}{\partial w_{12}} \cdot dw_{12} + \frac{\partial L}{\partial w_{22}} \cdot dw_{22} + \frac{\partial L}{\partial w_{32}} \cdot dw_{32} = x_1 \cdot dw_{12} + x_2 \cdot dw_{22} + x_3 \cdot dw_{32}$$

- 第三步：把第二步代回第一步（连连看）

现在，我们把 dy_{pre1} 和 dy_{pre2} 的展开式，塞回到 dL 的标准公式里：

$$dL = \frac{\partial L}{\partial y_{pre1}} \overbrace{(x_1 dw_{11} + x_2 dw_{21} + x_3 dw_{31})}^{dy_{pre1}} + \frac{\partial L}{\partial y_{pre2}} \overbrace{(x_1 dw_{12} + x_2 dw_{22} + x_3 dw_{32})}^{dy_{pre2}}$$

我们把括号拆开，重新按 dw 归类项：

$$dL = \left(x_1 \frac{\partial L}{\partial y_{pre1}} \right) dw_{11} + \left(x_2 \frac{\partial L}{\partial y_{pre1}} \right) dw_{21} + \left(x_3 \frac{\partial L}{\partial y_{pre1}} \right) dw_{31} + \left(x_1 \frac{\partial L}{\partial y_{pre2}} \right) dw_{12} + \left(x_2 \frac{\partial L}{\partial y_{pre2}} \right) dw_{22} + \left(x_3 \frac{\partial L}{\partial y_{pre2}} \right) dw_{32}$$

- 第四步：见证奇迹的时刻根据全微分的终极定义， L 对 $\mathbf{W}$ 整体的全微分必须满足：

$$dL = \sum \frac{\partial L}{\partial w_{ij}} dw_{ij}$$

对照第三步归类后的各项系数，每一项 dw_{ij} 前面的那团括号，就是 L 对该 w_{ij} 的偏导数！也就是说：

$$\frac{\partial L}{\partial w_{11}} = x_1 \frac{\partial L}{\partial y_{pre1}}, \quad \frac{\partial L}{\partial w_{21}} = x_2 \frac{\partial L}{\partial y_{pre1}}, \quad \dots$$

把这些独立出来的偏导数，重新拼回与 $\mathbf{W}$ 同形的 3×2 矩阵中：

$$\nabla_{\mathbf{W}} L = \begin{bmatrix} x_1 \frac{\partial L}{\partial y_{pre1}} & x_1 \frac{\partial L}{\partial y_{pre2}} \\ x_2 \frac{\partial L}{\partial y_{pre1}} & x_2 \frac{\partial L}{\partial y_{pre2}} \\ x_3 \frac{\partial L}{\partial y_{pre1}} & x_3 \frac{\partial L}{\partial y_{pre2}} \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \cdot \begin{bmatrix} \frac{\partial L}{\partial y_{pre1}} & \frac{\partial L}{\partial y_{pre2}} \end{bmatrix} = \mathbf{X}^T \cdot \nabla_{\mathbf{Y}_{pre}} L$$

$$\nabla_{\mathbf{W}} L = \mathbf{X}^T \cdot \nabla_{\mathbf{Y}_{pre}} L$$

5.6. python

```
1  import torch
2
3  # =====
4  # 5.6 PyTorch 验证：单样本-多输出的前向与反向传播
5  # 核心公式： $\nabla_W L = X^T @ \nabla_{\{Y_{pre}\}} L$ 
6  # =====
7
8  # --- 1. 数据定义 ---
9  X = torch.tensor([[2.0, -1.0, 3.0]]) # 单样本, 3 个特征 (1, 3)
10 Y = torch.tensor([[1.0, 0.0]]) # 目标值, 2 个输出 (1, 2)
11
12 # --- 2. 初始化权重 (需要梯度) ---
13 torch.manual_seed(42)
14 W = torch.randn(3, 2, requires_grad=True) # 权重矩阵 (3, 2)
15
16 # --- 3. 前向传播 ---
17 Y_pre = X @ W # 预测输出 (1, 2)
18
19 # --- 4. 计算损失 (MSE = sum(squared) / 2, 与讲义一致) ---
20 L = torch.sum((Y_pre - Y) ** 2) / 2 # 标量损失
21
22 # --- 5. 手动解析梯度 ---
23 grad_Y_pre = Y_pre - Y #  $\partial L / \partial Y_{pre}$  (1, 2)
24 grad_W_analytic = X.T @ grad_Y_pre #  $\nabla_W L = X^T @ \text{grad\_Y\_pre}$  (3, 2)
25
26 # --- 6. 自动求导获取梯度 ---
27 L.backward() # 反向传播, 得到 W.grad
28
29 # --- 7. 结果对比 ---
30 print("输入 X (1,3):\n", X)
31 print("权重 W (3,2):\n", W)
32 print("预测 Y_pre (1,2):\n", Y_pre)
33 print("目标 Y (1,2):\n", Y)
34 print(f"MSE 损失: {L.item():.6f}")
35
36 print("\n解析梯度  $\nabla_W L (X^T @ \text{grad\_Y\_pre})$ :\n", grad_W_analytic)
37 print("PyTorch 自动梯度 W.grad:\n", W.grad)
38 print("\n两者之差 (应全部接近 0) :\n", grad_W_analytic - W.grad)
```

6. 依靠"向量-Jacobian乘积", 绕过Jacobian Matrix!

我们刚刚经历一场酣畅淋漓的战役, 我们且停下来, 修整一番。

当你轻拭手中的宝剑时, 惊讶的发现剑身已经出现了轻微的裂痕。战后复盘, 你发现在对战Jacobian矩阵中, 显示构筑了一个 2×6 的“巨大”矩阵, 这把利剑在应对这庞然大物时险些招架不住。

好在, 山穷水尽疑无路, 柳暗花明又一村。线性代数里早已写好了破局之道。

如果修整完毕, 且看我们抖擞精神再出发!

6.1. 繁杂的Jacobian Matrix

- 在上述例子中, 我们计算 Y_{pre} 对 W 的梯度时, 仅仅一个 1×2 矩阵对 3×2 矩阵求导, 就显示构筑了一个 2×6 的“巨大”矩阵:

$$\frac{\partial \mathbf{Y}_{pre}}{\partial \mathbf{W}} = \begin{bmatrix} x_1 & x_2 & x_3 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_1 & x_2 & x_3 \end{bmatrix}$$

- 在现代神经网络中，假设单层全连接：

$$\mathbf{z} = \mathbf{W}\mathbf{x}, \quad \mathbf{W} \in \mathbb{R}^{m \times n}, \quad \mathbf{x} \in \mathbb{R}^{n \times 1}, \quad \mathbf{z} \in \mathbb{R}^{m \times 1}, \quad \text{其中 } \mathbf{W} \in \mathbb{R}^{256 \times 784}$$

雅可比矩阵的尺寸灾难。输出 $\mathbf{z}$ 是 m 维向量，权重 $\mathbf{W}$ 是 mn 矩阵（共 $m \times n$ 个标量参数）。

把 $\mathbf{W}$ 展平为一维向量 $\mathbf{w} \in \mathbb{R}^{mn}$ ，则 Jacobian 矩阵：

$$\mathbf{J} = \frac{\partial \mathbf{z}}{\partial \mathbf{w}} \in \mathbb{R}^{m \times mn}$$

参数总数：

$$mn = 256 \times 784 = 200704$$

雅可比形状：

$$256 \times 200704$$

元素总量：**51380224**。

单精度浮点数就要占约 200MB 内存。

如果多层网络、批量输入（batch），Jacobian 尺寸会爆炸式增长，绝对不能显式存储 / 相乘。

6.2. 现代神经网络反向传播的做法

- 深度学习从一开始就不用完整雅可比，依靠迹技巧，直接推导梯度表达式，全程只做小规模矩阵运算。

在开始之前，我们先补充/回顾一点数学知识：

- 矩阵的 Frobenius 内积，本质就是两个矩阵同位置元素两两相乘，再全部加起来。对任意两个同尺寸实矩阵 $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m \times n}$ ：

$$\langle \mathbf{A}, \mathbf{B} \rangle_F = \sum_{i=1}^m \sum_{j=1}^n a_{ij} b_{ij}$$

就是逐元素相乘、再求和。在标量对张量的求导中，标量对张量中每一个元素的导数*对应每一个元素的微分，再求和，便是标量对张量的全微分。

- Frobenius 内积可以用迹技巧运算： $\langle \mathbf{A}, \mathbf{B} \rangle_F = \text{tr}(\mathbf{A}\mathbf{B}^T)$
 - L 对 $\mathbf{W}$ 的全微分： $dL = \frac{\partial L}{\partial \mathbf{W}} \cdot d\mathbf{W}$ 。因为 $\frac{\partial L}{\partial \mathbf{W}}$ 和 $d\mathbf{W}$ 同尺寸，所以
- $$dL = \left\langle \frac{\partial L}{\partial \mathbf{W}}, d\mathbf{W} \right\rangle_F$$

以上述例子为例：

- 第一步，明确目标求 $\nabla_{\mathbf{W}} L$ ，也就是 $\frac{\partial L}{\partial \mathbf{W}}$ 。
- 第二步，用微分的形式表达(使用迹技巧变形)：

$$dL = \left\langle \frac{\partial L}{\partial \mathbf{W}}, d\mathbf{W} \right\rangle_F = \text{tr} \left(\frac{\partial L}{\partial \mathbf{W}} \cdot d\mathbf{W}^T \right) = \text{tr} (d\mathbf{W} \cdot \left(\frac{\partial L}{\partial \mathbf{W}} \right)^T) = \text{tr} \left(\left(\frac{\partial L}{\partial \mathbf{W}} \right)^T \cdot d\mathbf{W} \right) \quad (1)$$

同理，还有：

$$dL = \left\langle \frac{\partial L}{\partial \mathbf{Y}_{\text{pre}}}, d\mathbf{Y}_{\text{pre}} \right\rangle_F = \text{tr} \left(\frac{\partial L}{\partial \mathbf{Y}_{\text{pre}}} \cdot d\mathbf{Y}_{\text{pre}}^T \right) \quad (2)$$

注：由于 $\frac{\partial L}{\partial \mathbf{Y}_{\text{pre}}}$ 是 1×2 的行向量， $d\mathbf{Y}_{\text{pre}}^T$ 是 2×1 的列向量，两者相乘刚好是一个 1×1 的标量。而标量的迹（Trace）就等于它本身，这就完美吻合了左边的标量 dL 。

以及中间层的全微分公式：

$$d\mathbf{Y}_{\text{pre}} = \mathbf{X} \cdot d\mathbf{W} \quad (3)$$

• 第三步,将 (3) 代入 (2)， $(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T$ ：

$$dL = \left\langle \frac{\partial L}{\partial \mathbf{Y}_{\text{pre}}}, d\mathbf{Y}_{\text{pre}} \right\rangle_F = \text{tr} \left(\frac{\partial L}{\partial \mathbf{Y}_{\text{pre}}} \cdot d\mathbf{Y}_{\text{pre}}^T \right) = \text{tr} \left(\frac{\partial L}{\partial \mathbf{Y}_{\text{pre}}} \cdot d\mathbf{W}^T \cdot \mathbf{X}^T \right) \quad (4)$$

• 第四步，使用迹技巧对 (4) 移形换影， $\text{tr}(\mathbf{M}) = \text{tr}(\mathbf{M}^T)$ ， $\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA})$ (循环置换)：

$$dL = \text{tr} \left(\frac{\partial L}{\partial \mathbf{Y}_{\text{pre}}} \cdot d\mathbf{W}^T \cdot \mathbf{X}^T \right) = \text{tr} \left(\left(\frac{\partial L}{\partial \mathbf{Y}_{\text{pre}}} \cdot d\mathbf{W}^T \cdot \mathbf{X}^T \right)^T \right) = \text{tr} \left(\mathbf{X} \cdot d\mathbf{W} \cdot \left(\frac{\partial L}{\partial \mathbf{Y}_{\text{pre}}} \right)^T \right) \quad (5)$$

$$\text{循环置换：} dL = \text{tr} \left(\mathbf{X} \cdot d\mathbf{W} \cdot \left(\frac{\partial L}{\partial \mathbf{Y}_{\text{pre}}} \right)^T \right) = \text{tr} \left(\left(\frac{\partial L}{\partial \mathbf{Y}_{\text{pre}}} \right)^T \cdot \mathbf{X} \cdot d\mathbf{W} \right) \quad (6)$$

• 第五步，对比 (1) 和 (6)，我们发现：

$$\left(\frac{\partial L}{\partial \mathbf{W}} \right)^T \cdot d\mathbf{W} = \left(\frac{\partial L}{\partial \mathbf{Y}_{\text{pre}}} \right)^T \cdot \mathbf{X} \cdot d\mathbf{W}$$

$$\left(\frac{\partial L}{\partial \mathbf{W}} \right)^T = \left(\frac{\partial L}{\partial \mathbf{Y}_{\text{pre}}} \right)^T \cdot \mathbf{X}$$

$$\frac{\partial L}{\partial \mathbf{W}} = \mathbf{X}^T \cdot \frac{\partial L}{\partial \mathbf{Y}_{\text{pre}}}$$

• 打完收工！

仅需计算 $\mathbf{X}^T$ 和 $\frac{\partial L}{\partial \mathbf{Y}_{\text{pre}}}$ （上游传来的梯度），即可得到 $\frac{\partial L}{\partial \mathbf{W}}$ ，避免了显示构筑 Jacobian 矩阵。如果存在多层神经网络，每层都可以以此方式传递下去。

希望通过本章，你可以看到另一种人生哲学：那些看似不可逾越的高山，可能侧面有一条高速公路。人生往往不是死磕到底，学会绕路，才会柳暗花明。

雄关漫道真如铁，而今迈步从头越。Jacobian 矩阵也不过如此~

7. 从权重的损失景观引发的思考

朋友们，我们从“单样本-多输出”的例子中三进三出，窥见了多个神经元协作下反向传播的数学基础。

但冷冰冰干巴巴的数字令人倍感无趣，接下来，我们放慢脚步，进入激动人心的图像世界，这可以帮助你从前几次战役建立直观的印象。

请记得，在本次关卡中，除了感受这光怪陆离的现象之外，背后的原因还需要我们多加思考！

话不多说，来直观地感受梯度下降之美！

7.1. “一个样本-一个神经元”模型

为了形象的向你展示权重在更新过程是如何滑落到底部的，我们将一个权重矩阵压缩为作为空间中一个点，再将空间中所有点连接，绘制为图像，这样，在权重更新过程中，你会直观的看到更新过程。

不幸的是，人类对空间的想象被限制在三维，在这仅有的可怜的3个维度中，我们还要抽出一个 z 轴作为 $loss$ 值。

所以我们设计的权重矩阵只能有两个维度： $\mathbf{W} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}$ 。将 w_1 作为 x 轴，将 w_2 作为 y 轴。

此时，我们需要固定一个输入和输出，令 $\mathbf{X} = [0.3 \quad 0.7]$ ， $Y = 0.4$ 。

而 $Y_{\text{pre}} = w_1 x_1 + w_2 x_2$ ， $L = (Y_{\text{pre}} - Y)^2$ 。

7.2. 构建欧几里得空间

很好，我们已经将模型所需的所有工具备好。

接下来就是让 w_1 在 $[-5, 5]$ 之间取 1000 个点，让 w_2 也在 $[-5, 5]$ 之间取 1000 个点。这会组合成 $1000 \times 1000 = 1000000$ 个坐标点。

代码中为了算力考量，我们用 1000 个点均匀撒网，但在数学扫描中可以无限稠密。

把这 1000000 组 (w_1, w_2) 挨个代入公式算出 Y_{pre} ，再算出 $L = (Y_{\text{pre}} - Y)^2$ 。

7.3. 解空间的退化 (Degeneracy)

我们用matplotlib画出这幅图像：

```

1  import torch
2  import numpy as np
3  import matplotlib.pyplot as plt
4  from matplotlib import cm
5
6  # =====
7  # 1. 固定数据
8  # =====
9  X = torch.tensor([0.3, 0.7]) # x1 = 0.3, x2 = 0.7
10 y_target = torch.tensor(0.4) # 真实标签 y = 0.4
11
12 # =====
13 # 2. 网格撒网 (扫描 [-5, 5] 的平直空间)
14 # =====
15 w1_range = torch.linspace(-5, 5, 1000)
16 w2_range = torch.linspace(-5, 5, 1000)
17 W1, W2 = torch.meshgrid(w1_range, w2_range, indexing='ij')
18
19 # =====
20 # 3. 计算 Z 轴 Loss 矩阵
21 # =====
22 Y_pre = W1 * X[0] + W2 * X[1]
23 Loss = (Y_pre - y_target) ** 2
24 # 设定一个高度阈值, 比如只保留 Loss 小于 6 的部分, 超过 6 的尖角直接丢弃
25 max_loss_threshold = 6.0
26 Loss[Loss > max_loss_threshold] = float('nan') # 强行转为 NaN, 使其在3D图中隐形
27
28 # =====
29 # 4. 【核心创新】精确计算谷底直线的坐标点
30 # =====
31 # 根据方程:  $0.3*w1 + 0.7*w2 = 0.4 \rightarrow w2 = (0.4 - 0.3*w1) / 0.7$ 
32 # 我们在 w1 的扫描范围内 (-5 到 5), 精准算出对应的 w2 轨迹
33 w1_line = np.linspace(-5, 5, 100)
34 w2_line = (y_target.item() - X[0].item() * w1_line) / X[1].item()
35 # 因为是完美最优解, 这条线上所有点的 Loss 理论上都为 0
36 z_line = np.zeros_like(w1_line)
37
38 # =====
39 # 5. 渲染 3D 图形
40 # =====
41 w1_np = W1.numpy()
42 w2_np = W2.numpy()
43 loss_np = Loss.numpy()
44
45 fig = plt.figure(figsize=(14, 9))
46 ax = fig.add_subplot(111, projection='3d')
47
48 # 绘制滑雪水槽一样的 3D 损失曲面
49 surf = ax.plot_surface(w1_np, w2_np, loss_np, cmap=cm.coolwarm,
50                        linewidth=0, antialiased=True, alpha=0.6)
51
52 # 用一条极其耀眼的【明黄色加粗实线】把谷底直线画出来!
53 ax.plot(w1_line, w2_line, z_line, color='gold', linewidth=4,
54         zorder=20, label='Optimal Solution Line (Loss=0)')
55
56 # 顺便在直线上点两颗珍珠(极小值), 验证我们的推导:
57 # 珍珠A:  $(0, 0.4/0.7) \rightarrow$  约  $(0, 0.571)$ 
58 ax.scatter(0, 0.4/0.7, 0, color='lime', s=100, zorder=25, label='Solution A (0, 0.571)')
59 # 珍珠B:  $(0.4/0.3, 0) \rightarrow$  约  $(1.333, 0)$ 
60 ax.scatter(0.4/0.3, 0, 0, color='magenta', s=100, zorder=25, label='Solution B (1.333, 0)')
61
62 # 视觉优化
63 ax.set_title("The Valley of Loss: Canvas is Curved into a Ski-Run", fontsize=14, pad=20)
64 ax.set_xlabel("$w_1$ (Parameter X)", fontsize=12)
65 ax.set_ylabel("$w_2$ (Parameter Y)", fontsize=12)

```

```

66 ax.set_zlabel("Loss ($Z$-axis)", fontsize=12)
67 fig.colorbar(surf, shrink=0.5, aspect=10, label='Loss Value')
68
69 # 限制坐标轴范围, 防止画出的斜线飞到天上去导致画面失真
70 ax.set_xlim([-5, 5])
71 ax.set_ylim([-5, 5])
72 # 这样即使矩阵里有 NaN, 也能正常拿到剩余数据的最大值
73 valid_max_loss = np.nanmax(loss_np)
74 ax.set_zlim([0, valid_max_loss])
75
76 plt.legend(loc="upper left")
77 ax.view_init(elev=30, azim=140) # 调换一个好视角
78
79 plt.show()

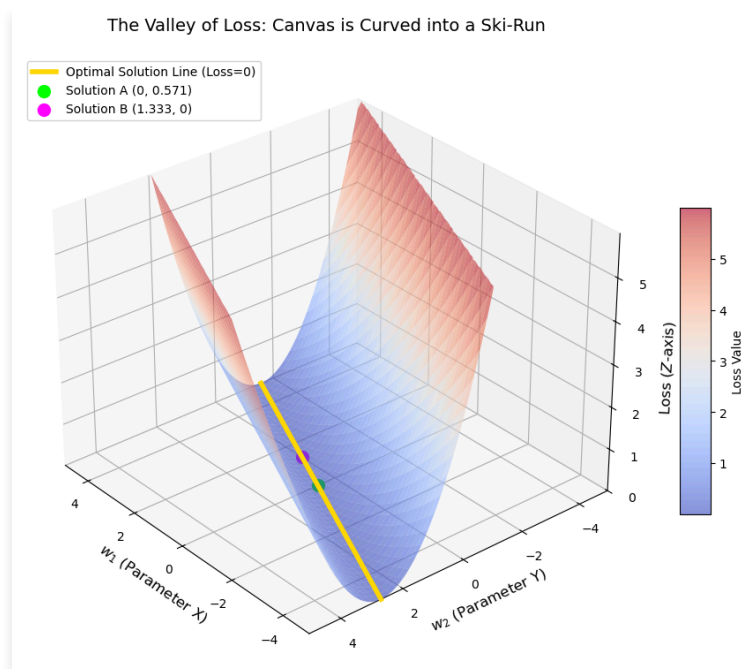
```

从图中我们可以看出, 整个曲面的最小值并非一个点, 而是一条直线。

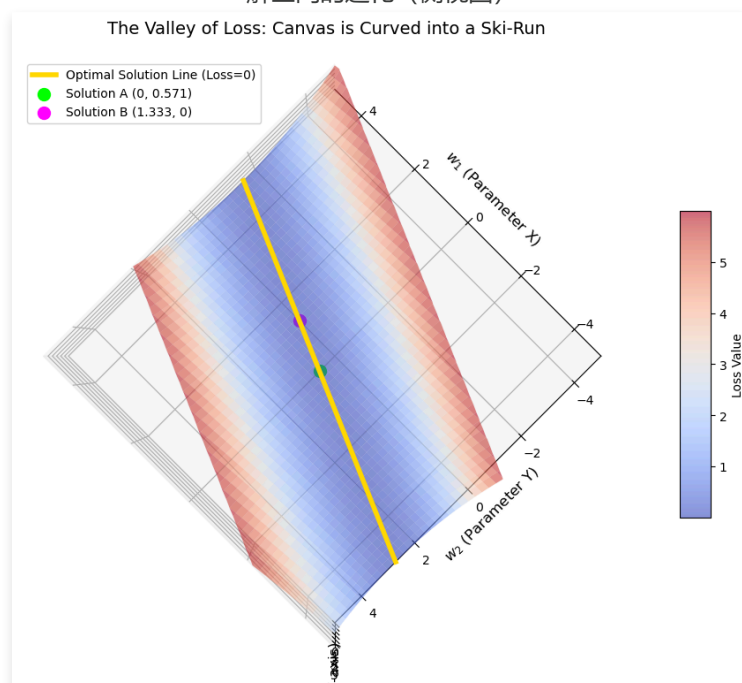
因为在上述简单的例子中, 我们的目标是让 $Y_{\text{pre}} = w_1x_1 + w_2x_2 = 0.4$ 。当模型达到完美状态 $L = (Y_{\text{pre}} - Y)^2 = 0$ 时, 必须满足:

$$0.3w_1 + 0.7w_2 = 0.4$$

这是一个二元一次方程! 在 $w_1 - w_2$ 的平直平面上, 它代表一条直线!



解空间的退化（侧视图）



解空间的退化（俯视图）

在数学和机器学习中，这被称为**解空间的退化（Degeneracy）**。因为我们只有 **1 条** 数据样本，却要解 **2 个** 未知数。数据量太少，根本无法锁定唯一的解，只能锁定一条“最优解直线”。

7.4. “两个样本-一个神经元” 模型

神挡杀神！我要打两个！

7.4.1. 再引入一个数据样本

为了解决**解空间的退化**问题我们再引入一个数据样本

$$\mathbf{X} = \begin{bmatrix} 0.3 & 0.7 \\ -0.4 & 0.5 \end{bmatrix}, \text{权重矩阵不变: } \mathbf{W} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}, \text{目标值矩阵为 } \mathbf{Y} = \begin{bmatrix} 0.4 \\ 0.6 \end{bmatrix}。$$

$$\mathbf{Y}_{\text{pre}} = \mathbf{X}\mathbf{W} = \begin{bmatrix} 0.3w_1 + 0.7w_2 \\ 0.4w_1 + 0.5w_2 \end{bmatrix}, L = \frac{(\mathbf{Y}_{\text{pre}} - \mathbf{Y})^2}{2}.$$

7.4.2. 碗状谷

运行如下代码：

```

1  import torch
2  import numpy as np
3  import matplotlib.pyplot as plt
4  from matplotlib import cm
5
6  # =====
7  # 1. 固定数据 (引入 Batch = 2 的矩阵)
8  # =====
9  X = torch.tensor([[0.3, 0.7],
10                  [-0.4, 0.5]])
11  Y_target = torch.tensor([[0.4],
12                          [0.6]])
13
14  # =====
15  # 2. 网格撒网 (扫描 [-5, 5] 的平直欧氏空间)
16  # =====
17  w1_range = torch.linspace(-5, 5, 1000)
18  w2_range = torch.linspace(-5, 5, 1000)
19  W1, W2 = torch.meshgrid(w1_range, w2_range, indexing='ij')
20
21  # =====
22  # 3. 计算 Z 轴 Loss 矩阵 (Batch 带来的叠加力量)
23  # =====
24  # 分别计算两条数据的预测值
25  Y_pre_1 = X[0, 0] * W1 + X[0, 1] * W2
26  Y_pre_2 = X[1, 0] * W1 + X[1, 1] * W2
27
28  # 根据截图公式  $L = (Y_{pre} - Y)^2 / 2$ 
29  # 这里的总 Loss 是两个样本误差平方之和除以 2
30  Loss = 0.5 * ((Y_pre_1 - Y_target[0, 0])**2 + (Y_pre_2 - Y_target[1, 0])**2)
31
32  # 【硬核切角操作】切掉高度过高的飞角, 这里阈值设为 1.5 就能得到极美的切面
33  max_loss_threshold = 1.5
34  Loss[Loss > max_loss_threshold] = float('nan')
35
36  # =====
37  # 4. 【核心解析】计算唯一的最优解
38  # =====
39  # 解方程组:  $0.3*w1 + 0.7*w2 = 0.4$  ;  $0.4*w1 + 0.5*w2 = 0.6$ 
40  # 得到唯一的 (w1, w2)
41  w1_opt = -22.0 / 43.0 # 约等于 -0.512
42  w2_opt = 34.0 / 43.0 # 约等于 0.791
43
44  # =====
45  # 5. 渲染 3D 图形
46  # =====
47  w1_np = W1.numpy()
48  w2_np = W2.numpy()
49  loss_np = Loss.numpy()
50
51  fig = plt.figure(figsize=(14, 9))
52  ax = fig.add_subplot(111, projection='3d')
53
54  # 绘制真正的“完美损失之碗”
55  surf = ax.plot_surface(w1_np, w2_np, loss_np, cmap=cm.coolwarm,
56                        linewidth=0, antialiased=True, alpha=0.8)
57
58  # 绘制那颗唯一的“绝世珍珠”——全局唯一最优解, 再也没有直线了!
59  ax.scatter(w1_opt, w2_opt, 0, color='lime', s=200, zorder=25, edgecolor='black',
60            label=f'Global Unique Minimum\n({w1_opt:.3f}, {w2_opt:.3f})')
61
62  # 视觉优化
63  ax.set_title("The True Bowl: Batch Size=2 Eliminates Degeneracy", fontsize=16, pad=20)
64  ax.set_xlabel("$w_1$ (Parameter X)", fontsize=12)
65  ax.set_ylabel("$w_2$ (Parameter Y)", fontsize=12)

```

```

66 ax.set_zlabel("Loss ($Z$-axis)", fontsize=12)
67 fig.colorbar(surf, shrink=0.5, aspect=10, label='Loss Value')
68
69 # 调整坐标轴和z轴高度限制
70 ax.set_xlim([-4, 5])
71 ax.set_ylim([-5, 5])
72 valid_max_loss = np.nanmax(loss_np)
73 ax.set_zlim([0, valid_max_loss])
74
75 plt.legend(loc="upper left")
76 ax.view_init(elev=35, azim=160) # 稍微提高俯视角，更清楚地看碗底
77
78 plt.show()

```

这样，我们就得到了一个碗状曲面。

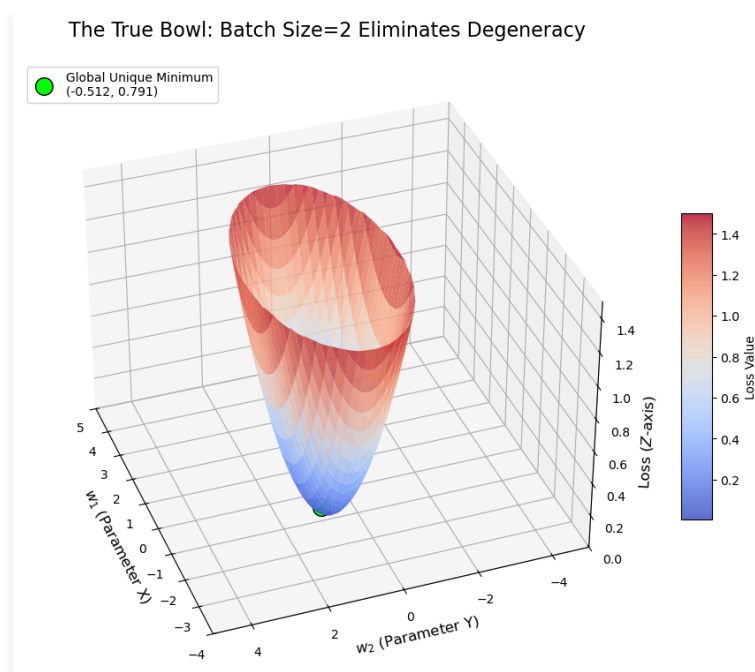
我们随机初始化一个权重矩阵 $W1$ ，这个矩阵可能落在碗状曲面的任意一个位置（极低的概率恰好出现在最低点）。

通过计算 $\nabla_{\mathbf{W}} L$ ，也就是该点的梯度，我们得到一个该点处的切线，这个切线代表了最陡方向。

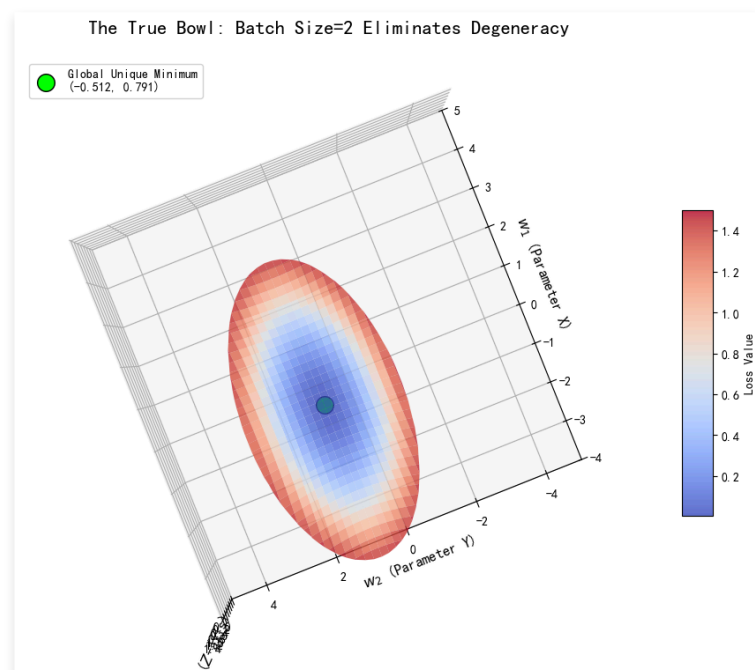
给这个梯度一个步长和一个负号，调整 $W1$ 的位置，得到 $W2$ 。

前向传播，反向传播，循环迭代，周而复始，循环 n 次之后， $loss$ 等于0或稳定在一个数值不再变化时，这个模型就完成了它的使命。

此时的 Wn 就是最低点，即训练后 $\mathbf{W} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}$ 中两个元素组成的坐标。



碗状谷（侧视图）



碗状谷（俯视图）

在现代大模型中，权重参数成百上千， $L(\mathbf{W})$ 构成一个 $n + 1$ 维的超曲面（这里的 n 表示矩阵中的元素数量），至于这个超曲面的高维形状，人类可能永远无法想象出来了，但是有一点我们可以大概确认：这个超曲面不会是上图中完美的碗状。

你可以想象这个曲面如同撒哈拉沙漠中的沙丘或秦岭高低起伏的山区，你站在沙丘或山上的一点，这个点代表了 n 维的矩阵的缩影，这个点是这个矩阵在二维平面上的投影，你所处的高度便是该矩阵对应的 $loss$ 值。

你前后左右移动，你脚下的点不断变化，这个点对应的高度也随之变化，你的终极目的是寻找整片沙漠/山区中最低的点，如同神经网络反向传播的终极目的是在寻找这个超曲面上最低的点。

看吧，有时候，你以为人生没有唯一答案，那是因为你见识的样本不够多！这又何尝不是一种人生智慧！

7.5. 过拟合与泛化

7.5.1. 单样本例子：极端的过拟合

在那个只有一条数据 $0.3w_1 + 0.7w_2 = 0.4$ 的例子里：

- **训练集表现：**完美。只要小球滚进黄金谷底直线上任何一个点（比如你选的点 A 或点 B）， $loss$ 都绝对等于 0。模型会骄傲地宣布：“我学会了！误差是 0！”
- **真实的危险：**此时，模型其实是通过作弊（走捷径）拿到的满分。因为未知数有两个（ w_1, w_2 ），它有无数种可以偷懒的组合来应付这唯一的一条数据。

当你拿测试集去考它时：

假设真实的宇宙规律（底层真理）其实是第二张图的 Batch 组合，也就是说真正完美的权重必须是 $w_1 \approx -0.512, w_2 \approx 0.791$ 。

但如果你第一次训练时，小球不小心滚到了黄金水槽的另一头，比如停在了坐标 $(5.0, -1.57)$ 。

- 这个坐标带入第一条数据，预测完全正确， $Loss = 0$ 。
- 但如果我们偷偷塞给它第二条测试数据 $x_1 = -0.4, x_2 = 0.5$ （正确标签 $y = 0.6$ ），用这个作弊学出来的权重去预测： $y_{pre} = -0.4 \cdot (5.0) + 0.5 \cdot (-1.57) = -2.0 - 0.785 = -2.785$
- **这离正确答案 0.6 差了十万八千里！测试 $Loss$ 直接爆炸！**

这就是过拟合的几何真相：**模型利用富余的参数维度，在损失景观里找到了一个“刚好能应付当前样本、但方向完全跑偏”的虚假谷底（退化峡谷）。**

7.5.2. 双样本例子：通过增加数据抑制过拟合

当你在矩阵 $(5.0, -1.57)$ 里加入第二条数据后，你亲眼看到那个长长的、可以到处乱滑的水槽**消失了**，坍塌成了一个唯一的、避无可避的圆碗中心。

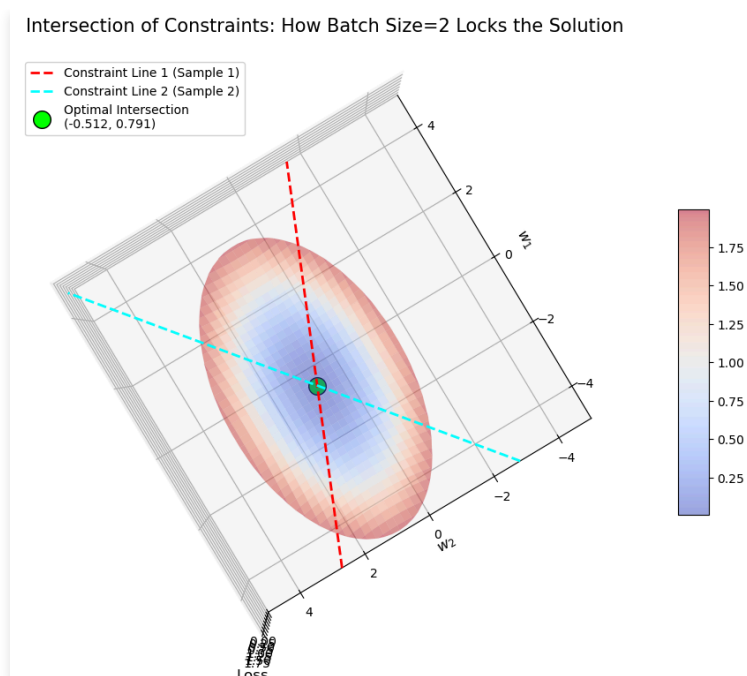
- 这时候，两条约束直线像剪刀一样，死死地把解卡在了 $(5.0, -1.57)$ 。
- 模型再也无法偷懒了，它必须拿出真本事，找到那个同时满足两个样本的“唯一真理”。
- 此时，模型在训练集上的解，就极大概率是宇宙的真实规律。当你再拿第三条、第四条测试数据去考它时，它依然能考出高分。这就是**泛化能力 (Generalization)**。

7.6. 由解空间的退化引发的思考（如下内容特指隐藏层）

7.6.1. 样本数量决定了什么？

一个样本只能提供一个线性约束： $0.3w_1 + 0.7w_2 = 0.4$ ；增加一个样本，相当于再增加一个线性约

束：
$$\begin{bmatrix} 0.3w_1 + 0.7w_2 \\ -0.4w_1 + 0.5w_2 \end{bmatrix} = \begin{bmatrix} 0.4 \\ 0.6 \end{bmatrix}$$
。两条相交的直线在平面上才能确定一个点。



两条直线相交确定最低点

同时，平行线不相交严格限制了两个样本的特征不可以是成比例的，这叫作：样本特征向量线性无关（列满秩）。

在一个线性层（无激活）中，若希望权重的解唯一确定（即损失函数有唯一全局最小值），通常需要至少 m 个线性无关的样本。

7.6.1.1. 样本数是决定模型性能的“天花板”

有一句流传很广的话：“数据和特征决定了机器学习的上限，而模型和算法只是逼近这个上限。”

对神经网络尤其如此。当样本量不够时，无论你怎么精调网络结构、损失函数、优化器，性能都会被数据的信息量锁死。**样本越多，理论上模型能达到的准确率、召回率等指标的上限就越高。**

7.6.1.2. 样本数决定泛化能力（过拟合 vs 欠拟合）

- 样本太少：网络极易死记硬背训练集，学到噪声和巧合，过拟合严重，在测试集上一塌糊涂。
- 样本充足：网络被迫捕捉数据背后的真实规律，泛化能力更强，对未见过的样本也能稳定预测。

深层网络尤其依赖大量样本：随着网络层数增加，可学习参数暴涨，必须用海量样本来约束它们，否则过拟合几乎必然发生。

7.6.2. 样本特征数量决定了什么？

如果说样本数量决定了模型“见多识广”的程度，特征数量 d 本质上决定了模型输入空间的维度，进而直接决定了每个神经元所必须处理的“原生信息复杂度”和“最低连接成本”。它直接影响以下几个方面：

7.6.2.1. 决定了每个神经元的“感受野”规模（突触数量）

因为矩阵的运算法则，所以：特征数量直接等于输入层的神经元权重个数。

- 无论第一隐藏层有多少个神经元 (n)，每一个神经元都必须伸出 d 条“突触”（权重）来接收全部输入特征。特征数 d 从10变成10000，意味着单个神经元要学习的权重也从10个暴涨到10000

个。这直接增加了单个神经元的“认知负荷”。

7.6.2.2. 决定“维度灾难”的严重程度

决定了“维度灾难”的触发门槛（根本原因）。

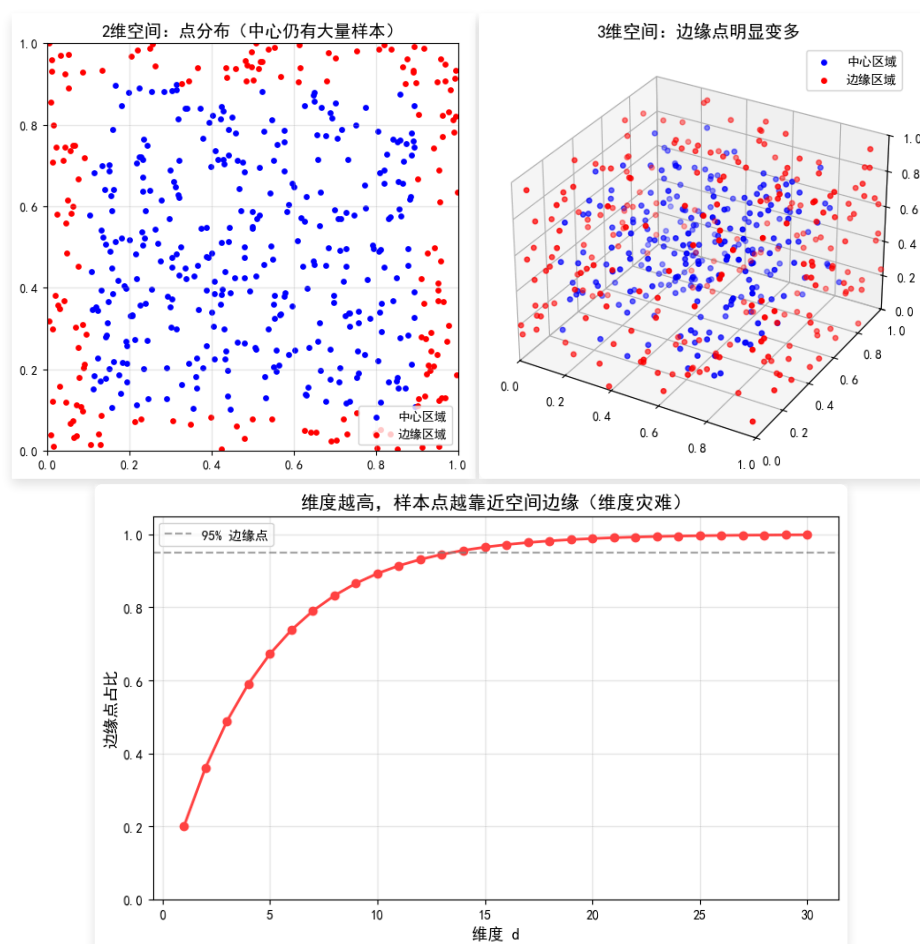
- 随着特征数量增加，数据空间体积呈指数增长，导致：在 d 维空间中，固定数量的样本会以指数速度变得稀疏。每个神经元看到的输入向量几乎都是“不相似”的，从稀疏点中学习稳定规律极为困难。
- 特征越多，为防止每个神经元的权重过拟合到噪声，所需的训练样本量就越大。这仍是“特征数 vs 样本数”的匹配问题，但视角落到了“每个神经元需要足够多的样本支撑其 d 个权重的学习”。

黄金法则：

特征数量与样本数量必须匹配，否则高维特征就是灾难。

行业通用经验法则：**样本数量 $\geq 10 \times$ 特征数量或 $\geq 50 \times$ 特征数量。**

维度灾难： 维度越高，绝大多数样本点会趋近于高维超立方体 / 超球体的边缘、表面，内部几乎没有数据。



彩蛋：这就是为什么在现代大模型中，我们绝对不会把成千上万维的原始稀疏特征直接喂给网络，而是要通过一层 **Embedding（嵌入层）**，强行把高维的稀疏空间压缩到一个紧凑、连续的低维稠密向量空间里。**Embedding 的本质，就是给维度灾难踩下一脚急刹车。**

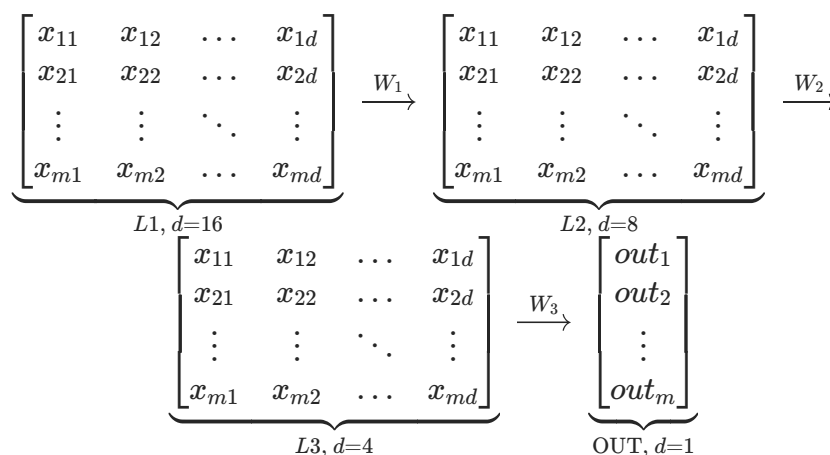
7.6.2.3. 决定了网络自动处理输入噪声的难度

- 对每个神经元而言，有意义的信号可能只分布在少数特征上。
- 特征数 d 越大，意味着每个神经元都要费力去“听”更多可能完全是噪声的输入，并精准地将无关输入的权重学到接近零。这个“权值稀疏化”过程在 d 很大时，需要大量数据和精细的正则化（如 L1）才能完成。

7.6.2.4. 决定了模型从“原始输入”到“精炼特征”的压缩比

深度学习常被理解作为一种表示学习：从原始 d 维输入，逐层压缩提取出本质特征。

- d 就是这个压缩链条的起点。 d 越大，意味着原始信息越芜杂，网络前几层需要承担的“筛选和压缩”任务越重。
- CNN和Transformer之所以有效，正是通过卷积核、自注意力机制，让每个神经元不再需要与全部 d 个输入建立全连接，而是只处理局部或相关部分，从而打破了“每个神经元必须承担 d 个权重”的魔咒。



整个前向传播过程，就是神经网络根据每个样本自己独特的特征向量，不断提取/压缩特征，最后得到一个特征标量（在回归任务中通常得到一个标量；分类任务中每个样本对应一个标量， m 个样本则对应一个 m 维的向量）的过程。

7.6.3. 神经元权重数量决定了什么？

一个神经元拥有的权重数量（即其输入连接数），直接决定了该神经元能捕捉的“模式复杂度”的下限。以下是它最关键的几个决定性作用：

7.6.3.1. 决定了神经元的“感受野”大小和粒度

在输入层，权重数量就等于特征数 d ，意味着这个神经元能看到的“原始信息广度”。在隐藏层，一个神经元的权重数量等于前一层的神经元总数。权重越多，它能融合的信息来源就越广，理论上能捕捉的模式就越全局化。反之，卷积神经网络（CNN）通过让一个神经元只看图片的一小块（如 3×3 的像素块），把权重数从可能的上百万降到9个，从而强制神经元只学习局部微小特征（如边缘、纹理）。

7.6.3.2. 决定了该神经元决策边界的维度

一个拥有 d 个权重的神经元，在 d 维输入空间中划分一个超平面。 d 越大，这个超平面所处空间的维度就越高，它能切割出的分类边界理论复杂度就越高。但这同样是双刃剑：在小样本下，高维度的超平面极易扭曲成怪异的形状，将本应分开的样本缠绕在一起。

7.6.3.3. 决定了该神经元被冗余或无关特征“误导”的风险

权重数量多，意味着输入中可能混入大量无关特征。神经元必须通过训练，将无关特征对应的权重精确地学到零。如果做不到，这些带有噪声的权重就像收音机里的杂音，会严重干扰神经元的输出。权重数量越多，这种“权值稀疏化”的筛选任务就越繁重。

7.6.4. 神经元数量决定了什么？

在已经厘清特征数 d 决定单个神经元“输入权重数量”的基础上，我们现在讨论的是另一个独立维度——该层有多少个并行的神经元（ n ）在同时工作。这直接决定了网络的表示宽度和整体容量。

7.6.4.1. 决定了模型的“特征检测器”丰富度（表示宽度）

每个神经元本质上都是一个特征检测器，它学习识别输入中某种特定的模式组合并输出新的矩阵，这个 n 相当于新的输入矩阵中的特征数量 d 。

- 神经元数量 n 就是这一层中不同模式检测器的个数。
- n 太小：比如某层只有 3 个神经元，那它就只能捕捉并输出 3 种最粗略的特征，大量有用信息会被强行压缩和丢弃，形成信息瓶颈。
- n 增大：网络有能力同时学习更多样化的特征。例如，在图像识别中，有的神经元对边缘敏感，有的对纹理敏感，有的对特定形状敏感。更多的神经元意味着更丰富、更完整的特征描述。

类比：如果把每个神经元比作一位专家，那么 n 就是你聘请的专家数量。专家太少，一些重要领域的知识就完全缺失；专家多，团队的知识覆盖就更全面。

7.6.4.2. 决定了模型的容量与过拟合风险（表示能力的上限）

根据通用近似定理，足够的神经元数量是网络逼近任意复杂函数的前提。

- 容量： n 越大，整个网络的假设空间越大，拟合复杂非线性关系的能力越强。
- 过拟合：但容量过大会带来副作用。如果 n 过多而样本量不足，这些多余的神经元就会闲着没事干，集体去“记忆”训练数据中的噪声，导致泛化性能急剧下降。
- 双刃剑：因此，神经元数量需要在“欠拟合”（太少，学不到规律）和“过拟合”（太多，死记硬背）之间寻找最佳平衡点。

7.6.4.3. 决定了网络的鲁棒性与冗余度

- 太少神经元（ n 过小）：网络极其脆弱，如果某个关键神经元的权重受损或未能正确激活，整个输出就可能完全错误。这种表示是“紧致但危险”的。
- 稍多一些的神经元（ n 适中偏大）：配合 Dropout 等正则化技术，网络会形成一种分布式表示——信息不是存在一个神经元里，而是分散在多个神经元中。这会带来很强的鲁棒性，少数神经元的失效不影响整体判断。这就像一份重要文件被多个副本同时记录，比只交给一个人保管更可靠。

7.6.5. 小结

	样本数量 (N) (决定泛化能力的“数据量”)	样本特征数量 (d) (决定输入空间的“维度”)
神经元数量 (n) (决定每层的“检测器丰富度”)	n 与 N 的容量匹配 • N 小 n 大: 神经元过多, 极易死记硬背噪声, 过拟合风险极高。在大模型时代, 这种闲着没事干转去‘记忆噪声’的神经元被称为冗余神经元。现代工程上通过‘模型剪枝’把它们直接砍掉, 或者利用 MoE (混合专家模型) 机制, 让这n个专家分班倒, 每次只激活一小部分, 既保留了庞大的特征检测器丰富度, 又彻底断了它们合伙死记硬背的后路 • N 大 n 小: 容量不足, 无法捕捉复杂规律, 陷入欠拟合。 • 法则: n 需随 N 增长而适度扩大, 并配合 Dropout/L2 正则化。	n 与 d 的成本协同 • 总参数量 $\approx d \times n$ (第一层)。 • d 大时增加 n: 参数与计算量爆炸, 极易过拟合。 • d 小时增加 n: 成本可控, 可安全扩充检测器种类, 提升表示能力。 • 法则: d 越小越可放心加大 n。
神经元权重数量 (每神经元的权重数, 输入层即为 d)	权重数与 N 的供需关系 • 每个神经元的权重越多, 需要更多的 N来准确估计这些权重。 • 权重多而 N 少: 神经元内部自由度太大, 极易将噪声当成规律, 权重过拟合。 • 法则: 高维输入必须配以大样本, 否则每个神经元都无法可靠学习。	权重数由 d 直接决定 • 输入层: 每个神经元的权重数 = d。 • d 增大 $\rightarrow$ 每个神经元的“认知负荷”上升, 维度灾难加剧。 • 降维/特征选择: 本质就是直接削减每神经元的权重数, 从而降低学习难度与样本需求。

7.7. 收拾行李再出发

上述关于样本数量/样本特征数量/神经元数量/神经元权重数量的思考是粗浅的, 这些参数我们称之为超参数 (Hyperparameter)。如果你觉得上一小节里的文风突变, 没错, 是本人“借鉴”的大模型的回答, 哈哈~

限于本人能力, 更多更精妙的调参方式不在我们本节的讨论之中, 需要各位各展神通, 自己去探索这些支线任务。

哦, 顺便提一句, 使用大模型学习是一种很不错的方式哦。

8. 打破线性诅咒!

从那些梯度下降图中走出来吧, 准备前往下一段冒险!

前方归来的剑客们细细簌簌地说着什么, 他们从你身边经过,用复杂的眼神看着即将踏上下段征途的你。

或许，在此之前，你已经隐隐听到过什么可怕的传说，一个关于神经网络深度的可怕诅咒！

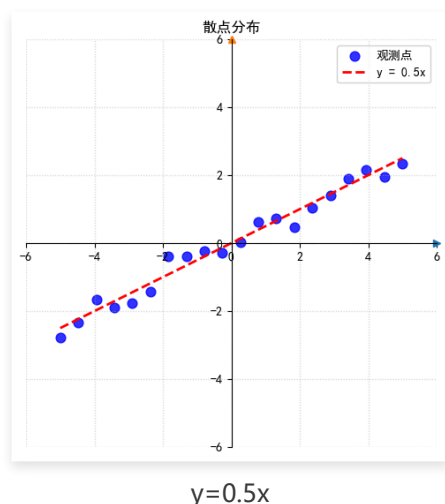
这位剑客，我可以明确的告诉你：这不只是一个可怕的传说，而是真实地藏匿在每层神经网络间的诅咒！

只有破除这个诅咒，我们才能走向更远的目标。此行，我们不破楼兰终不还！

8.1. 从线性变换到仿射变换

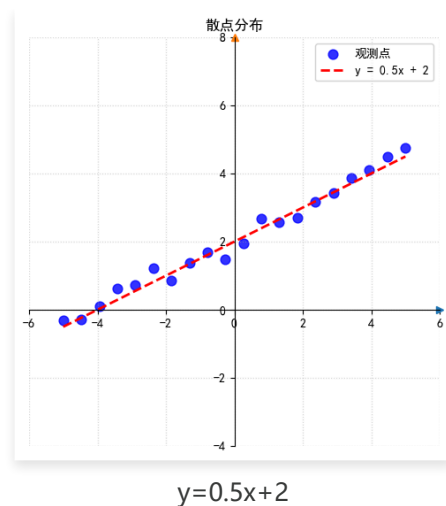
开始本节前，先引入一个概念：偏置。

- 一开始，我们的方程是这个样子： $y = wx$ ，学过初中数学的你，一定一眼看出，这是一个穿过原点的直线，我们随意调整 w 的值，这条直线也只是一个以原点为中心， 360° 旋转的固定直线。
- 有20个数据 (x, y) ：(-5.00, -2.50), (-4.47, -2.24), (-3.95, -1.97), (-3.42, -1.71), (-2.89, -1.45), (-2.37, -1.18), (-1.84, -0.92), (-1.32, -0.66), (-0.79, -0.39), (-0.26, -0.13), (0.26, 0.13), (0.79, 0.39), (1.32, 0.66), (1.84, 0.92), (2.37, 1.18), (2.89, 1.45), (3.42, 1.71), (3.95, 1.97), (4.47, 2.24), (5.00, 2.50)。
- 我们需要找到一条直线来拟合这组数据，你很快可以找到这条直线： $y = 0.5x$ ，你轻松找到了这个 $w = 0.5$ 。



现在再给你20个数据 (x, y) ：(-5.00, -0.50), (-4.47, -0.24), (-3.95, 0.03), (-3.42, 0.29), (-2.89, 0.55), (-2.37, 0.82), (-1.84, 1.08), (-1.32, 1.34), (-0.79, 1.61), (-0.26, 1.87), (0.26, 2.13), (0.79, 2.39), (1.32, 2.66), (1.84, 2.92), (2.37, 3.18), (2.89, 3.45), (3.42, 3.71), (3.95, 3.97), (4.47, 4.24), (5.00, 4.50)。

这时，你发现，无论如何转动 $y = 0.5x$ 这条线，都无法完美拟合上述数据点，聪明的你发现，只需将 $y = 0.5x$ 轻轻上移2个单位，这条直线又能完美拟合数据点了。



$$y=0.5x+2$$

- 这里，我们把偏移的这2个单位叫做：偏置，用 b 来表示。我们把加上偏置的线性函数升级为仿射函数。
- 在二维平面上，可以找到一个最优的仿射变换 $y = wx + b$ 来近似描述任意一组满足线性关系的散点数据（不垂直于 x 轴的数据点）。

什么叫线性？

“线性”这个词，核心含义是成比例、可叠加、像直线一样。满足两个性质的关系：

- 可加性（叠加性）： $f(x + y) = f(x) + f(y)$
- 齐次性（比例性）： $f(kx) = k \cdot f(x)$

放射变换：

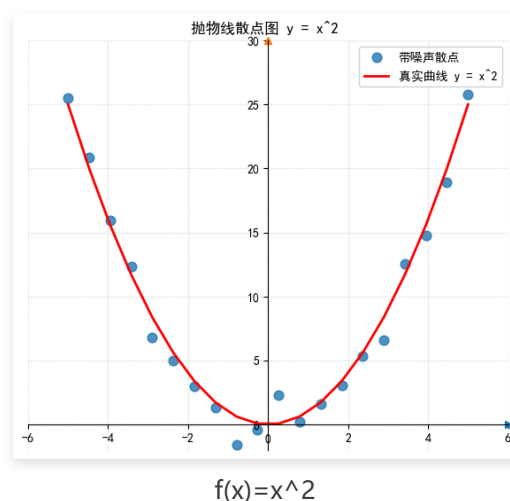
- 线性函数，例如： $y = 2x$ 是线性的（图像是过原点的直线）。但 $y = 2x + 1$ 严格说不是“线性函数”，而是“仿射函数”，因为不满足叠加性和齐次性，不过生活中常混用。

8.2. 激活函数：Sigmoid

好了，开胃小菜下肚了，接下来我们要面对更复杂的局面。

8.2.1. 来自非线性相关的难题

再次给你一组数据 $(x, f(x))$ ，它们在平面上的分布近似 $f(x) = x^2$ ，请你用任意仿射函数来模拟这条曲线：



$$f(x)=x^2$$

你很快就发现，无论如何旋转/平移任何仿射函数，都无法令一条直线弯曲，所以仿射函数永远无法拟合一组非线性相关的数据。

8.2.2. 让直线弯曲的奥秘

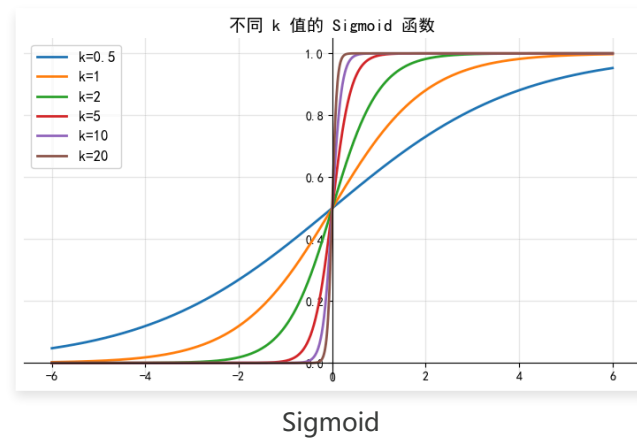
8.2.2.1. 初识Sigmoid

只有魔法才能打败魔法！我们请来一位强力帮手，它将带我们顺利打破诅咒！

为了引入曲线，我们必须也要加入一个曲线方程，这个方程就是大名鼎鼎的Sigmoid激活函数：

$$\sigma_k(x) = \frac{1}{1 + e^{-kx}} \circ$$

k 取不同值时，它们的图像长这样：



8.2.2.2. 近似逼近

我们用先2个Sigmoid的线性组合来逼近 $f(x) = x^2$ ，令他们的线性组合为 $g(x)$ ：

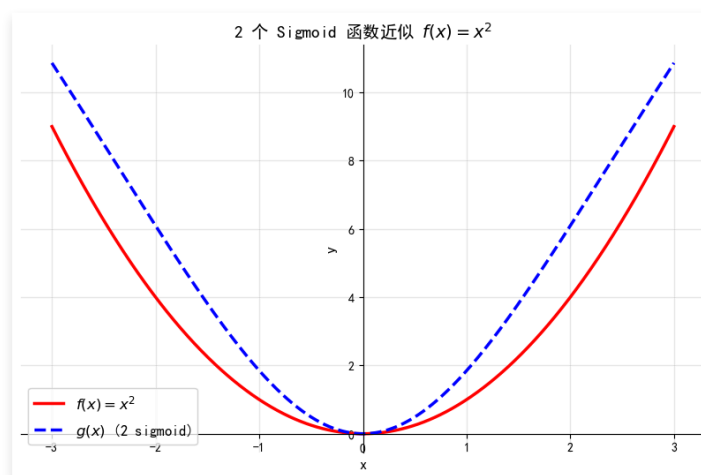
使用2个仿射函数： $-4x + 9$ ， $4x + 9$ ；

$$k = 1, \quad g(x) = \underbrace{(-4x + 9) \cdot \sigma(-kx)}_{x < 0} + \underbrace{(4x + 9) \cdot \sigma(kx)}_{x \geq 0} - 9$$

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  # 基础sigmoid函数
5  def sigmoid(x):
6      return 1 / (1 + np.exp(-x))
7
8  # ===== 核心: 2个sigmoid构成的 g(x) =====
9  def g(x):
10     k = 1 # 陡峭度
11     # 2个sigmoid门控
12     s_left = sigmoid(-k * x) # 控制 x < 0
13     s_right = sigmoid(k * x) # 控制 x >= 0
14
15     # 左右对称仿射函数 (一次函数 ax + b)
16     left_term = (-4 * x + 9) * s_left # x < 0
17     right_term = (4 * x + 9) * s_right # x >= 0
18
19     return left_term + right_term - 9
20
21 # ===== 绘图 (十字坐标轴) =====
22 x = np.linspace(-6, 6, 1000)
23 f_true = x ** 2 # 真实x^2
24 g_fit = g(x) # 2 sigmoid 拟合
25
26 # 创建画布
27 fig, ax = plt.subplots(figsize=(10, 6))
28
29 # 画图
30 ax.plot(x, f_true, label='$f(x) = x^2$', color='red', linewidth=2.5)
31 ax.plot(x, g_fit, label='$g(x)$ (2 sigmoid)', color='blue', linestyle='--', linewidth=2.5)
32
33 # 十字坐标轴
34 ax.spines['left'].set_position(('data', 0))
35 ax.spines['bottom'].set_position(('data', 0))
36 ax.spines['right'].set_color('none')
37 ax.spines['top'].set_color('none')
38
39 # 标签与网格
40 ax.set_xlabel('x', fontsize=12)
41 ax.set_ylabel('y', fontsize=12)
42 ax.legend(fontsize=12)
43 ax.grid(True, alpha=0.3)
44 plt.title('2 个 Sigmoid 函数近似 $f(x)=x^2$', fontsize=14)
45 plt.show()

```



2个Sigmoid

我们用4个Sigmoid的线性组合来逼近 $f(x) = x^2$:

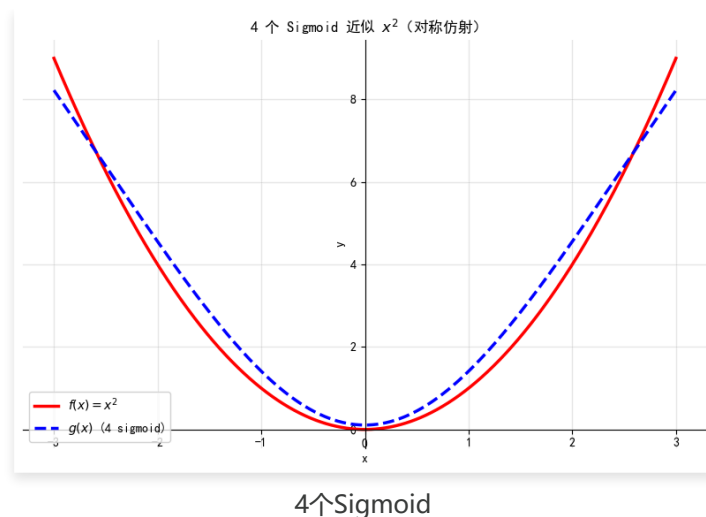
使用3种仿射函数: $x + 1, x - 1, x$;

$$k = 1, S = \begin{cases} s_1 = \sigma(-k(x + 1)) & (x < -1) \\ s_2 = \sigma(k(x + 1))\sigma(-kx) & (-1 \leq x < 0) \\ s_3 = \sigma(kx)\sigma(-k(x - 1)) & (0 \leq x < 1) \\ s_4 = \sigma(k(x - 1)) & (x \geq 1) \end{cases}, g(x) = (-3x + 2)s_1 + (-x + 1)s_2 + (x + 1)s_3 + (3x + 2)s_4 - 1.7$$

4个Sigmoid的线性组合

Python

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  def sigmoid(x):
5      return 1 / (1 + np.exp(-x))
6
7  # ===== 4 个 Sigmoid 的 g(x) 函数 =====
8  def g(x):
9      k = 1
10     # 4个区间门控
11     s1 = sigmoid(-k * (x + 1))      # x < -1
12     s2 = sigmoid(k*(x+1)) * sigmoid(-k*x) # [-1, 0]
13     s3 = sigmoid(k*x) * sigmoid(-k*(x-1)) # [0, 1]
14     s4 = sigmoid(k * (x - 1))      # x > 1
15
16     # 左右对称仿射函数 (一次函数)
17     t1 = -3 * x + 2   # x < -1
18     t2 = -1 * x + 1   # [-1, 0]
19     t3 = 1 * x + 1    # [0, 1]
20     t4 = 3 * x + 2    # x > 1
21
22     return t1*s1 + t2*s2 + t3*s3 + t4*s4 -1.7
23
24 # ===== 绘图 (十字坐标轴) =====
25 x = np.linspace(-3, 3, 1000)
26 f_true = x ** 2
27 g_fit = g(x)
28
29 fig, ax = plt.subplots(figsize=(10, 6))
30 ax.plot(x, f_true, label='$f(x)=x^2$', color='red', linewidth=2.5)
31 ax.plot(x, g_fit, label='$g(x)$ (4 sigmoid)', color='blue', linestyle='--', linewidth=2.5)
32
33 # 十字坐标轴
34 ax.spines['left'].set_position(('data', 0))
35 ax.spines['bottom'].set_position(('data', 0))
36 ax.spines['right'].set_color('none')
37 ax.spines['top'].set_color('none')
38
39 ax.set_xlabel('x')
40 ax.set_ylabel('y')
41 ax.legend()
42 ax.grid(alpha=0.3)
43 plt.title('4 个 Sigmoid 近似 $x^2$ (对称仿射)')
44 plt.show()
```



我们看到，4个Sigmoid已经可以把 $f(x) = x^2$ 近似逼近地比较完美了。

上述逼近只是在单位长度为1地区间去近似。

试想，我们不断细化区间，最终一定可以在在 $[-3, 3]$ 这个区间用多个仿射函数与Sigmoid的组合来逼近这个函数。

当然，这里并非要求你如何计算出这些仿射函数和Sigmoid函数的具体值，我们只需要知道，仿射函数在多个Sigmoid函数变换后，可以拟合非线性相关的数据。

尝试去体会这句话：**神经网络本质上一个巨大的函数。**

我们丢给函数一个输入，函数返回一个输出。

而神经网络是：我们丢进去一个单词，比如“我”，函数返回一个“爱”，再把“爱”丢给他，它返回一个“你”，这就是大语言模型；我们也可以丢给它一张小猫图片，它返回给我们“猫”这个标签，这个就是图像识别；我们丢给它一个音高“A”，它返回一个“C”，这就是音乐生成。

8.3. 为什么要引入非线性相关？

对于二维平面上的任意数据组，线性或非线性，只要它符合某种规律，理论上都可以找到一个曲线来拟合这组数据所在的。

那这个和我们的神经网络又有什么关系？神经网络为什么需要激活函数这种非线性函数？

8.3.1. 经典XOR问题

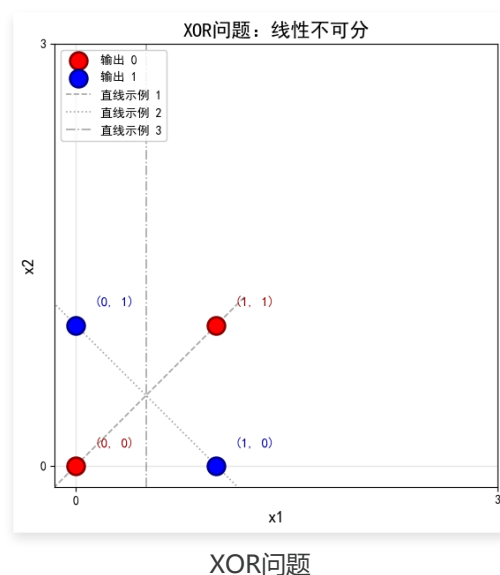
XOR问题是神经网络发展史上的一个经典难题，直接推动了人工智能从第一代感知机寒冬走向反向传播算法的复兴。

XOR（异或）的逻辑很简单：**两个输入相同时输出0，不同时输出1**。总结为表格的形式：

输入 x_1	输入 x_2	输出 y
0	0	0
0	1	1
1	0	1

1	1	0
---	---	---

画在二维平面上：



这意味着，无法找到任何一条仿射函数来将红点和蓝点区分开。

8.3.2. 多层线性层等价于单层线性层

关键在于：线性函数的复合，依然是线性函数。

1. 单层线性变换

一个没有激活函数的全连接层，其本质就是一个仿射变换（线性变换+偏置），公式为：

$y = Wx + b$ 。其中 W 是权重矩阵， b 是偏置向量。

2. 多层线性层的堆叠

假设我们有两个这样的“线性层”：

- 第一层： $h = W_1x + b_1$
- 第二层： $y = W_2h + b_2$

现在，我们将第一层的输出代入第二层，看看整个网络的复合函数是什么：

$$y = W_2(W_1x + b_1) + b_2$$

$$y = (W_2W_1)x + (W_2b_1 + b_2)$$

观察结果：令 $W' = W_2W_1$ ， $b' = W_2b_1 + b_2$ ，那么双层网络就变成了 $y = W'x + b'$ 。这依然是一个**单一的线性变换**。

3. 推广到任意多层

这个结论可以用数学归纳法轻易推广：

无论堆叠多少个无激活函数的全连接层，其效果都完全等价于一个单层线性模型。

因为每增加一层，只是将新的权重矩阵左乘到现有矩阵上，最终总是能得到一个等效的权重矩阵 W' 和偏置 b' 。这可以用公式表示为：

$$f(x) = W_n(\dots(W_2(W_1x + b_1) + b_2)\dots) + b_n = W'x + b'$$

4. 加入激活函数 σ

网络的前向公式为： $y = W_2 \cdot \sigma(W_1 x + b_1) + b_2$ ，这时候**绝不能**将 W_2 和 W_1 合并，因为被 σ 隔开了。

核心逻辑：

1. 隐藏层利用 Sigmoid 的非线性压缩，将原始的线性空间（XOR 问题中的四个点）映射到一个新的特征空间，使得 $(0, 0)$ 和 $(1, 1)$ 不再与另外两点线性不可分。
2. 在新的特征空间中，输出层只需要画一条简单的直线，就能把两类点分开。

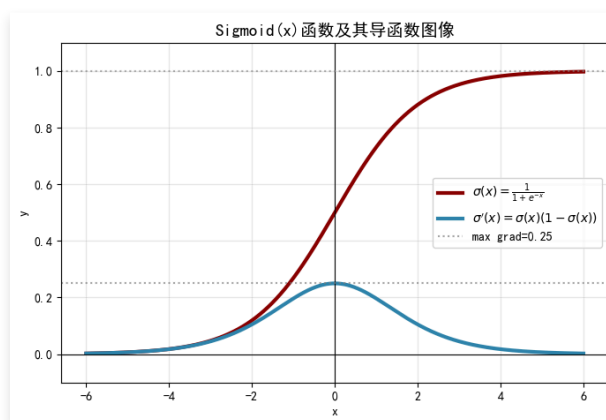
而这恰恰是无法从 $y = W_2 \cdot \sigma(W_1 x + b_1) + b_2$ 化简为 $y = W'x + b'$ 的原因。因为 σ 打破了分配律，你无法将 W_2 乘进 $\sigma()$ 内部，也无法将 W_1 提出来。

8.3.3. 更多激活函数

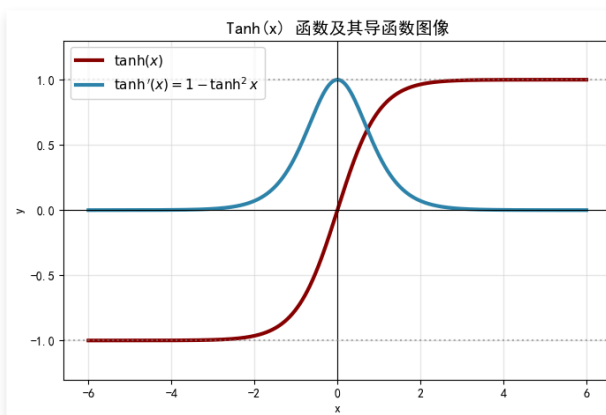
激活函数是随着神经网络发展，一路进化到现在的，其历史脉络大概如下：

1. 平滑饱和时代——Sigmoid 与 Tanh

上世纪80~90年代，随着反向传播的普及，**Sigmoid**成为标配。它把输出挤压到 $(0,1)$ 之间，可解释为概率。随后，**Tanh**将其改进为 $(-1,1)$ 的零中心输出，收敛更快。



Sigmoid(x)函数及其导函数图像



Tanh(x) 函数及其导函数图像

这个阶段的问题也随之暴露，即**梯度消失**：

- 因为两者为 S 型函数，两端接近平直，当输入绝对值稍大时，函数两侧导数趋近于 0。

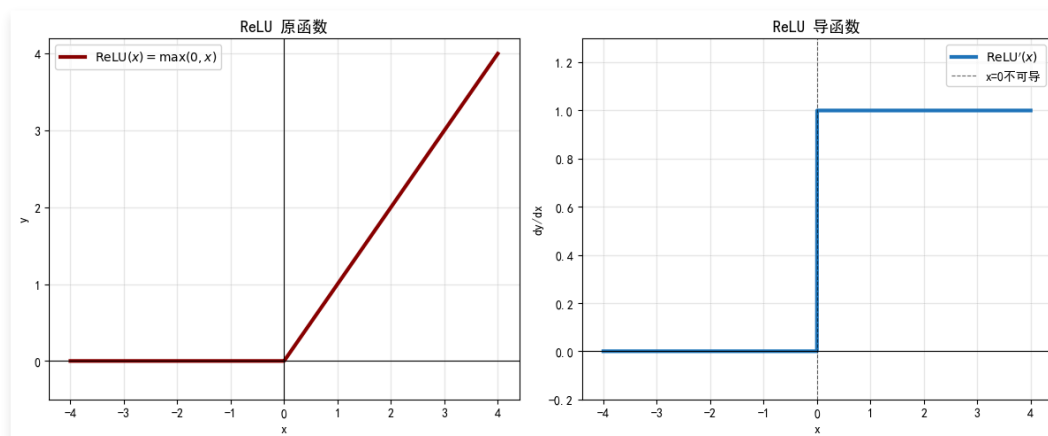
- 误差反向传播时，深层网络的梯度经多层Sigmoid连乘后会指数级衰减，导致浅层网络几乎无法学习。

于是迫切需要一种在正区间导数不衰减的函数。

2. 非线性革命——ReLU 的诞生

直到2010年前后，Hinton等人推动的**ReLU** (Rectified Linear Unit) 带来了质变。

$$\text{ReLU}(x) = \begin{cases} x, & x > 0 \\ 0, & x \leq 0 \end{cases}$$



它**非饱和**，正区间的导数恒为1，让深层网络不再受困于梯度消失。同时，它计算极简、能产生稀疏激活，配合**Dropout**和**权重初始化**等技术，一举开启了真正的“深度学习时代”。

但ReLU有一个致命缺陷——“**神经元死亡**”：

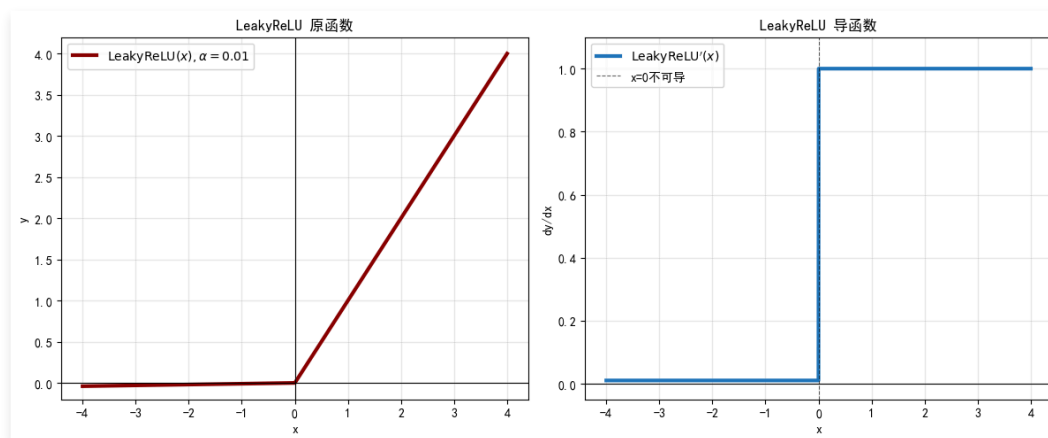
- 如果某个神经元对所有输入都输出负值，梯度为0且权重永不更新，它便永久失效了。
- 尤其在**高学习率**或**偏置参数不当**时，网络中可能有大量死亡神经元。

3. 为ReLU “续命” ——变体涌现

针对“死亡”问题，研究者开始给负半轴一点“气息”：

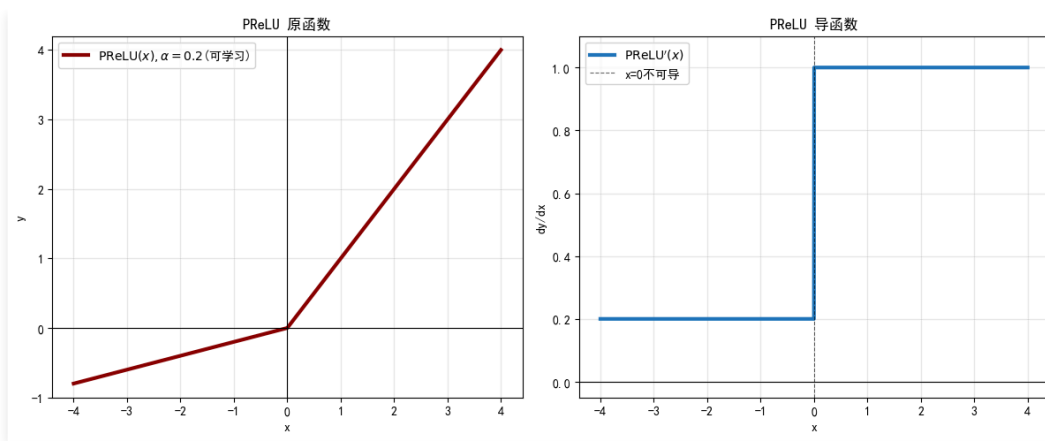
- **Leaky ReLU**：给负半轴一个极小的固定斜率（如0.01），让导数不再为0。

$$\text{LeakyReLU}(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases} \quad (\alpha \ll 1, \text{常用} \alpha = 0.01)$$



Leaky ReLU 公式与导数

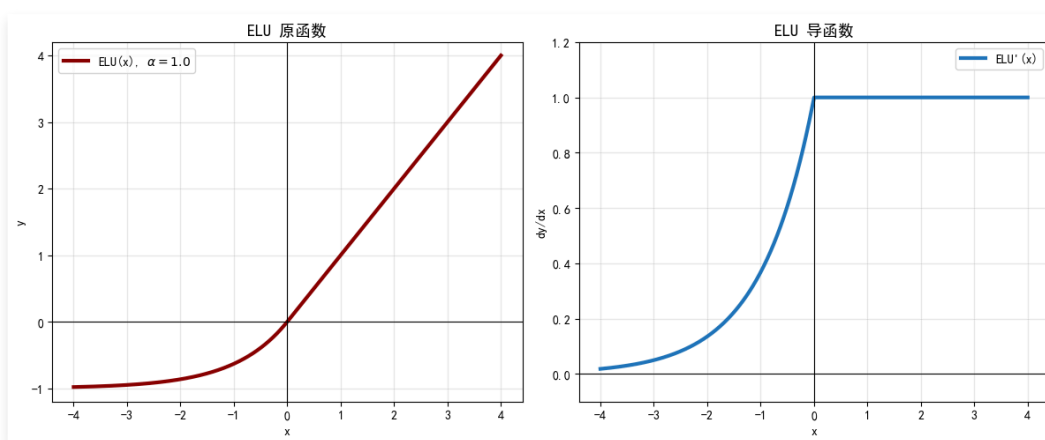
- **PReLU**：将负半轴斜率 α 变为一个**可学习的参数**，让网络自己决定激活形状。



PReLU

- **ELU**: 负半轴使用指数函数 $\alpha(e^x - 1)$ ，输出均值更接近0，使训练更稳健，但计算稍重。

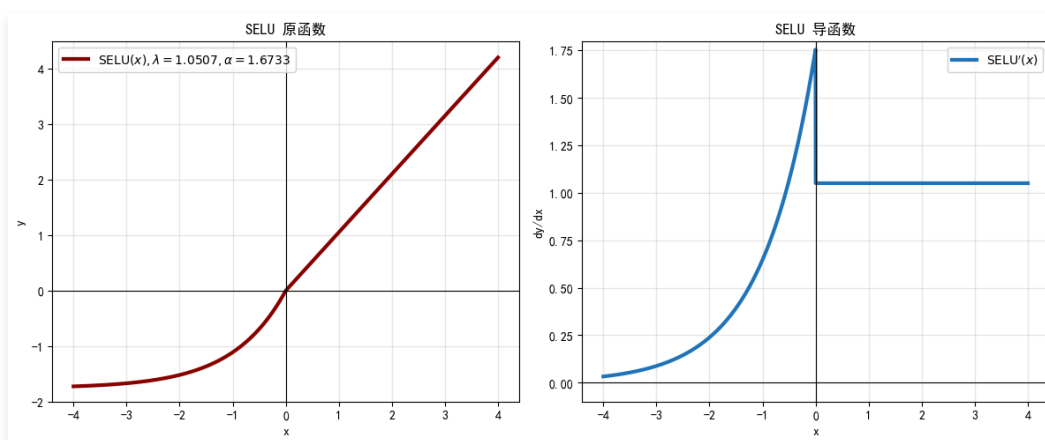
$$\text{ELU}(x) = \begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & x \leq 0 \end{cases}$$



ELU

- **SELU**: 在ELU上加入自归一化性质，可让网络在前向传播中自动维持均值为0、方差为1，适合极深的**全连接网络**。

$$\text{SELU}(x) = \lambda \cdot \begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & x \leq 0 \end{cases}, \quad \text{固定超参: } \lambda \approx 1.0507, \alpha \approx 1.6733$$



SELU

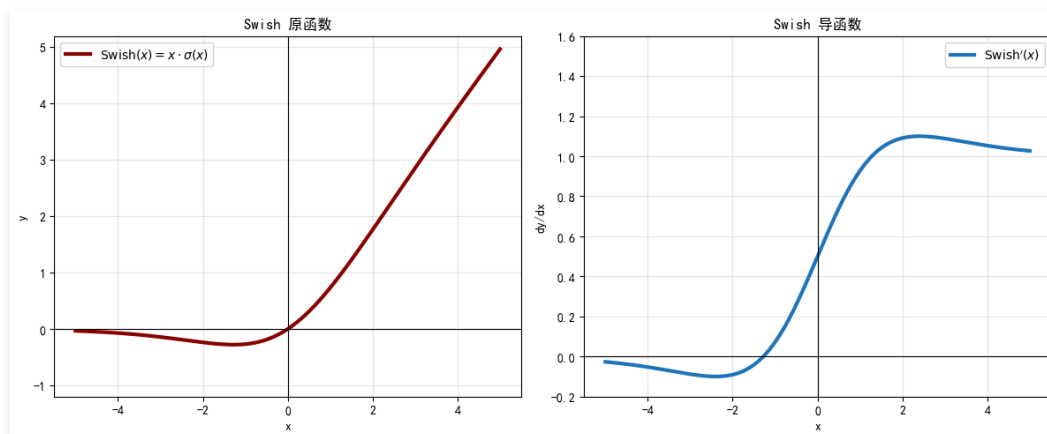
这些变体在不同场景弥补了ReLU的不足，但ReLU本身仍是卷积网络的默认首选。

4. 第五阶段：搜索与复合——GELU、Swish 与 “平滑ReLU”

2017年以后，网络设计开始摆脱“纯线性修正”的思维，转而追求**平滑、可学习的非线性**。

- **Swish**: 由Google用自动搜索发现, $x \cdot \sigma(x)$, 非单调的平滑曲线, 在深层模型上常优于ReLU。

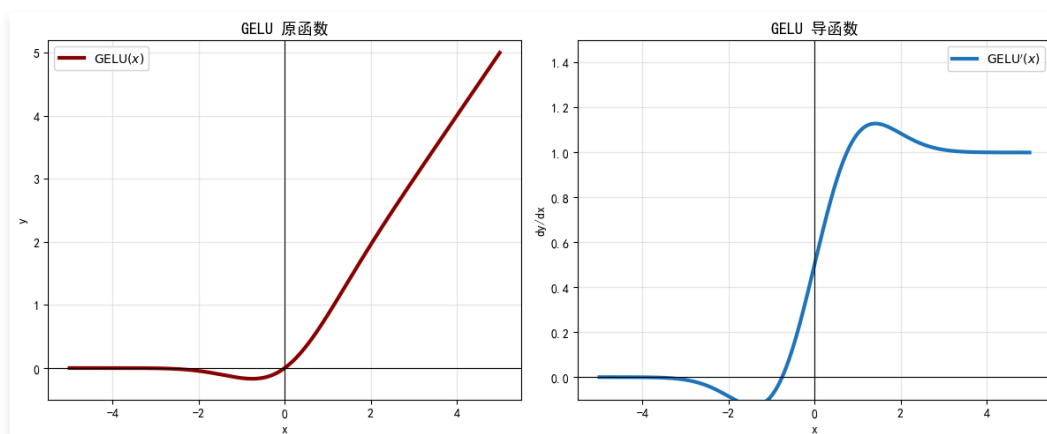
$$\text{Swish}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}, \quad \sigma(x) = \frac{1}{1 + e^{-x}}$$



Swish

- **GELU**: 受高斯分布启发, 可以看作ReLU的概率平滑版。它被**Transformer**家族 (如BERT、GPT-3等) 广泛采用, 成为NLP和现代视觉模型的首选激活之一。

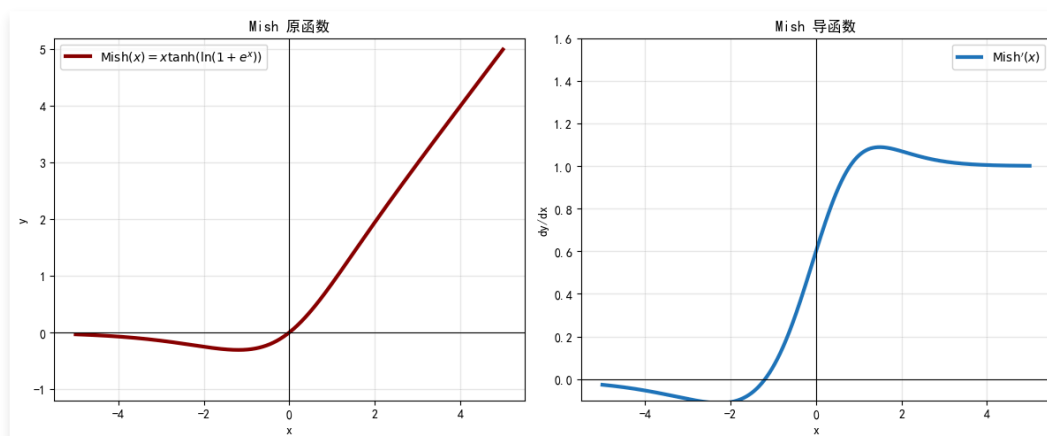
$$\text{GELU}(x) = x \cdot \Phi(x)$$



GELU

- **Mish**: 类似Swish但更平滑, 配合YOLOv4等模型表现抢眼。

$$\text{Mish}(x) = x \cdot \tanh(\text{softplus}(x)) = x \tanh(\ln(1 + e^x))$$



Mish

这类函数的共同思路是：**像ReLU一样有非饱和上界，但加上平滑下界和微小的负值隆起，同时处处可导**，让梯度和信息流动更加细腻。

8.3.4. 激活函数的抉择

1. 梯度消失与梯度爆炸的永恒博弈

网络的深度和架构不同，对梯度的容忍度就天差地别。

2. 特征表达与数值动态范围的冲突

不同类型的数据，需要不同的输出范围。

3. “神经元死亡”与“稀疏性”是把双刃剑

ReLU 最著名的“死亡”问题，在某些场景下其实是故意的。对于模型剪枝、稀疏推理加速，这种特性是求之不得的优点。

4. 计算效率与表达精度的权衡

大模型时代，这是最现实的矛盾。

激活函数的选择上，面临着数学的铁律：

- **万能逼近定理**：它要求激活函数**必须有非线性**，否则多层网络会坍缩成单一的线性变换。这是激活函数存在的根本。
- **梯度消失/爆炸定律**：Sigmoid/Tanh 的饱和区导数趋近于 0，连乘后梯度呈指数衰减。这是数学上的必然，直接宣告了它们无法用于极深网络的“死刑”。
- **内部协变量漂移**：输出的均值非零、方差失控，会从数学上证明训练会不稳定。这催生了对**零中心、有界方差**函数的需求。

这层理论规定了“什么函数**绝对不行**”，但它没有规定“什么函数**最好**”。ReLU 在正半轴导数恒为1，完美避开了梯度消失的数学死结，这是它革命性的理论贡献。

落地面临着工程上的严苛筛选：

理论只给方向，有效性是由炼丹师们在无数次实验中“试”出来的。这完全是经验科学的范畴。

- **ReLU 的“死亡”问题**：理论上，ReLU 完美的线性区解决了梯度消失。但实践中发现，大学习率或不当的偏置会导致大量神经元永不被激活。Leaky ReLU 和 PReLU 这些补救方案，就是典型的“发现问题-工程修补”过程。
- **平滑激活的意外胜出**：GELU 最初是受概率论中的高斯误差函数启发，形状和 Swish (由自动搜索发现) 很相似。它们之所以比 ReLU 好，并不是因为哪条定理规定了，而是**实验证明**，这种带微小负值隆起、处处可导的平滑过渡，能让 Transformer 等深层模型的损失函数地形更平滑，收敛更快更好。这就像发现一种新药，疗效显著，但其微观药理机制可能还需要多年才能完全解释清楚。

这一层是最残酷的筛选器。今天绝大多数的激活函数创新都死在了这一步：理论上完美无瑕，但一跑实验就各种发散、对超参极其敏感、或者毫无提升。

硬件和部署成本是终极裁判：

这是现实世界施加的硬约束，是激活函数能否大规模普及的“生死关”。

大模型在应用实践中往往是一个工程技术问题，笔者在这方面实在是欠功夫，本章大量资料来自于AI的总结，个人还差火候的很。

9. 从理论到实践---Dirty your hands

到此为止，我们已经简单地了解一下理论上的工具。当然，后边还有很多基础知识要补充，但太早留恋于理论之地，不是我的初衷。

这一章，我们要准备好Dirty hands，我们将跟随Karpathy的视角，开始用代码构建完整的项目。道阻且坚，行则将至。

这一章，算不上一个完整的章节，只是与大家分享一下设备、写代码的IDE和深度学习框架。

9.1. 设备

神经网络前向/反向传播基于张量运算，cuda为这种大量并行运算提供了硬件基础，cuda作为Nvidia的核心护城河，让黄老板在AI掘金时代成为最富有的铲子供应商。直到今日，性能最好的计算卡依然来自Nvidia，所以，显卡必选N卡。大显存，高带宽，奔着这个标准选吧。

CPU相对于显卡，重要性并不突出，但并不意味着CPU性能不重要。在GPU开始学习之前，CPU需要负责从硬盘读取数据、进行解码、裁剪、旋转等各类增强操作。如果CPU处理太慢，跟不上GPU的计算速度，GPU就会处于空闲等待状态，导致利用率低下。

前期的练手阶段，甚至CPU都可以解决，入门后，可以选择在本地训练小模型，如果后期上手了，还是要选云GPU服务器。

指导方针：遇事不决问AI。

9.2. IDE

传统IDE：pycharm和VS code。

目前在用VS code，开源IDE，插件丰富。最重要的是，目前AI编程IDE的框架基本来自于VS code，后期转AI IDE比较容易上手。编程语言自然是Python。

当然，从前几年开始，cursor/Tare等AI编程软件刷屏，Claude code/Codex火爆全网，层出不穷的AI IDE和skills令新手编程变得无比轻松。

我依然建议你关闭自动补全和AI功能，体验从零到一那种造物主的快感！

9.3. 深度学习框架

主流的深度学习框架有Pytorch、Tensorflow，曾经国内百度Paddle也火过一段时间，后来好像没什么动静了。

目前好像用的比较多的还是Pytorch。

9.4. 环境配置

Miniconda是Anaconda的轻量级“最小化”版本。它是一个环境和包的管理工具，核心作用是帮你隔离不同的项目，避免依赖冲突。

Anaconda太过庞大，多数内容用不到，推荐用Minicoda，需要什么下载什么。

用conda新建环境后，下载指定python版本，cuda工具包和常用的库等。

AI比我更熟悉整套流程，尽管去问。

10. 构建BigramMLP——MLP原理与前向传播

MLP 是 多层感知机 (Multi-Layer Perceptron) 的缩写，是最基础、最经典的前馈人工神经网络结构。

你可以把它理解为深度学习世界的“乐高积木”——虽然简单，但几乎是所有复杂模型（包括Transformer）的基本组件。

MLP的前向传播本质上是一个逐层将原始数据变形，直到它能用一条直线切开的过程。

我们将在本章带你从0构建一个完整的MLP，从前向传播到反向传播。

最后，你看会到loss稳定下降，然后用我们构建的MLP去生成内容！

10.1. 大模型如何学会生成名字？

本篇文档的核心逻辑来自Karpathy的系列课程：《Neural Networks: Zero to Hero》，所以本文大部分内容都有他的影子，代码部分则影响更深。所以本节这个问题是受bigram模型启发而来。

重申一遍，本文档基本是Karpathy视频的注脚，我基于个人理解和学习，补充了一些我认为比较重要且基础的数学和代码知识而已。

所以鉴于笔者笔力不足，一定许多理解不到位甚至错误的地方，希望各位辩证且逻辑地阅读本文。

总而言之，非常推荐大家去观看原视频。

为了回答本节的问题，我脑海中浮现出一句话，来自《三国志·魏志·王肃传》，文言：“人有从学者，遇不肯教，而云：‘必先读百遍’，言‘读书百遍而义自见。’”

后世精简为：读书百遍，其义自见。

我认为这是一个比较好理解的切入点。

小学，面对难以理解的复杂文章，往往被建议“全篇背诵”。这是人类理解文章的方法之一，让一个不懂的人，通过机械重复可以完整复述内容，甚至逐渐理解文章所要表达的意思。所以很多聪明人之所以聪明，其实是他记住的东西很多。

当一个人读的东西越多，质量越高，他所能表达的意思也就越丰富。这本质上是人类对语言中每个单词的位置产生了顺序上记忆。当一个人听到“**我爱吃苹**”这几个字，接着脑海中浮现出“**果**”这个字，因为生活中这样的语句出现的概率最大。在楚门的世界里，如果人们灌输给他“**苹果**”这种奇怪的水果名字，他的世界里，“**我爱吃苹果**”则是最大概率的事件。

维特根斯坦在《逻辑哲学论》中，他写下：“我的语言的界限，意味着我的世界的界限。”

我们还知道：人类无法想象不存在的东西。

你尽管去观察体会，身边的世界，只要是你可以用语言描述的，都在你的认知范围之内；凡认知之外的事，都无法用你自己的语言描述。

之所以大语言模型表现出令人惊叹的智慧，是因为大模型精炼了人类世界中，所有可描述之物的关系，这样的表述或许太难懂。我这样说：大模型某种程度上同你一样，不过他学习的知识是人类几千年来来的总和，更夸张的说，他是人类语言的高度浓缩，而人类对世界的理解就存在于语言中，语言又被凝练到大模型的层层网络中。

好吧，似乎扯的太远了。回到：“**我爱吃苹果**”这个例子，这里我们提到了概率这个东西。这是我们量化的根本。你生命中听别人提到“**我爱吃苹果**”的概率远远高于“**我爱吃苹果**”、“**我爱吃苹果**”……，假如你生命中听到的所有话都被记录了下来，我们就可以统计出“**我爱吃苹果**”出现的次数在“**我爱吃苹果**”、“**我爱吃苹果**”、“**我爱吃苹果**”……所有语句中的概率。当然，这里不是《冒牌天神》，没有上帝的柜子存放我们说过听到的话，但可以粗略估计一下，出现的“**我爱吃苹果**”的概率大概会是90%以上，甚至对于那些没有听说过“**苹果**”的朋友，这个概率会接近99%。

回到最开始的问题：大模型如何学会生成名字？

我们把成千上万的名字交给一个机器，这个机器会把名字中紧挨的两个字母拆出来，假设一个名字是“alex”，就被拆分为“al”、“le”和“ex”。当各式各样的名字被拆解，这些字符对可以依次放在一个 26×26 的表格中。

“a”后边会出现26种字母，但每一种的概率不同，假出现“l”的概率有10%(已经远高于随机分布： $1/26$ 了)，大模型会优先挑选概率高的去生成，于是，在给定“a”的情况下，10%的概率生成一个“l”；同上，“l”后边出现26种字母，但每一种的概率不同，假设出现“e”的概率有15%，于是，在给定“l”的情况下，15%的概率生成一个“e”；循环往复，直到给出一个完整的“alex”。

ok，至此，关于“大模型如何学会生成名字？”这个问题，我们已经有了一个基本的方法论——概率论。基于此，我们开始构建一个简单的小模型。来生成名字。

（哦对了，如果如下代码在你的终端里打印出的数字是四位小数，别慌，本文中很多数字是为了方便显示，只保留了两位小数，但它们在内存中是一致的！）

10.2. 导入与处理数据

训练大模型最重要的就是数据，一切关系都隐藏在字母与字母之间。为了让 26×26 的表格看起来饱满，我们需要尽可能多的名字。Karpathy在他的github项目中已经为我们提供了一份名字文档“names.txt”。我们还是用这份文档来演示。

文档中有32033个名字，我们先挑出前6个名字看一下，前几个名字会经常出现在下边的例子中。

```
1 emma
2 olivia
3 ava
4 isabella
5 sophia
6 charlotte
7 ...
```

10.2.1. 导入数据

在项目文件夹中新建一个.py或.ipynb文件，并把“names.txt”放在同一文件夹中。

```
1 import torch
2 # 导入数据文件
3 file_path = "names.txt"
4 with open(file_path, "r", encoding="utf-8") as f:
5     content = f.read()
6 data2 = content.splitlines() # splitlines()方法可以从换行符的位置将字符串分割并转换为列表，与join()方法相反
7 data = list(".".join(data2)) # join ()方法可以从将列表中的字符串，以"."的方式打断并连接
```

经过上述处理，我们的名字转换为（取前32个字符，包含前5个名字）：

```
1 ['e', 'm', 'm', 'a', '.', 'o', 'l', 'i', 'v', 'i', 'a', '.', 'a', 'v', 'a', '.', 'i', 's', 'a', 'b', 'e', 'l', 'l', 'a', '.', 's', 'o', 'p', 'h', 'i', 'a', '.']
```

10.2.2. 构建数据集和目标集

将字母排为一排，放在第一行，这一行我们称之为数据集；

将数据集整体左移一个单位，令数据集中的每个字母对应它在名字中的下一个字母，这一行我们称之为目标集；

```
1 数据集: [ 'e', 'm', 'm', 'a', '.', 'o', 'l', 'i', 'v', 'i', 'a', '.', 'a', 'v', 'a', '.',  
            'i', 's', 'a', 'b', 'e', 'l', 'l', 'a', '.', 's', 'o', 'p', 'h', 'i', 'a', '.']  
2 目标集: ['e', 'm', 'm', 'a', '.', 'o', 'l', 'i', 'v', 'i', 'a', '.', 'a', 'v', 'a', '.', 'i',  
            's', 'a', 'b', 'e', 'l', 'l', 'a', '.', 's', 'o', 'p', 'h', 'i', 'a', '.']
```

这时，数据集最左侧空出来一个位置，用"."来填充，以此表示开始。同时，这也代表了以不同字母开头的名字的频数。

与此同时，目标集左移后，在末尾空出来一个位置，我们也用"."来填充，以此表示结束。这也代表了以不同字母结束的名字的频数。

```
1  A_data = ["."] + data # 错位对齐  
2  data_B = data + ["."]
```

得到如下

```
1 数据集: ['.', 'e', 'm', 'm', 'a', '.', 'o', 'l', 'i', 'v', 'i', 'a', '.',  
            'a', 'v', 'a', '.', 'i', 's', 'a', 'b', 'e', 'l', 'l', 'a', '.', 's', 'o',  
            'p', 'h', 'i', 'a']  
2 目标集: ['e', 'm', 'm', 'a', '.', 'o', 'l', 'i', 'v', 'i', 'a', '.', 'a',  
            'v', 'a', '.', 'i', 's', 'a', 'b', 'e', 'l', 'l', 'a', '.', 's', 'o', 'p',  
            'h', 'i', 'a', '.']
```

10.2.3. 映射

计算机无法直接运算字符。

我们必须将一个个字母转换为数字。

```
1  set_data = sorted(set(data))  
2  s_to_i = {ch: i for i, ch in enumerate(set_data)}  
3  i_to_s = {i: ch for ch, i in s_to_i.items()}
```

所有符号与数字一一对应，构成27对键值对。

```
1  {'.': 0, 'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5, 'f': 6, 'g': 7, 'h': 8, 'i': 9, 'j': 10, 'k': 1  
    1, 'l': 12, 'm': 13, 'n': 14, 'o': 15, 'p': 16, 'q': 17, 'r': 18, 's': 19, 't': 20, 'u': 21,  
    'v': 22, 'w': 23, 'x': 24, 'y': 25, 'z': 26}
```

10.3. 基于计数的Bigram概率矩阵

这一小节的代码仅为展示神经网络学习的本质，本节中的所有代码删除，也不影响MLP的完整构建。

10.3.1. 计数

还记得我们说过的 26×26 表格吗？加入 "." 之后，行列+1，变为 27×27 。为了直观地感受这些字母对频数分布，我们将其放进这个 27×27 的巨大表格中。

我们使用torch.zeros()，构建一个 27×27 的空表格，从字典中遍历，每发现一种字母对，就将对应索引位置的计数+1。

```

1 N = torch.zeros(27, 27)
2 for s1, s2 in zip(A_data[:,], data_B[:,]):
3     N[s_to_i.get(s1), s_to_i.get(s2)] += 1
4 N = N + 1 # 拉普拉斯平滑

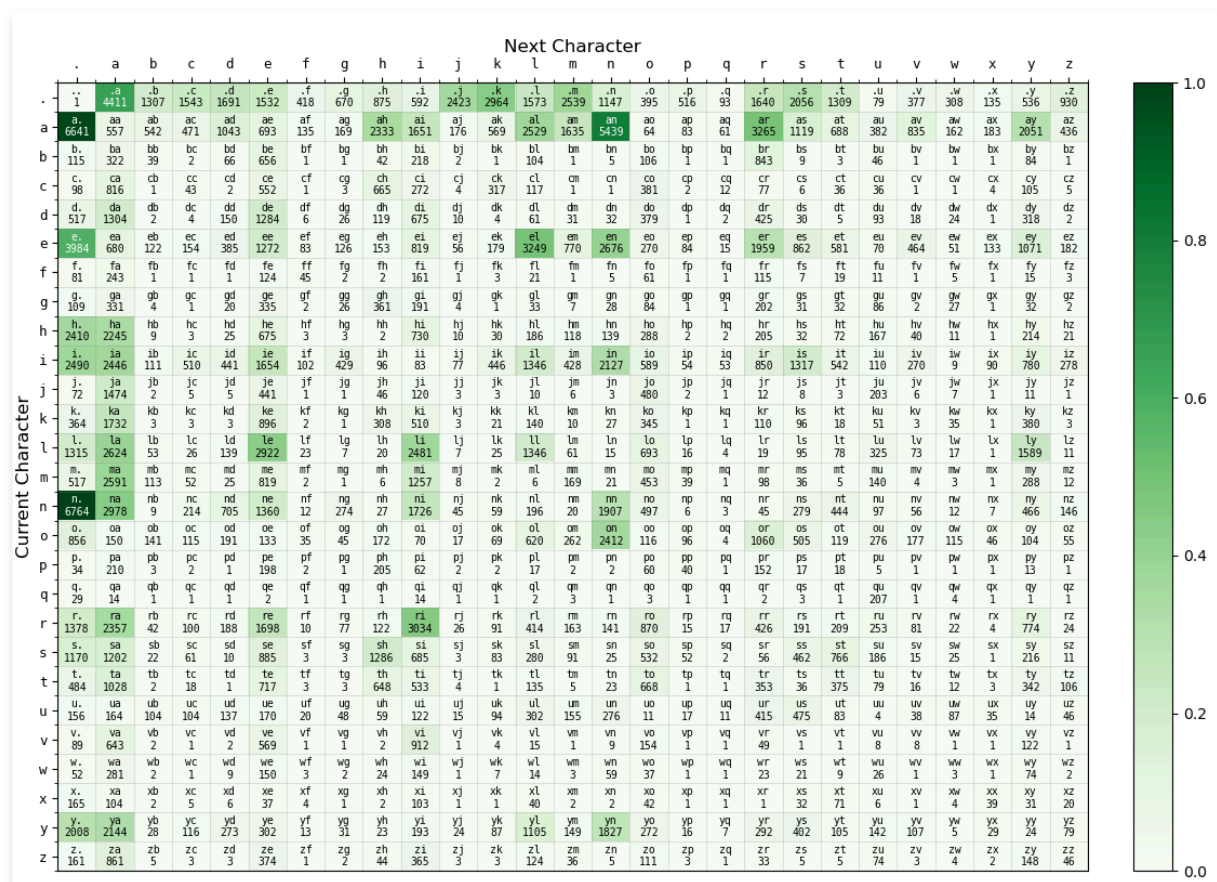
```

现在我们拿到的是关于每种字母对的频数，每一行代表了以某种字母开头的各种情况。

$N = N + 1$ 叫做拉普拉斯平滑，也叫加一平滑，是概率估计中防止零概率问题的最简单方法。

其他平滑方法

- 加 k 平滑: $N+k$, k 可以小于 1
- **Good-Turing 估计**: 根据出现频率动态调整
- **回退法**: 如果 trigram 为零, 退到 bigram, 再退到 unigram



计数

10.3.2. 归一化

例如，我们将第一行的频数加和，得到总频数（32060），用该行每个数据除以总频数，得到一个概率，意味着以这个字母开头的概率是多少。

```

1 N_sum = N.sum(dim=1, keepdim=True)
2 Probability = N / N_sum
3 print(torch.sum(probs, dim=1))

```

同理，我们计算每一行中每一个元素对应的概率。

于是，我们可以根据每一行的概率去寻找下一个字符啦！

示例：

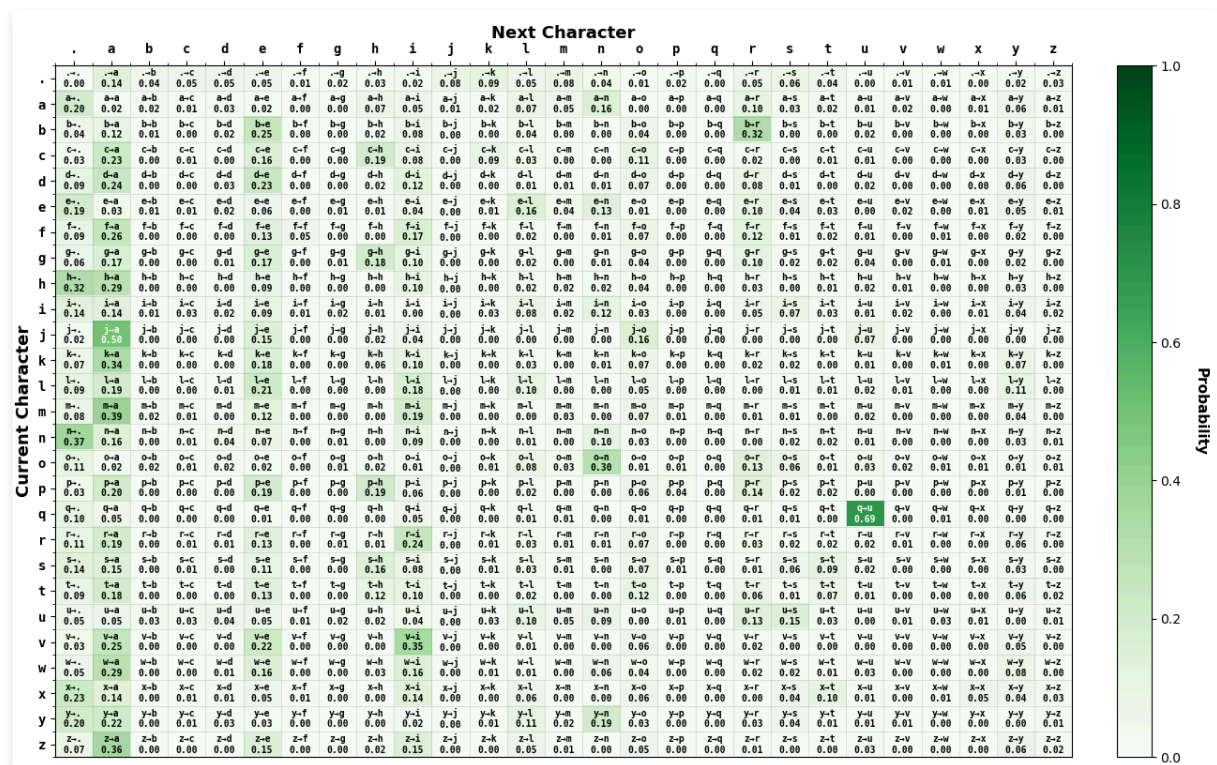
一个名字最有可能的开头是什么？我们去第一行里找，发现 “a”概率最高（0.14），于是第一个字母确定为“ a ”。

“ a ”为起始，我们根据“ a”去找下一个字符，应该看哪一行？答案是：第2行，因为第2行代表了“a”开头的所有可能性。发现 “an” 的可能性有 “0.16” （虽然不高，但也有可能被选中了）。

“ n ”为起始，我们根据“ n”去找下一个字符，应该看哪一行？答案是：第15行，因为第15行代表了“n” 开头的所有可能性。发现 “n.” 的可能性有 “0.37” 。

到了 “.” 就意味着本次预测到头了。

最终输出一个名字 “an”。你可能会大失所望，因为这很不像一个名字。这其实是因为二元预测模型是十分脆弱的，后边我们会建立多字符预测模型，届时生成的名字可能会更像一点！



Character Transition Probability Matrix

检验每行求和是否均为1.顺便大家看一下如下两个打印有什么不同，体会一下keepdim的作用。

```
1 print(torch.sum(probs, dim=1))
2 print(torch.sum(probs, dim=1, keepdim=True))
```

```

1  tensor([1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000,
2          1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000,
3          1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000, 1.0000])
4  tensor([[1.0000],
5          [1.0000],
6          [1.0000],
7          [1.0000],
8          [1.0000],
9          [1.0000],
10         [1.0000],
11         [1.0000],
12         [1.0000],
13         [1.0000],
14         [1.0000],
15         [1.0000],
16         [1.0000],
17         [1.0000],
18         [1.0000],
19         [1.0000],
20         [1.0000],
21         [1.0000],
22         [1.0000],
23         [1.0000],
24         [1.0000],
25         [1.0000],
26         [1.0000],
27         [1.0000],
28         [1.0000],
29         [1.0000],
30         [1.0000]])

```

10.4. 独热编码 (One-Hot)

独热编码 (One-Hot Encoding) 是一种将类别变量转换为机器学习算法易于处理的二进制向量表示法。简单说，就是 **“一个位置是1，其余全是0”**。

这里我们把每个字符映射为一个27维的向量，该字符在词汇表中对应的索引位置为1，其余位置为0。

举例：

“b” 在映射关系中对对应 “2” ，则索引位置为 “2” 时，该位置为1。

“z” 在映射关系中对对应 “27” ，则索引位置为 “27” 时，该位置为1。

“b” 和 “z” 的独热编码分别为：

```

1  a:[0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
2  z:[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]

```

在深度学习中，独热编码通常只是一闪而过的中间态，随后会被更强大的**嵌入层**所取代。这里仅作为教学演示。

我们取列表中的前10个字符为一组，看一下它的独热编码什么样，后续处理数据时，我们也以这10个为例。

第一步：找出每个字符所在的索引位置，组合前10个字符为向量。

```

1  xs, ys = [], []
2  for ch1, ch2 in zip(A_data[:10], data_B[:10]): # 取前10个字符为例
3      ix1 = s_to_i[ch1]
4      ix2 = s_to_i[ch2]
5      xs.append(ix1)
6      ys.append(ix2)
7
8  xs = torch.tensor(xs) # torch.nn.functional只接受PyTorch张量 (Tensor)，所以必须将列表转为张量形式
9  ys = torch.tensor(ys) # 同上

```

```

1  xs: tensor([ 0,  5, 13, 13,  1,  0, 15, 12,  9, 22])
2  ys: tensor([ 5, 13, 13,  1,  0, 15, 12,  9, 22,  9])

```

第二步：转换为独热编码

```

1  import torch.nn.functional as F
2  xenc = F.one_hot(xs, num_classes=27).float()
3  yenc = F.one_hot(ys, num_classes=27).float()

```

如下为xenc，“1”的位置分别索引于 [0, 5, 13, 13, 1, 0, 15, 12, 9, 22] 。

```

1  tensor([[1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
2         [0., 0., 0., 0., 0., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
3         [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
4         [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
5         [0., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
6         [1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
7         [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
8         [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
9         [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
10        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
11        [0., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.]])

```

朋友们，注意到了吗？独热编码中的数字后带有一个小数点“.”，这表明该数字的类型为浮点数，用于张量计算的数字必须为浮点数。

为什么是浮点数？

在神经网络中，每一层的输出并不只是最终的分类标签，它是一个置信度。必须是一个有连续变化的模拟量，整型的世界是非黑即白的。如果一个神经元只能输出 0 或 1，它就会做出一个极其生硬的判决，把所有细腻的可能性都粗暴地抹除了。在训练中，必须严格使用浮点数，而推理中，可以把权重的浮点值映射到整数区间，并用整数运算代替浮点运算来做前向推理。

到了这里，你可以将我们在前边翻来覆去提到的数据矩阵 X 等同于这里的xenc了！

10.5. 前向传播——线性变换+非线性激活

10.5.1. 线性变换（加权求和 + 偏置）

用 `torch.randn()` 构建随机权重矩阵和偏置矩阵。

```
1 g = torch.Generator().manual_seed(123987) # 固定随机种子可以确保每次生成的w和b都一样。
2 W1 = torch.randn((27, 27), generator=g)
3 b = torch.randn((1, 27), generator=g)
4 logits1 = xenc @ W1 + b
```

权重的作用：就像调音台上的推子。放大重要的信号，缩小或反转不重要的信号。

偏置的作用：像是一扇门的“门槛高低”。如果数据本身数值很小，激活门很难打开，偏置就是那个把曲线左右平移，让它能刚好卡住数据的螺丝。

结果：得到了净输入，它还是这些神经元的原始电压信号，可正可负，可大可小。

权重为什么需要打破对称性？

权重需要打破对称性，是为了让同一个隐藏层里的不同神经元，能够学到不同的特征。

如果所有权重初始化成一样的值，就意味着，无论你有多少个隐藏神经元，**实际上都等效于只有一个神经元**。多个神经元变得完全冗余，网络容量被严重浪费。

为什么多数情况下使用`randn()`（高斯分布）采样？

- 正态分布更“无偏”。在没有任何先验知识的情况下，正态分布是“最自然”的选择，原因来自两个著名的理论：**中心极限定理和最大熵原理**。
- 从反向传播角度看：方差的稳定传递。1.因为两个独立正态变量的乘积仍然是正态（在特定条件下），方差传播公式更简洁。2.如果权重初始是正态的，经过线性变换后，输出趋近于正态（中心极限定理再次起作用），这使得各层的信号分布保持一致，更容易控制和理论分析。

在神经网络和机器学习中，**logits** 是指模型最后一层输出的**未归一化的原始预测值**。

10.5.2. 非线性激活

这里我们使用`tanh`激活函数。它把logits中的每个元素压缩到到 $(-1, 1)$ 之间。

```
1 h = torch.tanh(logits1)
```

如下是`tanh`转换后的张量，形状是: 10×27 .它们的每个元素都在 $(-1, 1)$ 之间。

```

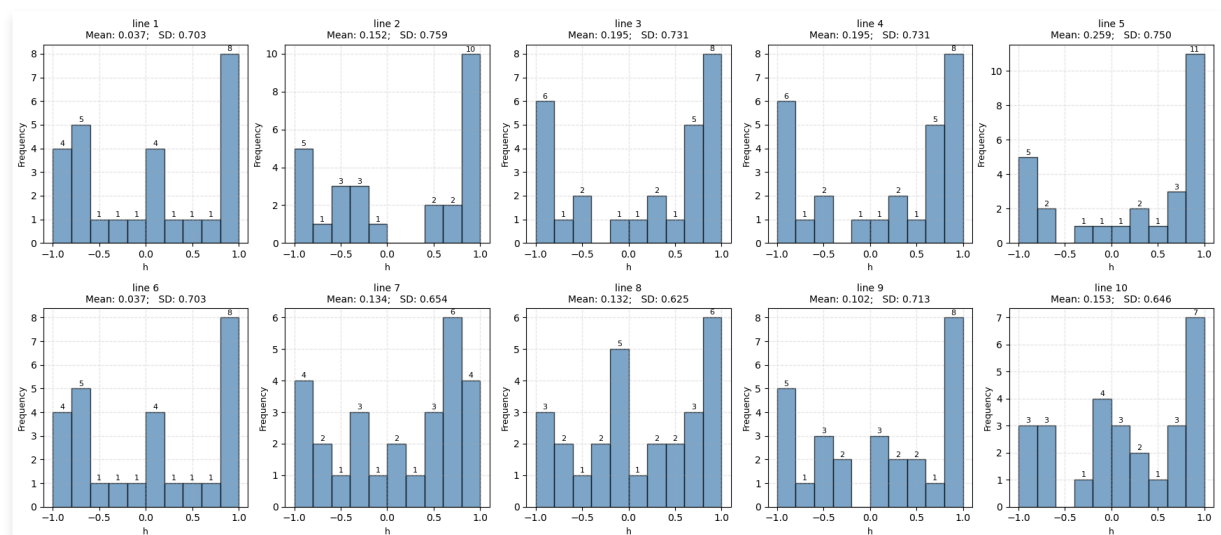
1  tensor([[ 0.84,  0.05, -1.00, -0.91,  0.80,  0.70, -0.99,  0.98, -0.62, -0.
    61,  0.98, -0.64,  0.13,  0.30, -0.20, -0.16, -0.76,  0.97,  0.84,  0.87, -
    0.61, -0.94,  0.14, -0.51,  0.85,  0.08,  0.44],
2      [ 0.75, -0.30, -0.90, -0.99,  0.56, -0.60, -0.55, -0.31,  0.92,  0.
    93, -0.86,  0.96, -0.23,  0.94,  0.98,  0.97,  0.67, -0.84,  0.96,  0.99, -
    0.71, -0.49,  0.91, -0.12,  0.88, -0.91,  0.49],
3      [ 0.25, -0.84, -0.86, -0.97,  0.96,  0.86,  0.74,  0.93, -0.88, -0.
    66,  0.54, -0.41,  0.80,  0.94,  0.98,  0.60, -0.02,  0.90,  0.82,  0.92,
    0.20, -0.96,  0.77,  0.30, -0.42, -0.90,  0.71],
4      [ 0.25, -0.84, -0.86, -0.97,  0.96,  0.86,  0.74,  0.93, -0.88, -0.
    66,  0.54, -0.41,  0.80,  0.94,  0.98,  0.60, -0.02,  0.90,  0.82,  0.92,
    0.20, -0.96,  0.77,  0.30, -0.42, -0.90,  0.71],
5      [-0.93, -0.14, -1.00, -0.99,  0.61,  0.35, -0.66,  1.00, -0.79,  0.
    09,  0.41, -0.29,  0.37,  0.99,  0.99,  0.83,  0.91,  0.95,  0.98,  0.99, -
    0.87,  0.86,  0.73,  0.90,  0.93, -0.87,  0.65],
6      [ 0.84,  0.05, -1.00, -0.91,  0.80,  0.70, -0.99,  0.98, -0.62, -0.
    61,  0.98, -0.64,  0.13,  0.30, -0.20, -0.16, -0.76,  0.97,  0.84,  0.87, -
    0.61, -0.94,  0.14, -0.51,  0.85,  0.08,  0.44],
7      [ 0.76, -0.28, -0.92,  0.13, -0.50, -0.85, -0.26,  0.89, -0.94,  0.
    82, -0.93, -0.76, -0.09,  0.66,  0.99,  0.40,  0.56,  0.68,  0.74,  0.73,
    0.07,  0.71, -0.62,  0.35, -0.24,  0.90,  0.60],
8      [ 0.35, -0.33, -0.86, -0.88,  0.42, -0.10, -0.59,  0.94, -0.06, -0.
    97, -0.03, -0.22,  0.93,  0.65,  0.96, -0.80, -0.03,  0.77,  0.83,  0.87,
    0.41, -0.10, -0.61,  0.31,  0.61,  0.12,  0.99],
9      [-0.41,  0.16, -0.43,  0.44, -0.32,  0.69, -0.90,  0.94, -0.76,  0.
    87, -0.83,  0.59,  0.10,  0.99,  0.88, -0.93,  0.90, -0.32,  1.00,  0.99,
    0.08,  0.31, -0.59, -0.98,  0.25, -0.93,  0.98],
10     [ 0.93,  0.95, -0.76, -0.91, -0.13, -0.77, -0.87,  0.96,  0.37,  0.
    41,  0.20,  0.03,  0.88,  0.65,  0.90,  0.68,  0.74, -0.78,  0.83,  0.18, -
    0.02, -0.84, -0.17, -0.13,  0.96, -0.26,  0.07]])

```

10.5.3. 潜在危机——梯度消失

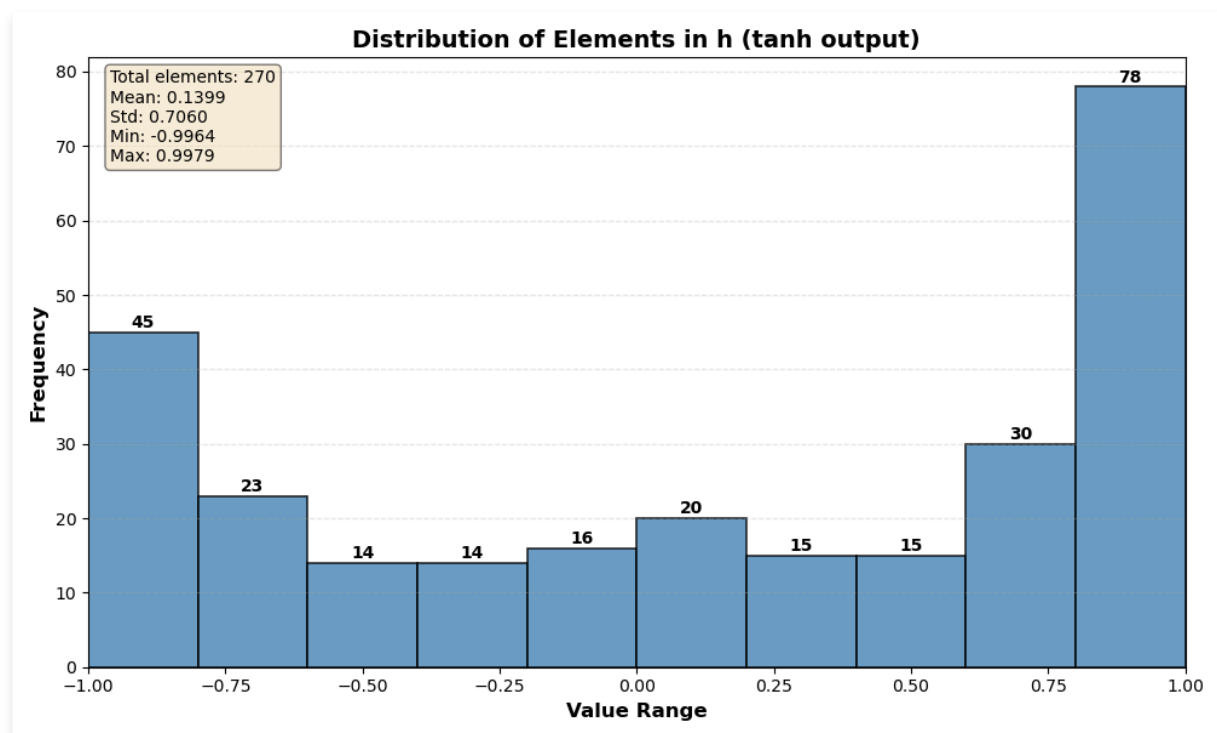
我们来看一个很有趣的分布现象。

我们把h张量中每一行的数据统计一下：将 $(-1, 1)$ 切割为10个区间，统计每个区间内的频数并绘制为直方图。



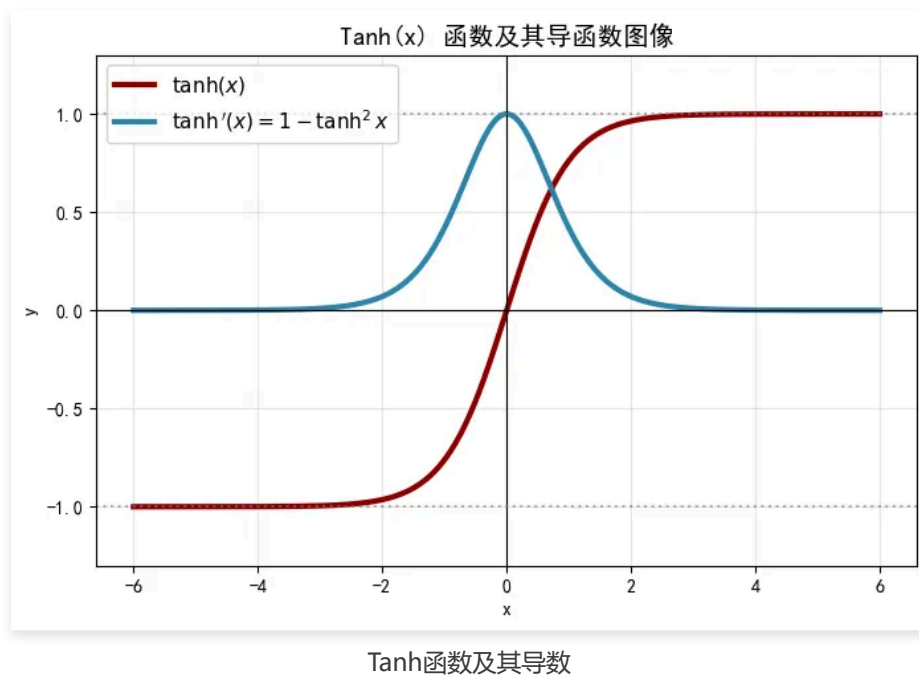
Histogram of h value distribution for each sample

你是否发现这些数字分布呈现“两头高中间低”的趋势。如果还不够清楚，我们将h中所有元素统计出来，同上述方法绘制直方图。



其中，分布在 $(-1, -0.75)$ 和 $(0.75, 1)$ 之间的数字占比高达45.6%！

这意味着经过Tanh的转换，大多数数字分布在Tanh函数的两段尾部，而这一段的导数趋近于0。



在反向传播过程中，该区域的局部梯度趋近于0，导致传递给参数的梯度也变得极小，参数几乎不会更新。

你可以想象，经过多层 Tanh 处理后，来自尾部的梯度在反向传播中会被每一层的局部小梯度不断缩小（梯度会是 $0.1 \times 0.1 \times 0.1 \dots$ ，迅速指数级衰减），最终几乎完全消失，导致浅层参数无法得到有效调整。

这就是深度学习历史上臭名昭著的“**梯度消失**”！

留个悬念，等后边我们在解决这个问题。

10.6. 前向传播——从隐藏层到输出层

现在，我们把隐藏层的输出 (h)当成新的输入，扔给输出层。

10.6.1. 线性变换

```
1 W2 = torch.randn((27, 27), generator=g)
2 b2 = torch.randn((1, 27), generator=g)
3 logits2 = h @ W2 + b2
```

logits2结果：

```
1 tensor([[ -7.40,  -3.01,  -0.88,  -1.31,  -0.95,   2.51,  -0.81,  -2.93,
   0.83,  -6.20,   2.03,  -5.94,   1.09,  -5.75,   0.50,  -3.22,   1.90,   0.1
   9,  -0.06,   3.73,  -1.34,  -5.64,   3.82,  -1.73,   1.61,  -0.98,  -0.70],
2         [ -1.56,   0.43,  -2.95,   0.14,  -0.62,  -2.31,   0.16,   7.11,
   4.76,   4.69,   7.86,  -5.43,   2.58,  -3.69,   2.49,   1.54,  -2.43,  -2.5
   2,  -1.03,   0.54,  -3.34,  -0.40,   8.40,   4.48,   5.38,   0.19,  -7.67],
3         [ -8.50,  -3.22,  -1.90,  -0.56,  -1.81,   0.52,   0.29,   1.24,  -
   1.03,   0.53,  -2.03,  -4.08,   4.91,  -3.48,   1.60,  -2.75,   6.55,  -1.5
   5,   4.80,   0.23,  -1.61,  -4.60,   2.67,   4.09,   0.26,  -2.45,  -5.14],
4         [ -8.50,  -3.22,  -1.90,  -0.56,  -1.81,   0.52,   0.29,   1.24,  -
   1.03,   0.53,  -2.03,  -4.08,   4.91,  -3.48,   1.60,  -2.75,   6.55,  -1.5
   5,   4.80,   0.23,  -1.61,  -4.60,   2.67,   4.09,   0.26,  -2.45,  -5.14],
5         [-11.04, -11.08,  -6.96,  -1.62,  -3.21,  -4.79,  -1.14,   2.24,
   2.12,   3.43,   5.13,  -6.70,   4.64,  -1.47,  -4.04,   0.15,   7.73,  -0.4
   0,   2.38,   4.18,  -1.30,  -6.05,   8.20,   3.98,   4.13,  -2.15,  -9.29],
6         [ -7.40,  -3.01,  -0.88,  -1.31,  -0.95,   2.51,  -0.81,  -2.93,
   0.83,  -6.20,   2.03,  -5.94,   1.09,  -5.75,   0.50,  -3.22,   1.90,   0.1
   9,  -0.06,   3.73,  -1.34,  -5.64,   3.82,  -1.73,   1.61,  -0.98,  -0.70],
7         [ -6.00,  -2.10,  -2.32,  -6.47,   4.47,  -0.11,  -7.60,  -0.19,
   5.69,  -1.67,   4.17,  -0.11,  -0.40,  -2.85,   0.27,  -3.58,   3.27,  -1.5
   0,   2.97,   1.48,  -2.64,  -3.02,   3.79,   2.42,   3.39,   2.66,  -0.41],
8         [ -5.50,  -1.86,  -1.74,  -2.08,   1.17,   3.43,  -3.14,   0.11,
   3.53,  -1.31,   2.68,   0.31,  -1.77,  -4.12,  -1.17,  -5.23,   3.48,  -2.4
   1,  -0.58,   1.25,  -1.60,  -5.34,   2.50,   5.01,   1.54,  -1.13,  -4.68],
9         [ -3.79,  -3.04,  -2.17,  -7.58,   0.69,  -6.18,  -7.58,   5.43,
   1.66,   5.64,   5.92,   1.25,  -2.54,  -2.66,   3.49,  -1.61,   3.83,  -3.2
   0,   1.47,   4.78,  -0.73,  -1.20,   2.02,   2.14,  -1.90,  -2.65,   0.81],
10        [ -4.27,   3.36,   0.69,  -2.62,   3.11,   1.73,  -1.19,   0.49,
   2.95,  -1.44,   5.98,  -1.98,   6.44,  -4.93,  -2.44,  -1.97,   2.52,  -1.3
   9,  -3.82,   1.40,   0.34,  -4.33,   2.15,   2.36,   1.83,  -2.26,  -1.3
   7]])
```

logits 中依然可正可负。

10.6.2. Softmax

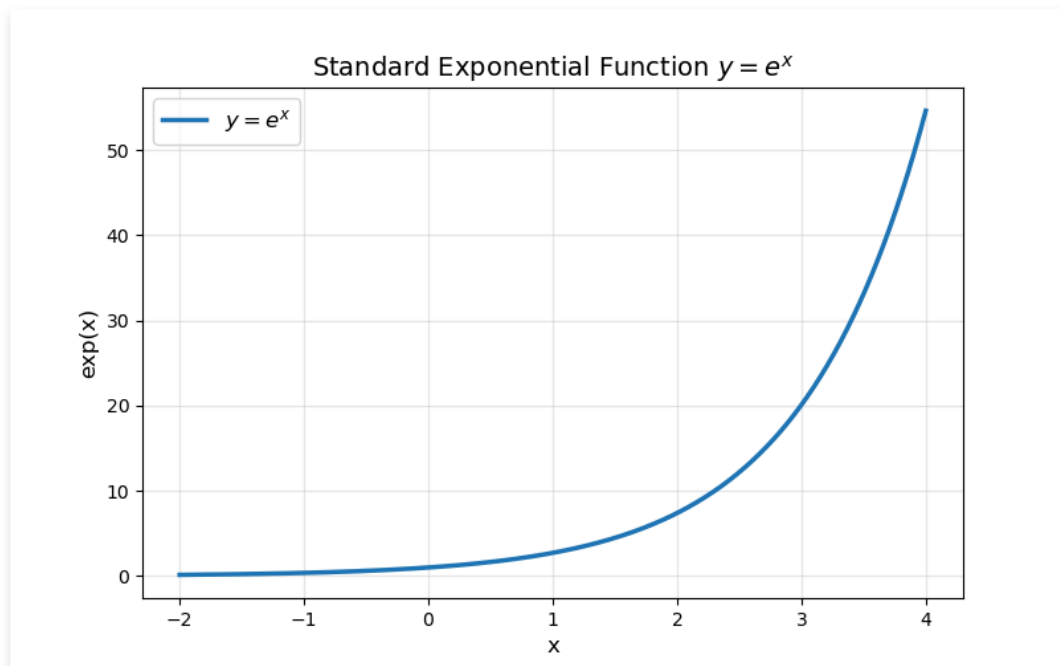
我们这里的MLP是一个**多分类问题**，即：区分下一个字符是27种里的哪一个。

回忆我们上边讲过的“归一化”。

我们把每行字符对除以每一行的计数总和，得到一个概率。然后，我们拿这个概率去预测下一个字符。这里我们借用这个思想来完成最后一击。

既然我们需要用概率来表示，那么logits2中的每个数字都应该时正数（没有概率是负的！）。

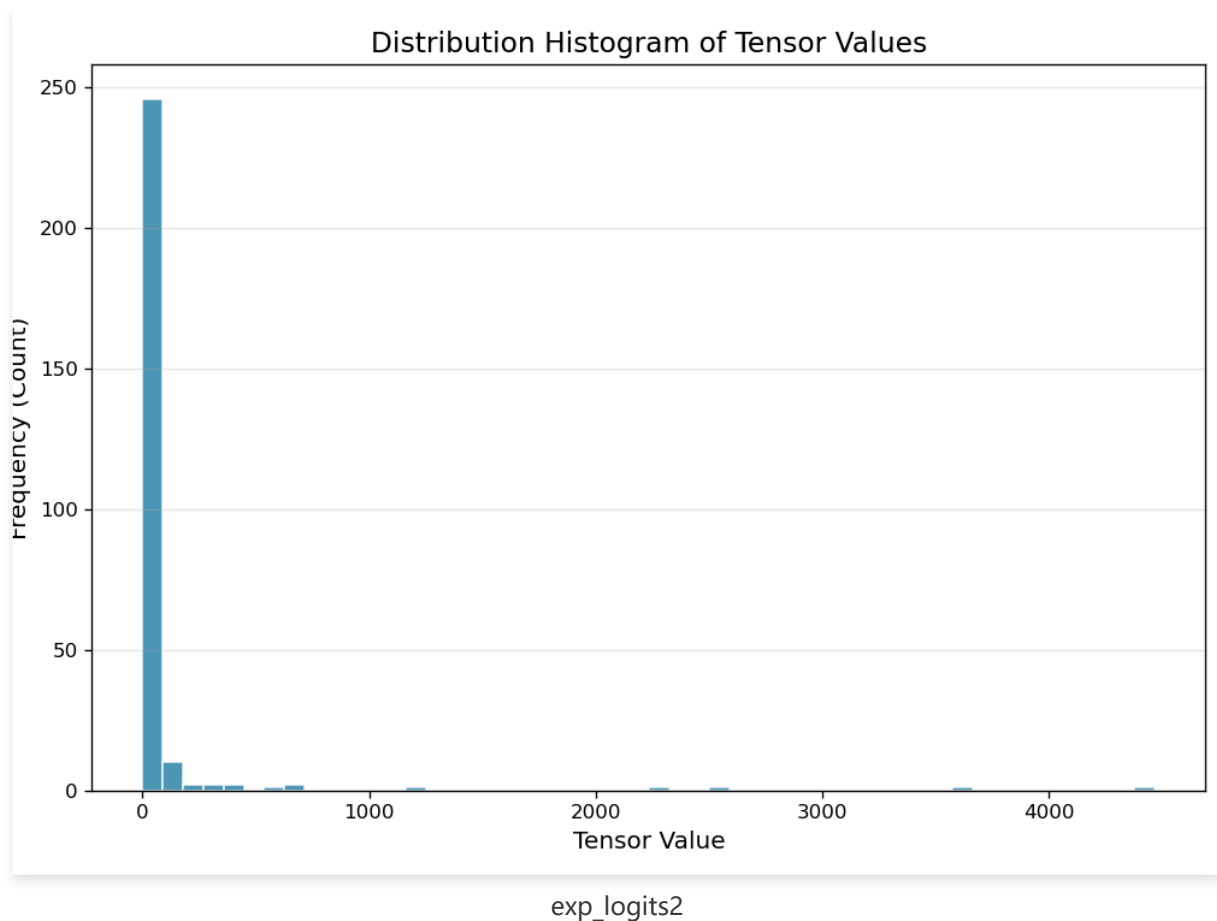
所以我们使用一个转换器： $\exp(x)$ ，它可以将负数转换为正数，且正数依然为正数。**指数函数**是连接 `logits`（原始得分，可正可负）和 **概率**（非负且和为1）之间最自然、数学上最优雅的桥梁。



指数函数 (exp)

```
1 exp_logits2 = torch.exp(logits2)
```

exp_logits2 的图像也非常有意思：



用exp_logits2里每一行的数据分别除以对应行的总和。

```
1 probs = exp_logits2 / torch.sum(exp_logits2, dim=1, keepdims=True)
2 # 后边为了方便手动反向传播的计算，我们将这行代码拆解为如下四行（名称沿用自Karpathy）：
3 counts = torch.exp(logits2)
4 counts_sum = torch.sum(counts, dim=1, keepdims=True)
5 counts_sum_inv = (counts_sum + 1e-8) ** -1
6 probs = counts * counts_sum_inv
```

注意，这里counts_sum需要增加一个极小值，来保证计算中不会出现inf/NaN.

这里的probs并非真正的概率，我叫它伪概率，因为它并非来自真正的计数，但形似概率的计算方式（打印的数据中包含0.0000，其实是后边的小数被隐藏了）。

```

1  tensor([[0.0000,0.0004,0.0032,0.0021,0.0030,0.0937,0.0034,0.0004,0.0175,0.0
000,0.0585,0.0000,0.0228,0.0000,0.0126,0.0003,0.0512,0.0092,0.0072,0.3178,
0.0020,0.0000,0.3484,0.0014,0.0383,0.0029,0.0038],
2      [0.0000,0.0002,0.0000,0.0001,0.0001,0.0000,0.0001,0.1388,0.0132,0.0
123,0.2918,0.0000,0.0015,0.0000,0.0014,0.0005,0.0000,0.0000,0.0000,0.0002,
0.0000,0.0001,0.5050,0.0099,0.0246,0.0001,0.0000],
3      [0.0000,0.0000,0.0001,0.0005,0.0002,0.0016,0.0013,0.0033,0.0003,0.0
016,0.0001,0.0000,0.1296,0.0000,0.0047,0.0001,0.6664,0.0002,0.1160,0.0012,
0.0002,0.0000,0.0138,0.0573,0.0012,0.0001,0.0000],
4      [0.0000,0.0000,0.0001,0.0005,0.0002,0.0016,0.0013,0.0033,0.0003,0.0
016,0.0001,0.0000,0.1296,0.0000,0.0047,0.0001,0.6664,0.0002,0.1160,0.0012,
0.0002,0.0000,0.0138,0.0573,0.0012,0.0001,0.0000],
5      [0.0000,0.0000,0.0000,0.0000,0.0000,0.0000,0.0000,0.0015,0.0013,0.0
048,0.0264,0.0000,0.0162,0.0000,0.0000,0.0002,0.3541,0.0001,0.0017,0.0101,
0.0000,0.0000,0.5655,0.0084,0.0096,0.0000,0.0000],
6      [0.0000,0.0004,0.0032,0.0021,0.0030,0.0937,0.0034,0.0004,0.0175,0.0
000,0.0585,0.0000,0.0228,0.0000,0.0126,0.0003,0.0512,0.0092,0.0072,0.3178,
0.0020,0.0000,0.3484,0.0014,0.0383,0.0029,0.0038],
7      [0.0000,0.0002,0.0002,0.0000,0.1451,0.0015,0.0000,0.0014,0.4887,0.0
003,0.1077,0.0015,0.0011,0.0001,0.0022,0.0000,0.0437,0.0004,0.0324,0.0073,
0.0001,0.0001,0.0732,0.0187,0.0493,0.0238,0.0011],
8      [0.0000,0.0005,0.0006,0.0004,0.0111,0.1057,0.0001,0.0038,0.1170,0.0
009,0.0503,0.0047,0.0006,0.0001,0.0011,0.0000,0.1118,0.0003,0.0019,0.0119,
0.0007,0.0000,0.0420,0.5172,0.0160,0.0011,0.0000],
9      [0.0000,0.0000,0.0001,0.0000,0.0018,0.0000,0.0000,0.0000,0.2046,0.0047,0.2
519,0.3347,0.0031,0.0001,0.0001,0.0294,0.0002,0.0411,0.0000,0.0039,0.1069,
0.0004,0.0003,0.0068,0.0076,0.0001,0.0001,0.0020],
10     [0.0000,0.0252,0.0017,0.0001,0.0195,0.0049,0.0003,0.0014,0.0166,0.0
002,0.3464,0.0001,0.5449,0.0000,0.0001,0.0001,0.0109,0.0002,0.0000,0.0036,
0.0012,0.0000,0.0075,0.0093,0.0055,0.0001,0.0002]])

```

还记得我们在独热编码中打印的另一个编码吗？

```

1  ys: tensor([ 5, 13, 13,  1,  0, 15, 12,  9, 22,  9])

```

10.6.3. 交叉熵

我们的目标是通过一个标量loss，通过寻找loss的最小值来训练MLP。

聪明的你可能想到了，我们之前学过均方误差（MSE），通过MSE构建了一个二次方程，进而找到最小值。其实MSE 更适合前几章那样的回归问题。

这里我们需要介绍你另一位重量级嘉宾，也就是我在一开始向你提起、却始终未曾露面的：**交叉熵（Cross Entropy）**。

在MLP中，交叉熵 + Softmax 的组合在数学上和实践上都是分类问题的最优选择。

交叉熵 (Cross Entropy) :

$$H(p, q) = - \sum_x p(x) \log q(x)$$

- x : 随机变量所有可取值 (在任务中为27 个字符的数字映射) 。
- $p(x)$: 真实分布在取值 x 处的概率。例如在名字 “emma” 中, x 表示取 “e” 这个事件, 则后边真实值应该为 “m” , 我们经过一系列处理, 改行 “m” 所在的索引位置为1, 其余自动归0.

$$p(x) = \begin{cases} 1 & x = y_i \text{ (该样本真实下一个字符)} \\ 0 & x \neq y_i \end{cases}, p(x) \text{ 等价于上述 } y_{\text{enc}}.$$

- $q(x)$: 模型 Softmax 输出、 x 类别的预测概率 (代码 `probs` 张量, `probs[i, x]` = 第 i 个样本预测为 x 字符的概率)

交叉熵是来自信息论领域的强大外援, 但看起来令人头大, 别害怕, 我带你一点一点了解他。

10.6.3.1. 真实概率 $p(x)$

还记得上边我们的`ys`和`yenc`变量吗?

`ys`的10个元素来自于 X 中10个字符的预测值索引。

而`yenc`是`ys`的独热编码。

```
1  ys:  ([ 5, 13, 13,  1,  0, 15, 12,  9, 22,  9])
2  yenc: ([[0., 0., 0., 0., 0., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
3         [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 0., 0., 0., 0., 0., 0.],
4         [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 0., 0., 0., 0., 0., 0.],
5         [0., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
6         [1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
7         [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 0., 0., 0., 0.],
8         [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
9         [0., 0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
10        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
11        [0., 0., 0., 0., 0., 0., 0., 0., 1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.]])
```

$p(x)$ 表示的是真实分布在取值 x 处的概率。

我们拿`yenc`的第一行来解释:

在数据集 “emma” 中, 第一个样本是 “.” , 对应的真实标签是 “e”, “e”的映射值是5, 所以`ys`中第一个索引是5, 我们将5进行独热编码, 于是得到了`yenc`的第一行, 这是它的由来。

这意味着, 第一行这个真实数据告诉我们, 在给出 “.” 的情况下, 其下一个字符是 “e”的概率为1, 同时意味着: `yenc`第一行索引位置为5的地方应该是1!

乘法是交叉熵的灵魂，它代表的是：**求加权平均**。

在概率论里，求**平均值（期望）**的公式长这样： $E[X] = \sum_i p_i x_i$ 。

和交叉熵是同一个结构： $\text{交叉熵} = - \sum \text{真实概率} p(x) \times \text{该事件的信息量} \log q(x)$

所以乘号的意思是：**用真实概率做权重，对"模型以为的信息量"求加权平均。**

交叉熵是用**模型的置信度**去衡量**真实世界**带给模型的惊讶程度。

交叉熵衡量的是**从真实世界出发**，对模型眼中的世界进行期望评估。

计算后得到总的信息量的期望，随着权重不断更新，最终期望越来越低，这表明模型对数据的预测越来越准确，达到一种见怪不怪的程度，就好像它记住了这些内容。

10.6.4. 像素级推导交叉熵的计算

yenc中 $p(x_{ij})$ 表示真实世界的发生事件 j 的真实概率;probs中 $q(x_{ij})$ 表示大模型目对的发生事件 j 的置信度。

我作为严厉的训练师，

指着probs的第1行第2个数字，对大模型说：好啊，既然你觉得后边出现“a”的概率是0.0004，那我就告诉你，其实真实数据表明，这个位置是“a”的概率是0.

...

指着probs的第1行第6个数字，对大模型说：好啊，既然你觉得后边出现“e”的概率是0.0937，那我就告诉你，其实真实数据表明，这个位置是“a”的概率是1.

...

我们从头到尾把每个数字都与这个自以为是的大模型做了对齐。

```
1  对应预测字母: [ '.', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j',
   'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x',
   'y', 'z']
2  置信度       : [0.0000,0.0004,0.0032,0.0021,0.0030,0.0937,0.0034,0.0004,0.0175,0.0000,0.0585,0.000
   0,0.0228,0.0000,0.0126,0.0003,0.0512,0.0092,0.0072,0.3178,0.0020,0.0000,0.3484,0.0014,0.0383,0.0
   029,0.0038]
3  真实概率     : [ 0.,    0.,    0.,    0.,    0.,    1.,    0.,    0.,    0.,    0.,    0.,
   0.,    0.,    0.,    0.,    0.,    0.,    0.,    0.,    0.,    0.,    0.,    0.,
   0.,    0.]
```

大模型也根据我的信息把数据做计算： $p(a|.) \times -\log q(a|.) = 0 \times -\log(0.0004) = 0 ; \dots$

$p(e|.) \times -\log q(e|.) = 1 \times -\log(0.0937) = 2.3678 \dots$

看起来运算量很大，但你要注意到，大部分（260个）都是0，只有索引部分（10个）才是非零值。

最终拿到10个样本，共 $10 \times 27 = 270$ 个数据，并将他们每一行求和，得到一个总的交叉熵：

$$H_i(p, q) = - \sum_{j=1}^{27} p(x_{ij}) \cdot \log q(x_{ij})$$

这样的交叉熵一共10个，它们构成一个一阶张量。

我们在代码中用两行就可以实现：

```
1  log_probs = -1 * yenc * torch.log(probs) # 显式 p(x) 相乘,所有索引位置为1的保留下来,为0的地方消
   失。
2  sum = torch.sum(log_probs, dim=1)
```

10.6.5. 最后一击——求loss

loss必须是个标量，那么我们如何将上述交叉熵构成的张量变成一个标量呢？

答案是求均值。

用一行代码搞定：

```
1 L = torch.mean(sum)
```

到此，前向传播结束。

请君移步下一章。

11. 构建BigramMLP——反向传播

11.1. 反向传播的另一种视角

我们还是看ys和yenc。

ys的10个元素来对应 X 中10个字符的真实预测值的索引。yenc是ys的独热编码。

从yenc看，除了要索引位置的数值是1，其余位置都是零，所以只用看ys就知道哪个位置重要。

进一步，用ys索引probs中每行的数值的过程就像是在挑选一个幸运儿！

probs中每行都有一个幸运儿，它们所在的位置天生肩负神圣使命！

11.1.1. 《生命法典》

我们先拿probs中的第一行来解释这个过程：

```
1 line1:[0.0000,0.0004,0.0032,0.0021,0.0030,0.0937,0.0034,0.0004,0.0175,0.0000,0.0585,0.0000,0.0228,0.0000,0.0126,0.0003,0.0512,0.0092,0.0072,0.3178,0.0020,0.0000,0.3484,0.0014,0.0383,0.0029,0.0038]
```

ys就像氮星中定义命运的《生命法典》。它规定了第一行中，第5个元素（从0计数）是注定不平凡的，需要肩负起重任！我们找到这个数字：0.0937！

同理，我们根据ys，依次将probs中每一个幸运儿找到：

```
1 O_0 = probs[torch.arange(10), ys]
```

他们是：

```
1 tensor([ 0.093689, 0.000003, 0.000030, 0.000038, 0.000000, 0.000307, 0.001108, 0.000923, 0.006763, 0.000208])
```

虽然他们天资过人，但它们的先天自身条件不够完美，因为按照真实值来看，这些幸运儿对应的概率应该是1，可是在这里，它们都远小于1。

它们经过取负对数，得到：

```
1 tensor([ 2.3678, 12.7778, 10.4299, 10.1734, 19.8066,  8.0898,  6.8055,  6.9879,  4.9962,  8.4802])
```

求平均值，得到：

```
1 L = tensor(9.0915)
```

11.1.2. 肩负反向传播的天命人

这些幸运儿因为“天资聪慧”，天生被选中。

却因自身先天劣势，导致最终的loss过高，有9.0915之大！

但严厉的loss教会会把最好的资源分配给这些幸运儿，保证他们得到最充足的梯度资源。

希望这是你在神经网络发现的另一个人生哲学：选择往往比努力更重要，极致成功的路，往往需要更多运气。

下一节我们开始反向传播。

11.2. 广播机制

在开始反向传播前，我需要先向各位介绍一下广播（broadcasting）。

11.2.1. 概述

广播并非纯数学概念，而更多的是工程上的技巧。

在深度学习中，在前向传播中，我们已经遇到样本（Batch）与偏置（Bias）相加的场景。

一个形状为（10, 27）的权重矩阵，要加上一个形状为（1, 27）的偏置向量。

你可能并未注意到他们在代码中报错，那是因为计算机在运算时自动将其隐式地广播了。

- 数学上：不同维度的矩阵是不能直接进行元素对应运算（Element-wise）的。
- 工程上：如果每次运算都手动复制数据去把矩阵填满，会浪费大量的内存和 CPU/GPU 算力。

为了解决这个矛盾，计算机引入了隐式广播机制。

11.2.2. 隐式广播

当我们在计算机中生成一个（10, 27）的张量，内存中的形态并非终端显示的那样，是一个相同形状的大矩阵。

在计算机内存中，任何多维张量（Tensor）本质上都是一段**一维连续的内存块**。计算机为了知道怎么把一维数组看成多维矩阵，需要依靠两个核心元数据：

- **Shape（形状）**：每个维度的大小。
- **Strides（步长）**：在内存中移动到该维度下一个元素需要跨过的**字节数**（或者元素个数）。

举个例子，一个形状为（3, 2）的标准矩阵，在内存中是一维排列的：`[A1, A2, B1, B2, C1, C2]` 它的步长是（2, 1）——意味着切换到下一行要跨过 2 个元素，切换到下一列要跨过 1 个元素。

当一个形状为 `(10, 1)` 的列向量想要隐式广播成 `(10, 27)` 时，计算机内部发生了什么？

1. **维度检查**：计算机从后往前检查两个张量的维度。如果某个维度满足 `1` 或者是合法的倍数关系，就可以广播。
2. **内存虚化**：计算机完全没有克隆数据，它只是把这个张量的元数据偷偷改了：
 - **Shape** 变成了 `(10, 27)`
 - **Strides** 变成了 `(1, 0)`

重点就在这个 `0` 步长！ 当矩阵运算执行到第 2 列、第 3 列.....直到第 27 列时，因为列维度的步长是 `0`，计算机在内存指针移动时**原地不动**，始终读取的还是第 1 列的那个数字。

隐式广播是一场数据魔术。计算机通过把内存步长设为 0，让同一个内存在空间中‘分身’了 27 次，既实现了矩阵对齐，又实现了 **0 内存开销**！

11.2.3. 手动广播

为了向你展示完整的反向传播过程，我们在反向传播时会通过代码实现手动广播。为此我们牺牲了计算性能，但磨刀不误砍柴工，一切都值。

后边当你看到：

```
1 torch.ones_like()
```

那大概是在进行手动广播了。目的同样是讲维度对齐，方便计算。

11.3. 反向传播 (Backpropagation)

在本文一开始，我们就使用极大篇幅来向你介绍反向传播的原理。

走到这里，想必这利剑之刃早已锋利无比，我们就从 L 开始，看一下这梯度是如何从劈开层层网络，直抵权重之心的！

11.3.1. 梯度之源

分配梯度之前，必须要有梯度可以分配。

最开始的梯度来自 L 对它自己的导数： $L = L$ ， $\frac{\partial L}{\partial L} = 1$ （为保持连贯性与严谨性，本节后边的数学表达全部使用 ∂ ，而不是 d ）。

```
1 dL = 1 # 在神经网络中，dxxx代表 $\partial \text{loss} / \partial \text{xxx}$ ，这与数学中的偏微分书写规则不一致，只是一种约定俗成的写法。
```

11.3.2. $L \rightarrow \text{sum_loss}$

L 教父会平等的对待每一个幸运儿，将梯度均匀分配下去。

因为 $L = \frac{\text{sum_loss}}{10}$ ，所以 $\frac{\partial L}{\partial \text{sum_loss}} = \frac{1}{10}$ 。

```
1 dsum_loss = 0.1
```


注意，这里并非手动广播，因为不涉及形状变化。

sum_loss将每个梯度均匀分配到下边。

不要以为这是平权时代的到来，这只是欺骗其余众人的表面功夫！

11.3.4. $\log_probs \rightarrow probs$

回顾一下 $f(x) = \log(x)$ 的导数: $f'(x) = x^{-1}$.

$\log_probs_{ij} = -1 \times yenc_{ij} \times \log(probs_{ij})$, 则 $\frac{\partial \log_probs_{ij}}{\partial probs_{ij}} = -1 \times yenc_{ij} \times probs_{ij}^{-1}$.

逐元素计算并组合得到 $probs^{-1}$:

```

1 probs^-1:([[213878.70, 2650.24, 315.31, 482.47, 337.05, 10.67,
293.56, 2452.30, 57.05, 64694.73, 17.10, 49714.66, 43.79, 41185.1
3, 79.18, 3260.97, 19.53, 108.55, 139.16, 3.15, 499.27, 36
830.13, 2.87, 737.67, 26.12, 348.11, 264.16],
2 [41935.22, 5740.35, 169041.97, 7726.33, 16506.12, 89153.73, 7
568.69, 7.21, 75.49, 81.42, 3.43, 2008756.12, 667.30, 35425
0.41, 732.34, 1891.70, 100818.30, 109567.36, 24814.72, 5168.58, 249114.
80, 13219.60, 1.98, 100.64, 40.71, 7297.59, 18960416.00],
3 [5161364.00, 26198.21, 6995.67, 1824.82, 6419.97, 620.86,
785.64, 303.59, 2924.61, 615.41, 7967.51, 61845.20, 7.71, 33857.
23, 211.72, 16360.93, 1.50, 4910.74, 8.62, 830.70, 5220.72, 1
04321.59, 72.32, 17.45, 808.13, 12178.12, 178567.61],
4 [5161364.00, 26198.21, 6995.67, 1824.82, 6419.97, 620.86,
785.64, 303.59, 2924.61, 615.41, 7967.51, 61845.20, 7.71, 33857.
23, 211.72, 16360.93, 1.50, 4910.74, 8.62, 830.70, 5220.72, 1
04321.59, 72.32, 17.45, 808.13, 12178.12, 178567.61],
5 [399856832.00, 415370752.00, 6747150.00, 32542.74, 159266.75, 77
5110.44, 20007.50, 680.58, 768.61, 208.88, 37.90, 5231589.00, 6
1.80, 27811.48, 364598.75, 5505.98, 2.82, 9598.51, 597.11, 98.5
2, 23515.48, 2734063.50, 1.77, 119.55, 103.67, 54915.86, 69869904.0
0],
6 [213878.70, 2650.24, 315.31, 482.47, 337.05, 10.67,
293.56, 2452.30, 57.05, 64694.73, 17.10, 49714.66, 43.79, 41185.1
3, 79.18, 3260.97, 19.53, 108.55, 139.16, 3.15, 499.27, 36
830.13, 2.87, 737.67, 26.12, 348.11, 264.16],
7 [242226.05, 4943.14, 6145.72, 390536.94, 6.89, 673.86, 1
203673.38, 732.43, 2.05, 3191.12, 9.29, 673.95, 902.78, 1044
3.14, 460.00, 21661.29, 22.86, 2715.22, 30.83, 137.09, 8410.7
4, 12370.33, 13.66, 53.61, 20.30, 42.09, 910.73],
8 [70918.67, 1866.82, 1655.01, 2324.39, 90.27, 9.46, 67
51.53, 261.29, 8.54, 1083.44, 19.88, 212.64, 1705.20, 17866.1
5, 938.25, 54164.66, 8.95, 3255.37, 519.83, 83.71, 1446.13, 60
965.50, 23.83, 1.93, 62.53, 896.79, 31426.09],
9 [49630.18, 23284.15, 9760.43, 2186496.25, 557.86, 539803.31,
2176015.25, 4.89, 212.36, 3.97, 2.99, 320.48, 14175.19, 160
22.41, 33.97, 5591.03, 24.31, 27472.72, 256.39, 9.36, 2308.6
3, 3703.79, 147.85, 131.08, 7440.89, 15846.70, 496.86],
10 [82322.98, 39.75, 573.92, 15784.93, 51.31, 202.37, 37
78.96, 701.23, 60.08, 4818.32, 2.89, 8293.23, 1.84, 159155.5
5, 13147.74, 8233.11, 92.15, 4607.51, 52374.25, 281.33, 812.45, 87
032.20, 133.18, 107.96, 183.35, 10989.10, 4503.72]])

```

看起来这些数字很吓人，还记得yenc是个几乎全是0的独热编码吗？他会挑选天命人！

得到 $\frac{\partial \log_probs}{\partial probs}$:

```

1  tensor([[ -0.00, -0.00, -0.00, -0.00, -0.00, -10.67, -0.00, -0.00, -0.00, -0.
2      [-0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.
3      [-0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.
4      [-0.00, -26198.21, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.0
5      [-399856832.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -
6      [-0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.
7      [-0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.
8      [-0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -108
9      [-0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.
10     [-0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -481
    8.32, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.0
    0, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00]])

```

最后别忘了链式法则：

$$\frac{\partial L}{\partial probs} = \frac{\partial L}{\partial sum} \times \frac{\partial sum}{\partial \log_probs} \times \frac{\partial \log_probs}{\partial probs} = d \log_probs \times \frac{\partial \log_probs}{\partial probs}$$

```

1  tensor([[ -0.00, -0.00, -0.00, -0.00, -0.00, -1.07, -0.00, -0.00, -0.00, -0.
    00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00],
2      [-0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.
    00, -0.00, -0.00, -0.00, -35425.04, -0.00, -0.00, -0.00, -0.00, -0.00, -0.0
    0, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00],
3      [-0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.
    00, -0.00, -0.00, -0.00, -3385.72, -0.00, -0.00, -0.00, -0.00, -0.00, -0.0
    0, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00],
4      [-0.00, -2619.82, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.
    00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -
    0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00],
5      [-39985684.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.
    00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -
    0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00],
6      [-0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.
    00, -0.00, -0.00, -0.00, -0.00, -0.00, -326.10, -0.00, -0.00, -0.00, -0.00, -
    0.00, -0.00, -0.00, -0.00, -0.00, -0.00],
7      [-0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.
    00, -0.00, -0.00, -90.28, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -
    0.00, -0.00, -0.00, -0.00, -0.00, -0.00],
8      [-0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -10
    8.34, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.
    00, -0.00, -0.00, -0.00, -0.00, -0.00],
9      [-0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.
    00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -
    0.00, -0.00, -14.79, -0.00, -0.00, -0.00],
10     [-0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -48
    1.83, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.00, -0.
    00, -0.00, -0.00, -0.00, -0.00, -0.00]])

```

用代码写出来就是：

```

1  dprobs = dlog_probs * -1 * yenc * probs ** -1

```

到此为止，梯度全部分配到这几个特殊位置。

有意思的是，这几个数字，从 -1.07 到 -39985684.00 ，差距巨大。

11.3.5. *probs* $\rightarrow$ *counts* & *counts_sum_inv*

先看一下代码：

```

1  counts = torch.exp(logits2)
2  counts_sum = torch.sum(counts, dim=1, keepdims=True)
3  counts_sum_inv = counts_sum ** -1
4  probs = counts * counts_sum_inv

```

发现这里这有一点不同：

- *probs*由两个变量构成，我们对*counts*求导时需要将*counts_sum_inv*视为常数；对*counts_sum_inv*求导时需要将*counts*视为常数

- `counts`既是`probs`的变量，也是`counts_sum`的变量，所以`counts`在这里并未一次计算完成，而是需要将多个 $\frac{\partial L}{\partial counts}$ 多次累加。

由第一点可知： $\frac{\partial probs}{\partial counts} = counts_sum_inv$; $\frac{\partial probs}{\partial counts_sum_inv} = counts$

通过链式法则得到：

$$\frac{\partial L}{\partial counts} = \frac{\partial L}{\partial sum} \times \frac{\partial sum}{\partial \log_probs} \times \frac{\partial \log_probs}{\partial probs} \times \frac{\partial probs}{\partial counts} = dprobs \times$$

值得注意的是，`counts_sum_inv`的形状为 10×1 ,而 $dprobs \times counts$ 的形状为 10×27 ,需要沿 $dim = 1$ 求和：

$$\frac{\partial L}{\partial counts_sum_inv_i} = \frac{\partial L}{\partial sum} \times \frac{\partial sum}{\partial \log_probs} \times \frac{\partial \log_probs}{\partial probs} \times \frac{\partial probs}{\partial counts_sum_inv_i} = \sum_j \frac{\partial L}{\partial probs_j} \times counts_{ij}$$

```
1  counts_sum_inv:([[0.0076],
2                  [0.0001],
3                  [0.0010],
4                  [0.0010],
5                  [0.0002],
6                  [0.0076],
7                  [0.0017],
8                  [0.0034],
9                  [0.0009],
10                 [0.0009]])
```

用代码来写：

```
1  dcounts = dprobs * (counts_sum_inv * torch.ones_like(counts))
2  dcounts_sum_inv = torch.sum(dprobs * counts, dim=1, keepdim=True)
```

看这里，`dprobs`形状是 (10×27) ，`counts_sum_inv`则是 (10×1) ，这里的 `torch.ones_like(counts)`相当于一次手动广播。

11.3.6. `counts_sum_inv` $\rightarrow$ `counts_sum`

回顾一下 $f(x) = x^{-1}$ 的导函数： $f'(x) = -x^{-2}$

因此 `counts_sum_inv` = `counts_sum`⁻¹ 的导函数为：

$$\frac{\partial counts_sum_inv}{\partial counts_sum} = -counts_sum^{-2}$$

通过链式法则得到：

$$\frac{\partial L}{\partial counts_sum} = \frac{\partial L}{\partial sum} \times \frac{\partial sum}{\partial \log_probs} \times \frac{\partial \log_probs}{\partial probs} \times \frac{\partial probs}{\partial counts_sum_inv} \times \frac{\partial counts_sum_inv}{\partial counts_sum} = dcounts_sum_inv \times -counts_sum^{-2}$$

用代码来写：

```
1 dcounts_sum = dcounts_sum_inv * -1 * ((counts_sum + 1e-8) ** -2)
```

11.3.7. $counts_sum \rightarrow counts$

第二个counts姗姗来迟。

还记得 $sum_i = \sum_j^{27} \log_probs_{ij}$ 吗，同样的，也是计算求和过程的梯度。我们直接套用这个代码。

值得注意的是，因为counts从属于多个函数的变量，梯度需要累加，在代码中 “=” 要换成 “+=”：

```
1 dcounts += dcounts_sum * torch.ones_like(counts) # 相当于一次手动广播
```

至此，我们也可以看一下dcounts的全貌：

```

1  dcounts:([[0.000764,0.000764,0.000764,0.000764,0.000764,-0.007395,0.000764,
2  [0.000011,0.000011,0.000011,0.000011,0.000011,0.000011,0.000011,
3  [0.000095,0.000095,0.000095,0.000095,0.000095,0.000095,0.000095,
4  [0.000095,-2.501131,0.000095,0.000095,0.000095,0.000095,0.000095,
5  [-6227.279297,0.000016,0.000016,0.000016,0.000016,0.000016,0.0000
6  [0.000764,0.000764,0.000764,0.000764,0.000764,0.000764,0.000764,
7  [0.000166,0.000166,0.000166,0.000166,0.000166,0.000166,0.000166,
8  [0.000344,0.000344,0.000344,0.000344,0.000344,0.000344,0.000344,
9  [0.000090,0.000090,0.000090,0.000090,0.000090,0.000090,0.000090,
10 [0.000087,0.000087,0.000087,0.000087,0.000087,0.000087,0.000087,

```

11.3.8. *counts* $\rightarrow$ *logits2*

先来回顾一下 $f(x) = e^x$ 的导函数: $f'(x) = e^x$ 。

所以这里很简单, $\frac{\partial counts}{\partial logits2} = e^{logits2}$ 。

通过链式法则得到:

$$\frac{\partial L}{\partial \text{logits2}} = \frac{\partial L}{\partial \text{sum}} \times \frac{\partial \text{sum}}{\partial \log_probs} \times \frac{\partial \log_probs}{\frac{\partial probs}{e^{\text{logits2}}}} \times \frac{\partial probs}{\partial \text{counts}} \times \frac{\partial \text{counts}}{\partial \text{logits2}} = d\text{counts} \times$$

代码为:

```
1 dlogits2 = dcounts * torch.exp(logits2)
```

11.3.9. logits2 → h & W2 & b2

前几步反向传播全部都是逐元素相乘，这里我们才算第一次涉及到张量内积的反向求导，请朋友们坐稳扶好。

三部分，一部分是 $\frac{\partial \text{logits2}}{\partial \mathbf{W2}}$ ，第二部分是 $\frac{\partial \text{logits2}}{\partial \mathbf{h}}$ ，第三部分是 $\frac{\partial \text{logits2}}{\partial \mathbf{b2}}$ 。

• 第一部分:

$\frac{\partial \text{logits2}}{\partial \mathbf{W2}}$ 如果显示地构造Jacobian矩阵会是什么样子? logits2是 10×27 的张量, W2也是 10×27 的张量, 将他们各自展平为向量, 则是两个270维的巨大一阶张量, 用logits2中的每一个元素分别对W2中每一个元素求偏导数, 所有偏导数则构成一个 270×270 的超大二阶张量。这对计算并不友好!

于是, 我们请来之前的强大帮手: VJP (Vector-Jacobian Product, 向量-雅可比积),

$$\mathbf{v} \cdot \mathbf{J} = \frac{\partial L}{\partial \mathbf{Y}_{pre}} \times \frac{\partial \mathbf{Y}_{pre}}{\partial \mathbf{W}} = \mathbf{X}^T \cdot \nabla_{\mathbf{Y}_{pre}} L.$$

其中 $\frac{\partial L}{\partial \mathbf{Y}_{pre}}$ 表示从上游传来的梯度, 在这里就是 $\frac{\partial L}{\partial \text{logits2}}$,

$$\frac{\partial L}{\partial \text{logits2}} = \frac{\partial L}{\partial \text{sum}} \times \frac{\partial \text{sum}}{\partial \log_probs} \times \frac{\partial \log_probs}{\frac{\partial probs}{e^{\text{logits2}}}} \times \frac{\partial probs}{\partial \text{counts}} \times \frac{\partial \text{counts}}{\partial \text{logits2}} = d\text{counts} \times$$

我们要求的 $\frac{\partial \text{logits2}}{\partial \mathbf{W2}}$ 就被隐式的藏在: $\frac{\partial L}{\partial \mathbf{W2}} = \frac{\partial L}{\partial \text{logits2}} \times \frac{\partial \text{logits2}}{\partial \mathbf{W2}} = \mathbf{h}^T \cdot \frac{\partial L}{\partial \text{logits2}}$

($\frac{\partial L}{\partial \mathbf{W}} = \mathbf{X}^T \cdot \frac{\partial L}{\partial \mathbf{Y}_{pre}}$ 的证明可以去回顾 “现代神经网络反向传播的做法”)

写为代码:

```
1 dW2 = h.T @ dlogits2 # 顺序绝对不可以写反!
```

• 第二部分:

同理可得 $\frac{\partial L}{\partial \mathbf{h}} = \frac{\partial L}{\partial \text{logits2}} \cdot \mathbf{W2}^T$

写为代码:

```
1 dh = dlogits2 @ W2.T # 顺序绝对不可以写反!
```

- 第三部分：

这部分就很简单啦，只需要注意一下形状就好了。

由于b2在前向传播中沿 $dim = 0$ 方向广播了，这里要将它再压缩回一阶张量。

$$\frac{\partial \text{logits2}_{ij}}{\partial \mathbf{b2}_j} = \sum_i^{10} \frac{\partial L}{\partial \text{logits2}_{ij}}$$

写为代码：

```
1 db2 = torch.sum(dlogits2, dim=0, keepdim=True)
```

11.3.10. $\mathbf{h} \rightarrow \text{logits1}$

我们先回顾一下 $f(x) = \tanh(x)$ 的导函数： $(\tanh x)' = 1 - \tanh^2 x$ 。

所以 $\frac{\partial \mathbf{h}}{\partial \text{logits1}} = 1 - \mathbf{h}^2$ 。

加入链式法则： $d\text{logits1} = \frac{\partial L}{\partial \mathbf{h}} \times \frac{\partial \mathbf{h}}{\partial \text{logits1}} = \frac{\partial L}{\partial \text{logits1}} \times (1 - \mathbf{h}^2) = d\mathbf{h} \times (1 - \mathbf{h}^2)$ 。

写为代码：

```
1 dlogits1 = dh * (1 - h**2)
```

11.3.11. $\text{logits1} \rightarrow \mathbf{W1} \ \& \ \mathbf{b1}$

xenc为数据矩阵，不再进行求梯度运算。

$\mathbf{W1}$ 和 $\mathbf{b1}$ 的计算方法同上，推导过程不再展开。

代码：

```
1 dw1 = xenc.T @ dlogits1
2 db1 = torch.sum(dlogits1, dim=0, keepdim=True)
```

11.3.12. 反向传播完整代码

代码如下：

```

1  dL = 1.0
2  dsum_loss = 0.1
3  dlog_probs = dsum_loss * torch.ones_like(log_probs)
4  dprobs = dlog_probs * -1 * yenc * probs ** -1
5  dcounts = dprobs * counts_sum_inv
6  dcounts_sum_inv = torch.sum(dprobs * counts, dim=1, keepdim=True)
7  dcounts_sum = dcounts_sum_inv * -1 * counts_sum ** -2
8  dcounts += dcounts_sum * torch.ones_like(counts)
9  dlogits2 = dcounts * torch.exp(logits2)
10 dw2 = h.T @ dlogits2
11 dh = dlogits2 @ W2.T
12 db2 = torch.sum(dlogits2, dim=0, keepdim=True)
13 dlogits1 = dh * (1 - h**2)
14 dw1 = xenc.T @ dlogits1
15 db1 = torch.sum(dlogits1, dim=0, keepdim=True)

```

到此为止，我们完成了反向传播的每一步梯度计算。

你可以松一口气了。

但这仅仅是一个反向传播框架，对整个MLP而言，它还是脆弱的。

后边我们会面对很多问题，但我们会一一解决他们！

12. 构建BigramMLP——梯度更新与批次训练

我们通过两章内容，介绍了MLP如何将名字转换为字符对，如何将字符映射为机器可识别的整型，数据如何被编码，如何前向传播，然后如何一步一步求梯度，将梯度反向传播回各个张量。

一整套工具已然建立，本章只需要一点总结工作，就可以看到整个训练过程。

12.1. 通过梯度更新

在最一开始，我们就给出了通过损失寻找最优解的方法： $W_{new} = W_{old} - \Delta W$ 。

第一次随机初始化的权重矩阵为 W_1 ，第 n 轮训练更新后的权重矩阵为 W_n ， lr (learning rate) 为学习率，则梯度更新的公式为：

$$W_{n+1} = W_n - lr \times \nabla_{W_n} L$$

W 可以替换为反向传播过程中任意一个变量，但是在本例中，只有 $W1, W2, b1, b2$ 需要我们更新。

lr 一般设置为一个小数。太大会导致训练震荡、不收敛、梯度爆炸；太小则收敛极慢、陷入局部最优、训练超时。

上述过程写为代码：

```
1 lr = 0.5
2 W2 = W2 - lr * dW2
3 W1 = W1 - lr * dW1
4 b2 = b2 - lr * db2
5 b1 = b1 - lr * db1
```

12.2. loss下降

将整套代码放进循环里。让计算机自动完成多轮训练。

在这里，我们需要先重新整理一下我们的代码了：

```
1 import torch
2 import torch.nn.functional as F
3 torch.set_printoptions(linewidth=300)
4
5 # 导入与处理数据
6 file_path = "names.txt"
7 with open(file_path, "r", encoding="utf-8") as f:
8     content = f.read()
9 data2 = content.splitlines()
10 data = list(".".join(data2))
11
12 # 映射
13 A_data = ["."] + data
14 data_B = data + ["."]
15 set_data = sorted(set(data))
16 s_to_i = {ch: i for i, ch in enumerate(set_data)}
17 i_to_s = {i: ch for ch, i in s_to_i.items()}
18
19 # 构建数据集与目标集并完成独热编码
20 batchsize = 10
21 xs, ys = [], []
22 for ch1, ch2 in zip(A_data[:batchsize], data_B[:batchsize]):
23     ix1 = s_to_i[ch1]
24     ix2 = s_to_i[ch2]
25     xs.append(ix1)
26     ys.append(ix2)
27 xs = torch.tensor(xs)
28 ys = torch.tensor(ys)
29 xenc = F.one_hot(xs, num_classes=27).float()
30 yenc = F.one_hot(ys, num_classes=27).float()
31
32 # 参数设置
33 g = torch.Generator().manual_seed(123987)
34 W1 = torch.randn((27, 27), generator=g)
35 b1 = torch.randn((1, 27), generator=g)
36 W2 = torch.randn((27, 27), generator=g)
37 b2 = torch.randn((1, 27), generator=g)
38
39 # 前向传播
40 logits1 = xenc @ W1 + b1
41 h = torch.tanh(logits1)
42 logits2 = h @ W2 + b2
43 counts = torch.exp(logits2)
44 counts_sum = torch.sum(counts, dim=1, keepdims=True)
45 counts_sum_inv = (counts_sum + 1e-8) ** -1 # 在counts_sum中加一个极小的常量
    (如 1e-8),避免NaN/inf出现
46 probs = counts * counts_sum_inv
47 # probs = exp_logits2 / torch.sum(exp_logits2, dim=1, keepdims=True) # 可以
    写为一行,减少中间变量,提高运行速度,避免NaN/inf出现
48 log_probs = -1 * yenc * torch.log(probs)
49 sum_loss = torch.sum(log_probs, dim=1)
50 L = torch.mean(sum_loss)
```

```

51
52 # 反向传播
53 dL = 1.0
54 dsum_loss = dL / batchsize
55 dlog_probs = dsum_loss * torch.ones_like(log_probs)
56 dprobs = dlog_probs * -1 * yenc * (probs) ** -1
57 dcounts = dprobs * (counts_sum_inv * torch.ones_like(counts)) # 相当于一次
手动广播
58 dcounts_sum_inv = torch.sum(dprobs * counts, dim=1, keepdim=True)
59 dcounts_sum = dcounts_sum_inv * -1 * ((counts_sum + 1e-8) ** -2)
60 dcounts += dcounts_sum * torch.ones_like(counts) # 相当于一次手动广播
61 dlogits2 = dcounts * torch.exp(logits2)
62 # dlogits2 = (probs - yenc) / batch_size # 将 Softmax 和 Cross-Entropy 结合
在一起从数学公式上化简, 你会发现一个极其优雅的结果
63 dW2 = h.T @ dlogits2
64 dh = dlogits2 @ W2.T
65 db2 = torch.sum(dlogits2, dim=0, keepdim=True)
66 dlogits1 = dh * (1 - h**2)
67 dW1 = xenc.T @ dlogits1
68 db1 = torch.sum(dlogits1, dim=0, keepdim=True)
69
70 # 梯度更新
71 lr = 0.5
72 W2 = W2 - lr * dW2
73 W1 = W1 - lr * dW1
74 b2 = b2 - lr * db2
75 b1 = b1 - lr * db1

```

- 增加batchsize变量, 便于统一管理全局设置。
- 在probs和counts_sum中加一个极小的常量 (如 1e-8) ,避免NaN/inf出现。

假设我们需要将前十个字符训练10个 epochs 。

用for循环包裹前向传播、反向传播和梯度更新, 并在每次循环后打印loss, 看一下loss是否会按照预期下降。

在上述完整代码中补充:

```

▼ Python |
1 epochs = 10
2 for epoch in range(epochs):
3     # 前向传播
4     ...
5     # 反向传播
6     ...
7     # 梯度更新
8     ...
9     print(f"loss:{L.item()}")

```

特别强调的点: `L.item()`

如果打印`L`, `L` 张量被变量`print` 持有引用,就不会被回收机制清除, 单次循环还好, 后边完整循环需要计算海量数据, `L`持续持有会极大占用显存。

打印`L.item()`, `.item()` 将张量转换为纯粹的 Python 浮点数, 主动切断计算图的依赖 则不会保留`L`在计算图中。

点击运行按钮, 激动人心的时刻就要来了!

如果你用的随机种子与我一致, 那么你的终端窗口将会输出如下内容:

▼ Plain Text

```
1  loss:9.09150505065918
2  loss:6.4462690353393555
3  loss:4.555797100067139
4  loss:3.23270845413208
5  loss:2.3163325786590576
6  loss:1.6715431213378906
7  loss:1.2211062908172607
8  loss:0.9164503216743469
9  loss:0.7085820436477661
10 loss:0.5808719992637634
```

恭喜! 这大概是你人生中第一次成功训练了一个模型!

`loss`在稳定的下降。

接下来, 让我们调整`epochs`的大小, 来试探一下`loss`的极限。

不妨将`epochs`设置大一点, 令`epochs = 50`。

```
1 loss:9.09150505065918
2 loss:6.4462690353393555
3 loss:4.555797100067139
4 loss:3.23270845413208
5 loss:2.3163325786590576
6 loss:1.6715431213378906
7 loss:1.2211062908172607
8 loss:0.9164503216743469
9 loss:0.7085820436477661
10 loss:0.5808719992637634
11 loss:0.5092260241508484
12 ...
13 loss:0.3096591532230377
14 loss:0.30873844027519226
15 loss:0.30786848068237305
16 loss:0.30704522132873535
17 loss:0.30626487731933594
18 loss:0.3055242896080017
19 loss:0.3048204183578491
20 loss:0.30415067076683044
21 loss:0.30351248383522034
22 loss:0.30290383100509644
23 loss:0.3023225665092468
```

你会发现损失会稳定在0.3，似乎很难再推进下去。

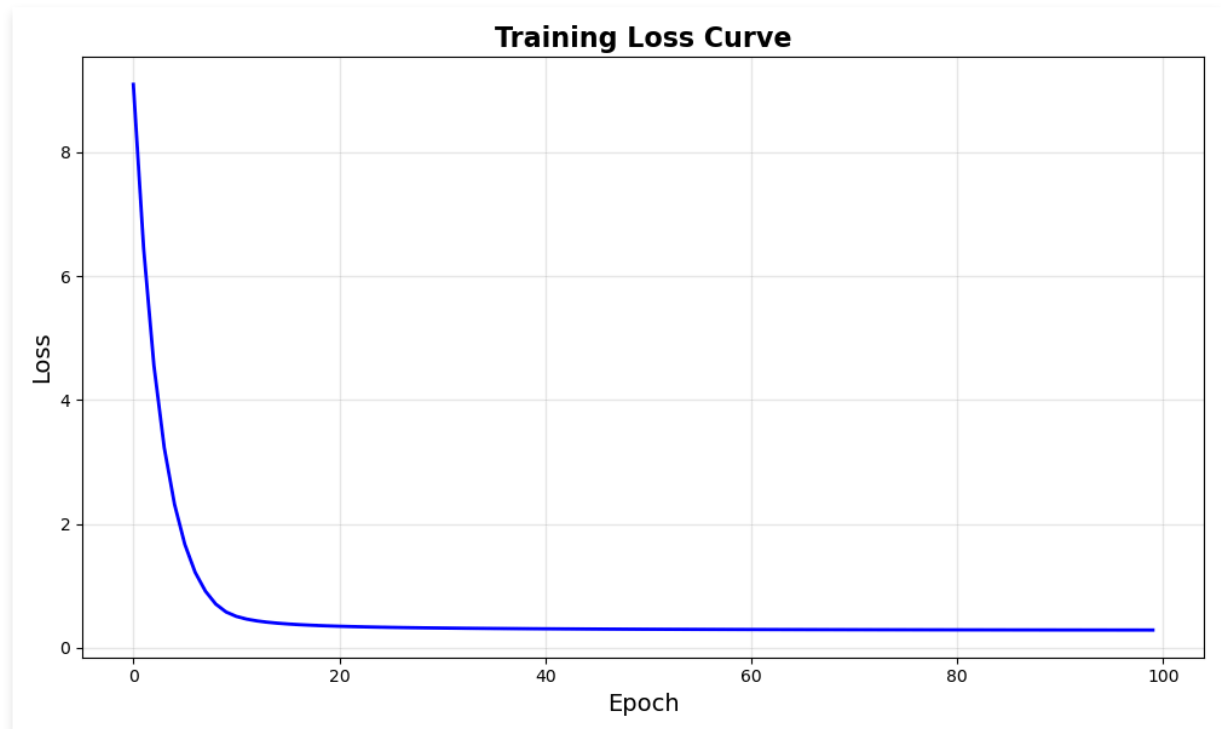
这次我们令epochs = 100，并添加如下代码，结束时打印一个最终loss和一个曲线图。

```
1 import torch
2 import torch.nn.functional as F
3 import matplotlib.pyplot as plt # 增加绘图库
4 torch.set_printoptions(linewidth=300)
5
6 ...
7 loss_history = [] # 存储每个epoch的loss
8 epochs = 100
9 for epoch in range(epochs):
10     ...
11     loss_history.append(L.item()) # 记录loss (转换为numpy以便绘图)
12 print(f"loss:{L.item()}")
13
14 # 训练结束后绘制loss曲线
15 plt.figure(figsize=(10, 6))
16 plt.plot(range(epochs), loss_history, 'b-', linewidth=2)
17 plt.xlabel('Epoch', fontsize=14)
18 plt.ylabel('Loss', fontsize=14)
19 plt.title('Training Loss Curve', fontsize=16, fontweight='bold')
20 plt.grid(True, alpha=0.3)
21 plt.tight_layout()
22 plt.show()
```

经过100轮循环，loss稳定在0.29左右：

1 loss:0.288970410823822

从loss下降图中也可以看出，在40-100轮之后，损失基本趋近于平稳。



12.3. 逐批次训练

记得我们用了多少个数据样本吗？

一批中数据样本的多少我们称之为batchsize，在上述例子中，batchsize=10。

我们每10个批次循环50个epochs，即使我们训练再多epochs，数据也仅仅来自于上述10个样本，我们剩下的名字完全没用到。

接下来，我们就将剩余的数据充分利用起来！

回顾代码：

```
1  # 构建数据集与目标集并完成独热编码
2  batchsize = 10
3  xs, ys = [], []
4  for ch1, ch2 in zip(A_data[:batchsize], data_B[:batchsize]):
5      ix1 = s_to_i[ch1]
6      ix2 = s_to_i[ch2]
7      xs.append(ix1)
8      ys.append(ix2)
9  xs = torch.tensor(xs)
10 ys = torch.tensor(ys)
```

A_data和data_B是原材料，这段代码则是我们生产“批次”的工厂！

问题是工厂目前的状态是产出第一批数据集和目标集。我们要对它技术改造一下！

最终效果应该是：

“批次”生产工厂根据batchsize的设置，依次将A_data和data_B切割为大小合适的xs和ys，然后独热编码工厂接手并将它们转换为输入矩阵，后边的前向传播工厂、反向传播工厂...依次加工数据，得到一批新鲜出炉的W2, W1, b2, b1。

然后工厂再次启动产线，切割剩余原材料。

循环往复，直到所有原材料用光！

这仅仅是将所有数据训练了一遍，epochs = 1，我们暂时叫它小循环。

用for循环将上述流程根据epochs的设置，循环多次，我们称之为大循环。

这就需要我们z把代码中参数设置的部分移到大循环之外，避免循环过程中训练好的权重被覆盖。

ok，开干！

我们用range(start, stop, step)函数来完成切割：

```
Python |  
1  batchsize = 256 # step参数；设置为设置为 2 的幂次方，适配GPU 的流多处理器调度线  
   程，且符合黄金法则（样本数是特征数的10-100倍）。  
2  Total_len = len(A_data) # stop参数  
3  for start in range(0, Total_len, batchsize): # 以batchsize为大小，将总长度切  
   割，余数部分转化为小于 batchsize的批次。  
4      end = start + batchsize  
5      batchA = A_data[start:end]  
6      batchB = data_B[start:end]  
7      # 构建数据集与目标集并完成独热编码  
8      xs, ys = [], []  
9      for ch1, ch2 in zip(batchA, batchB):  
10         ix1 = s_to_i[ch1]  
11         ix2 = s_to_i[ch2]  
12         xs.append(ix1)  
13         ys.append(ix2)  
14     xs = torch.tensor(xs)  
15     ys = torch.tensor(ys)
```

如果你是极简主义，也可以将代码浓缩为如下形式，而且跳过中间变量也是优化性能的好办法！

```
Python |  
1  batchsize = 256  
2  for start in range(0, len(A_data), batchsize):  
3      batchA = A_data[start:start + batchsize]  
4      batchB = data_B[start:start + batchsize]  
5      xs = torch.tensor([s_to_i[ch1] for ch1 in batchA]) # torch.tensor() 必须  
   接收一个「现成的列表 / 数组」，不能接收「生成器」  
6      ys = torch.tensor([s_to_i[ch2] for ch2 in batchB])
```

如果你的显存够大，追求机制的性能，代码还可以优化为：

```
1  # 在进入大循环之前（大循环外部）一次性处理完毕
2  xs, ys = [], []
3  for ch1, ch2 in zip(A_data, data_B):
4      xs.append(s_to_i[ch1])
5      ys.append(s_to_i[ch2])
6  X_all = torch.tensor(xs)
7  Y_all = torch.tensor(ys)
8
9  # 整个数据集的 one_hot 直接在外面做完（如果显存允许）
10 X_enc_all = F.one_hot(X_all, num_classes=27).float()
11 Y_enc_all = F.one_hot(Y_all, num_classes=27).float()
12
13 # ===== 进入大循环 =====
14 for epoch in range(epochs):
15     for start in range(0, Total_len, batchsize):
16         end = start + batchsize
17         # 内部直接切片！没有任何 Python 字符串操作，速度提升百倍
18         xenc = X_enc_all[start:end]
19         yenc = Y_enc_all[start:end]
```

一方面为了清晰地了解代码构建过程，一方面兼顾设备性能。我们暂时选择牺牲性能和时间。

最终版代码如下：

```

1  import torch
2  import torch.nn.functional as F
3  torch.set_printoptions(linewidth=300)
4
5  # 导入与处理数据
6  file_path = "names.txt"
7  with open(file_path, "r", encoding="utf-8") as f:
8      content = f.read()
9  data2 = content.splitlines()
10 data = list(".".join(data2))
11
12 # 映射
13 A_data = ["."] + data
14 data_B = data + ["."]
15 set_data = sorted(set(data))
16 s_to_i = {ch: i for i, ch in enumerate(set_data)}
17 i_to_s = {i: ch for ch, i in s_to_i.items()}
18
19 # 参数设置
20 g = torch.Generator().manual_seed(123987)
21 W1 = torch.randn((27, 27), generator=g)
22 b1 = torch.randn((1, 27), generator=g)
23 W2 = torch.randn((27, 27), generator=g)
24 b2 = torch.randn((1, 27), generator=g)
25
26 # 大循环
27 # 循环内参数
28 epochs = 50
29 batchsize = 256 # step参数; 设置为设置为 2 的幂次方, 适配GPU 的流多处理器调度线
    程, 且符合黄金法则 (样本数是特征数的10-100倍)。
30 Total_len = len(A_data) # stop参数
31 lr = 0.5
32
33 for epoch in range(epochs):
34     # 小循环
35     for start in range(0, Total_len, batchsize): # 以batchsize为大小, 将总长
        度切割, 余数部分转化为小于 batchsize的批次。
36         end = start + batchsize
37         batchA = A_data[start:end]
38         batchB = data_B[start:end]
39         # 构建数据集与目标集并完成独热编码
40         xs, ys = [], []
41         for ch1, ch2 in zip(batchA, batchB):
42             ix1 = s_to_i[ch1]
43             ix2 = s_to_i[ch2]
44             xs.append(ix1)
45             ys.append(ix2)
46         xs = torch.tensor(xs)
47         ys = torch.tensor(ys)
48         xenc = F.one_hot(xs, num_classes=27).float()
49         yenc = F.one_hot(ys, num_classes=27).float()
50

```

```

51     # 前向传播
52     logits1 = xenc @ W1 + b1
53     h = torch.tanh(logits1)
54     logits2 = h @ W2 + b2
55     counts = torch.exp(logits2)
56     counts_sum = torch.sum(counts, dim=1, keepdims=True)
57     counts_sum_inv = (counts_sum + 1e-8) ** -1 # 在counts_sum中加一个极
小的常量 (如 1e-8) ,避免NaN/inf出现
58     probs = counts * counts_sum_inv
59     # probs = exp_logits2 / torch.sum(exp_logits2, dim=1, keepdims=True)
e) # 可以写为一行,减少中间变量,提高运行速度,避免NaN/inf出现
60     log_probs = -1 * yenc * torch.log(probs)
61     sum_loss = torch.sum(log_probs, dim=1)
62     L = torch.mean(sum_loss)
63
64     # 反向传播
65     dL = 1.0
66     dsum_loss = dL / xenc.shape[0]
67     dlog_probs = dsum_loss * torch.ones_like(log_probs)
68     dprobs = dlog_probs * -1 * yenc * (probs) ** -1
69     dcounts = dprobs * (counts_sum_inv * torch.ones_like(counts)) # 相
当于一次手动广播
70     dcounts_sum_inv = torch.sum(dprobs * counts, dim=1, keepdim=True)
71     dcounts_sum = dcounts_sum_inv * -1 * ((counts_sum + 1e-8) ** -2)
72     dcounts += dcounts_sum * torch.ones_like(counts) # 相当于一次手动广
播
73     dlogits2 = dcounts * torch.exp(logits2)
74     # dlogits2 = (probs - yenc) / batch_size # 将 Softmax 和 Cross-Entr
opy 结合在一起从数学公式上化简,你会发现一个极其优雅的结果
75     dW2 = h.T @ dlogits2
76     dh = dlogits2 @ W2.T
77     db2 = torch.sum(dlogits2, dim=0, keepdim=True)
78     dlogits1 = dh * (1 - h**2)
79     dW1 = xenc.T @ dlogits1
80     db1 = torch.sum(dlogits1, dim=0, keepdim=True)
81
82     # 梯度更新
83     W2 = W2 - lr * dW2
84     W1 = W1 - lr * dW1
85     b2 = b2 - lr * db2
86     b1 = b1 - lr * db1
87
88     print(f"loss:{L.item()}")

```

开始坏笑~

12.4. 再优化

当你运行上述代码, 输出结果如下:



```
1  loss:2.6463077068328857
2  loss:2.5791244506835938
3  loss:2.5376198291778564
4  loss:2.5121636390686035
5  loss:2.497959852218628
6  loss:2.496058940887451
7  loss:2.5077245235443115
8  loss:2.5300450325012207
9  loss:2.557934045791626
10 loss:2.587160348892212
11 loss:2.6138761043548584
12 loss:2.6312851905822754
13 loss:2.6375434398651123
14 loss:2.6353211402893066
15 loss:2.6282434463500977
16 loss:2.61676025390625
17 loss:2.6011486053466797
18 loss:2.58591890335083
19 loss:2.575582981109619
20 loss:2.5727477073669434
21 loss:2.578793525695801
22 loss:2.5895800590515137
23 loss:2.598919630050659
24 loss:2.6033105850219727
25 loss:2.6029696464538574
26 loss:2.601879358291626
27 loss:2.6067774295806885
28 loss:2.6149532794952393
29 loss:2.6189522743225098
30 loss:2.6167714595794678
31 loss:2.612699508666992
32 loss:2.6090593338012695
33 loss:2.6042304039001465
34 loss:2.5979301929473877
35 loss:2.5921630859375
36 loss:2.58912992477417
37 loss:2.5893471240997314
38 loss:2.591583013534546
39 loss:2.5944244861602783
40 loss:2.5962655544281006
41 loss:2.5956947803497314
42 loss:2.593663215637207
43 loss:2.5925464630126953
44 loss:2.593156099319458
45 loss:2.594863176345825
46 loss:2.5963289737701416
47 loss:2.5956387519836426
48 loss:2.5919570922851562
49 loss:2.586557626724243
50 loss:2.5809881687164307
```

天都塌了！一顿操作猛如虎，一看损失二点五，还不下降！

损失基本从2.5左右一直跳动，上次每个epoch还稳稳当当的下降，这次为什么就不行了？

看我们这次动了哪些地方，哪些参数可以更改：

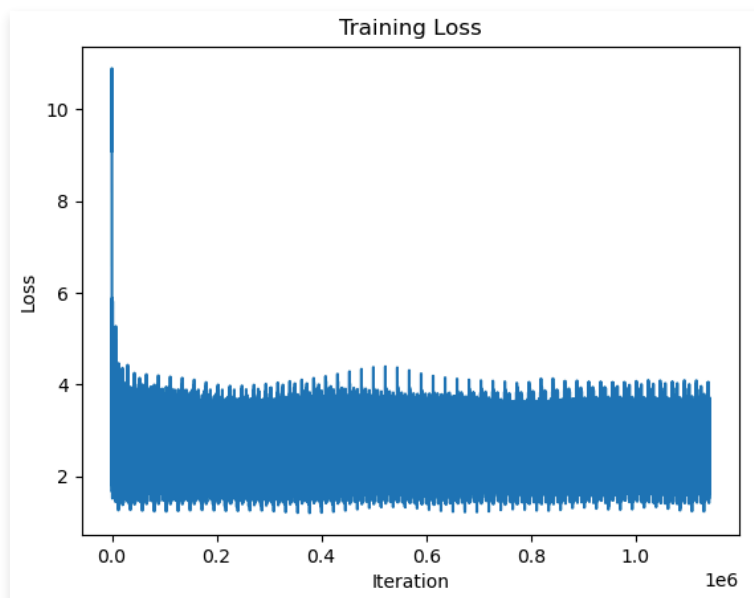
- 单batchsize样本从10变成了256个；
- 一个固定参数可以尝试修改：lr；

我们控制变量，逐个尝试。

12.4.1. batchsize

我们将batchsize调整为原来的10，并画图。（为了控制变量，记得把dsum_loss从固定为0.003906）

loss变化图如下：



一开始loss骤降，但每次epoch的loss稳定在3.5左右，基本不变，和我们之前训练得到的2.58左右相去甚远，看来和batchsize的黄金法则还是非常有用的！

而且你会发现，当batchsize太小，运行会非常慢，这是因为python在小循环中运行的次数为原先的25.6倍，真是卖力不讨好！

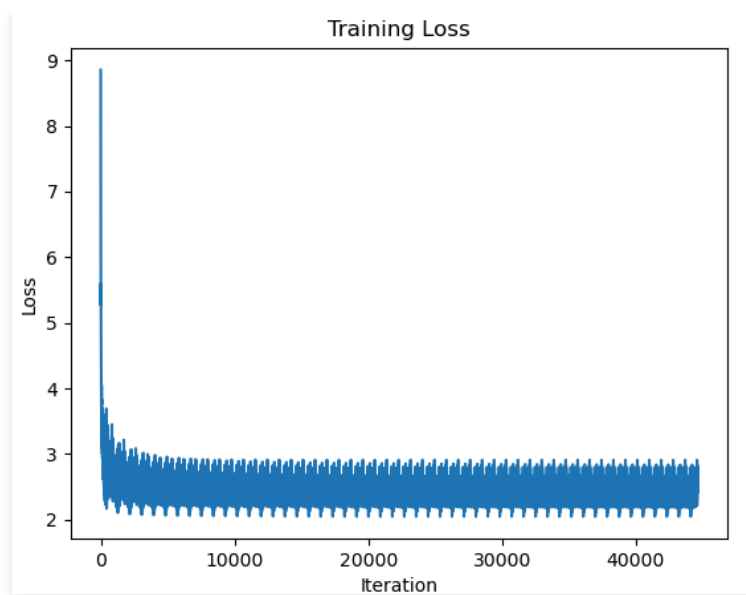
12.4.2. learning rate

只剩下一个learning rate可以修改了。

我们想一下，为什么epoch每次基本不变？ learning rate如何影响它？

还记得我们在最开始讲过的例子， $y = x^2$ 的导数在 $x = 2$ 和 $x = -2$ 是相反数，如果我们一步子迈地太大，会导致最终结果发散，所以，我们将learning rate调整小一点。

这次让lr=0.1:

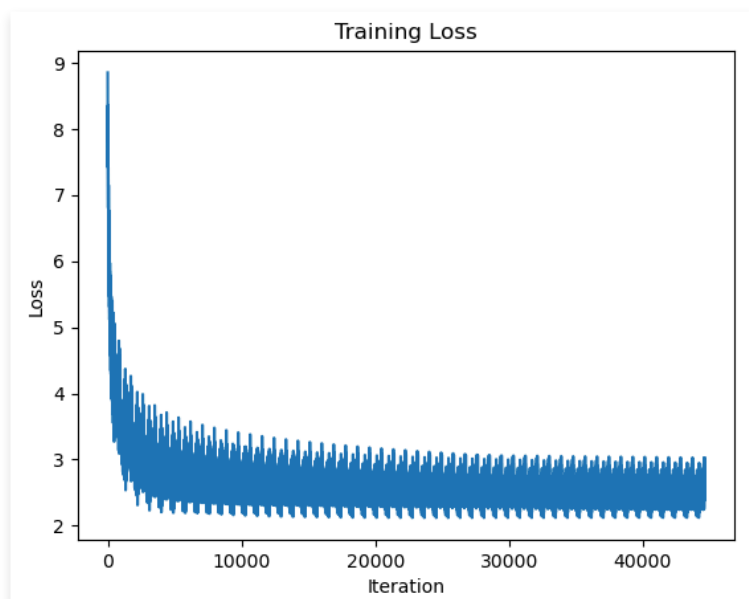




1	epoch 01	loss: 3.1262
2	epoch 02	loss: 3.0086
3	epoch 03	loss: 2.9784
4	epoch 04	loss: 2.9585
5	epoch 05	loss: 2.9386
6	epoch 06	loss: 2.9192
7	epoch 07	loss: 2.9016
8	epoch 08	loss: 2.8861
9	epoch 09	loss: 2.8725
10	epoch 10	loss: 2.8603
11	epoch 11	loss: 2.8494
12	epoch 12	loss: 2.8397
13	epoch 13	loss: 2.8310
14	epoch 14	loss: 2.8232
15	epoch 15	loss: 2.8163
16	epoch 16	loss: 2.8102
17	epoch 17	loss: 2.8048
18	epoch 18	loss: 2.8000
19	epoch 19	loss: 2.7958
20	epoch 20	loss: 2.7920
21	epoch 21	loss: 2.7887
22	epoch 22	loss: 2.7858
23	epoch 23	loss: 2.7831
24	epoch 24	loss: 2.7808
25	epoch 25	loss: 2.7787
26	epoch 26	loss: 2.7768
27	epoch 27	loss: 2.7751
28	epoch 28	loss: 2.7736
29	epoch 29	loss: 2.7722
30	epoch 30	loss: 2.7709
31	epoch 31	loss: 2.7698
32	epoch 32	loss: 2.7688
33	epoch 33	loss: 2.7680
34	epoch 34	loss: 2.7672
35	epoch 35	loss: 2.7665
36	epoch 36	loss: 2.7658
37	epoch 37	loss: 2.7653
38	epoch 38	loss: 2.7648
39	epoch 39	loss: 2.7644
40	epoch 40	loss: 2.7641
41	epoch 41	loss: 2.7638
42	epoch 42	loss: 2.7635
43	epoch 43	loss: 2.7633
44	epoch 44	loss: 2.7632
45	epoch 45	loss: 2.7631
46	epoch 46	loss: 2.7630
47	epoch 47	loss: 2.7629
48	epoch 48	loss: 2.7629
49	epoch 49	loss: 2.7629
50	epoch 50	loss: 2.7630

这次可以看到每轮epoch都在稳定下降了，说明学习率起效果了，但是最终稳定在2.76，依然比最开始的2.58还要高。

我们继续调整，这次我们这次让 $lr=0.01$ ：



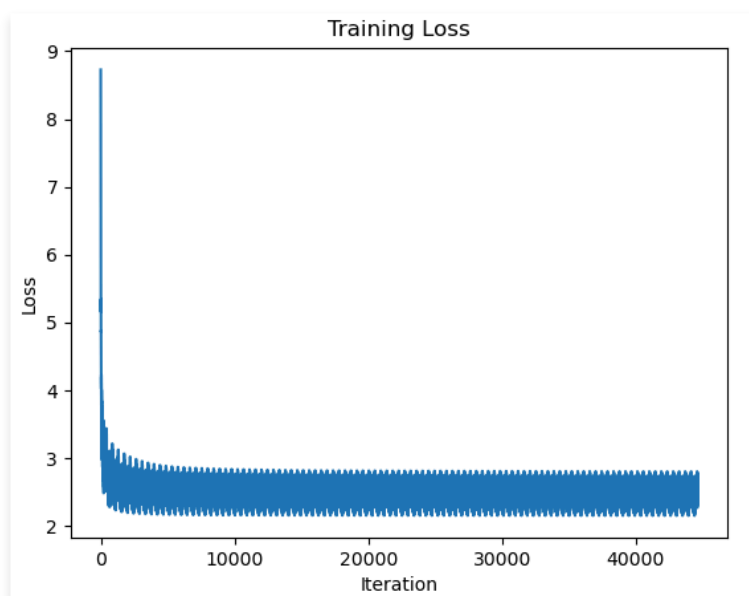
这次下降更平稳，但最终loss稳定在3.03，效果反而不好。

12.4.3. 终极尝试

聪明的你已经发现，学习率变小，但epoch未变。就像一个笨学生和聪明学生，都是在学校学习一天，最终效果肯定是不一样的。

同样，学习率小，那我们就多学几轮，所谓勤能补拙嘛！

我们将 lr 设置为0.1，epochs为100，顺便把batchsize提高到512（样本数是特征数的18.9倍，比9.5倍更符合黄金法则），看一下最后效果。

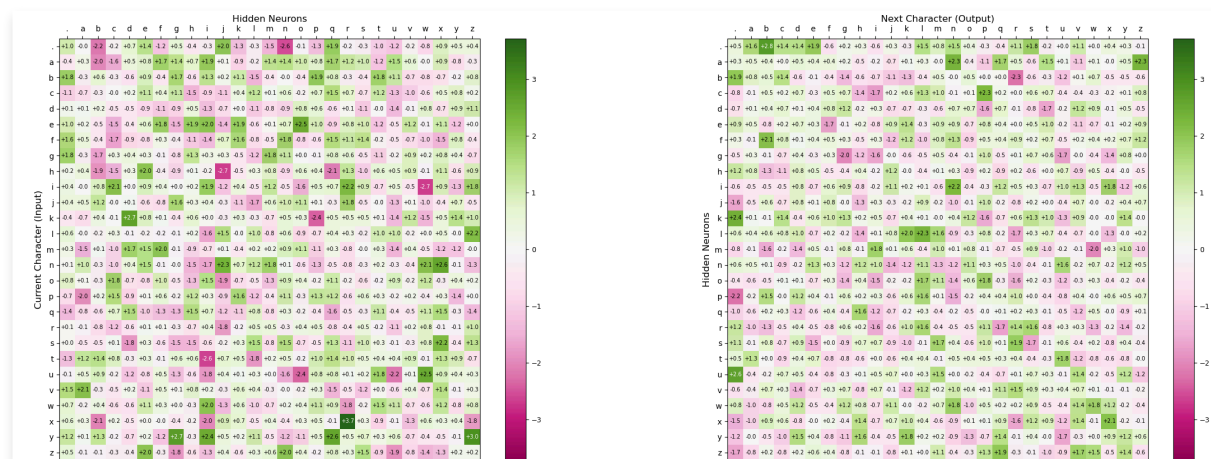


当你看到 lr 逐步下降到2.52左右的时候，就证明我们目前的优化已经起效果了。

彩蛋：为什么最开始的loss那么高？然后快速收敛到了2.52？

12.5. 权重指纹

我们将最后训练好的权重绘制为热力图，我称之为权重指纹（如果有专业的名称，请大家指正！）



其中绝对值大的区域是神经元上被强烈激活的突触，深绿色表示为强烈支持，深红色则表示强烈抑制。为0的突触则表示静默。

我们建立一个“虚拟实验室”，把 27 个英文字符作为标准的“单一变量”（一个 27×27 的单位矩阵，是按每个字符顺序排列的总的xenc编码），挨个输入给训练好的网络，把网络对这 27 种情况的所有反应（得分）整整齐齐地记录成一个 27×27 的表格。

还记得我们的logits怎么来的吗？

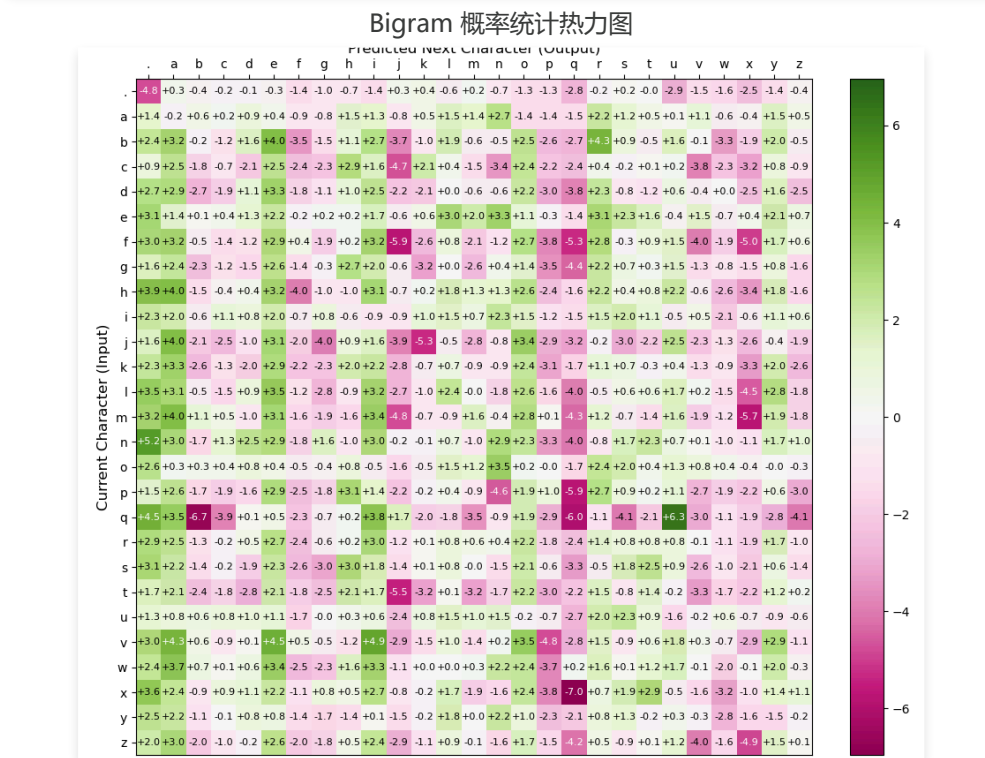
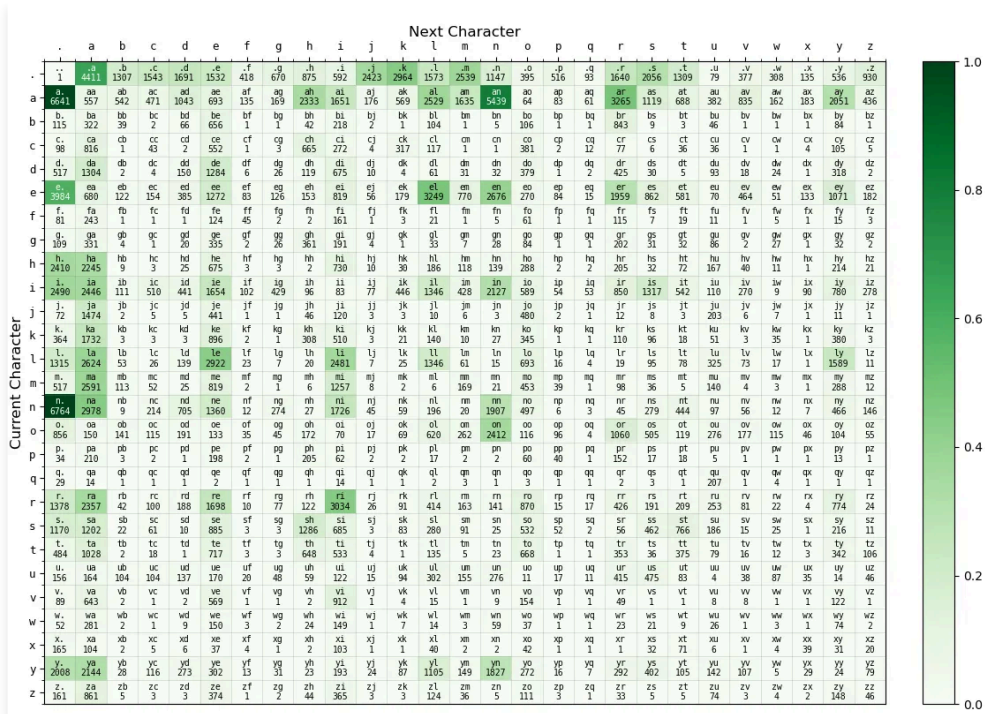
```
1 logits2 = (torch.tanh(xenc @ W1 + b1)) @ W2 + b2
2 counts = torch.exp(logits2)
```

counts为伪计数，数值相对大小等价于logits2，而Bigram 概率统计热力图就是实际的计数统计。

所以logits2的数据分布与Bigram 概率统计分布惊人的一致。

这也就意味着，我们的MLP训练过程，实际上就是对数据中字符之间关系的模拟！这与我们一开始讲得对上了。

Bigram 概率统计热力图与logits2对比:



logits2

虽然不是特别明显，但依然可以看出，左图深绿色的分布与右图深绿色的分布呈现某种相似性；左图浅绿色的分布与右图深红色的分布呈现某种相似性。

当增加神经元数量后，上述两张图会变得十分相似。每个神经元相当于一个专家，专家变多了，分工更细了，融合出来的logits2，其精度会达到顶峰，跟最初的大数据 Bigram 统计图贴合得严丝合缝。

神经网络的“消极防御”

如果你仔细观察生成的 `logits2` 热力图，你会发现一个很有意思的语言学特征：**品红/红色（低频抑制）的格子数量，往往比绿色（高频激活）的格子数量还要多。**

这是因为神经网络在学习语言时，很大一部分精力都在做“消极防御”。

在自然语言中，“合规”的拼写组合其实是少数，而“瞎拼”的组合（比如 `j` 后面接 `q`，`x` 后面接 `z`）是无穷无尽的。

所以，模型通过大面积的**红色得分**，在名字的语法森林里拉起了漫长的警戒线，警告采样器：“千万别往这些格子里走！”。而留给那些**绿色高频组合**的，则是少数几条踩出来的康庄大道（比如 `q` 后面必连 `u`，`t` 后面常连 `h`）。

13. 构建BigramMLP——生成名字

如果你还记得我们的初衷，你就不会仅仅因为看到完美的loss下降曲线而止步于此。

用训练好的MLP去生成一个名字才是我们最构建MLP的目的。

这里并不需要太多技巧。

13.1. 创建 `model.pt`

我们将训练好的参数保存在 `model.pt` 文件中，这个文件会记录我们最终生成的训练参数。

先在文件夹中新建一个`train.py`文件，将上述代码复制粘贴进去，然后仅需在代码中修改/添加如下内容：

```
1 import torch
2 import torch.nn.functional as F
3 torch.set_printoptions(linewidth=300)
4
5 # 导入与处理数据
6 ...
7 # 大循环
8 ...
9     # 梯度更新
10     W2 = W2 - lr * dW2
11     W1 = W1 - lr * dW1
12     b2 = b2 - lr * db2
13     b1 = b1 - lr * db1
14
15 print(f"Finshed! loss: {L.item():.4f}")
16 # 保存模型参数到硬盘
17 checkpoint = {
18     'W1': W1,
19     'b1': b1,
20     'W2': W2,
21     'b2': b2,
22     's_to_i': s_to_i, # 顺便把映射表也存进去，防止以后字符顺序变了导致索引对不上
23     'i_to_s': i_to_s
24 }
25 torch.save(checkpoint, 'model.pt') # 这里的 'model.pt' 就是文件名；python会生成并保存在同一文件夹下。
26 print("The results have been successfully exported as the file 'model.pt'!")
```

13.2. 创建 generate.py

在同一文件夹下创建generate.py文件，在文件中写入并保存如下代码：

```

1  import torch
2  import torch.nn.functional as F
3
4  # 直接从硬盘读取训练好的参数和映射表
5  print("loading...")
6  checkpoint = torch.load('model.pt')
7
8  W1 = checkpoint['W1']
9  b1 = checkpoint['b1']
10 W2 = checkpoint['W2']
11 b2 = checkpoint['b2']
12 s_to_i = checkpoint['s_to_i']
13 i_to_s = checkpoint['i_to_s']
14 print("Succeeded\n")
15
16 # 采样生成名字
17 g_gen = torch.Generator().manual_seed(123987) # 换个种子可以得到不同风格的名字
18 num_names = 10
19
20 print(f"The {num_names}names are as follows:\n")
21 for _ in range(num_names):
22     out = []
23     ix = s_to_i['.'] # 从起始符 '.' 开始
24
25     while True:
26         # 逐字预测：每次只输入一个字符，构建独热编码 (1, 27)
27         xenc = F.one_hot(torch.tensor([ix]), num_classes=27).float()
28
29         # 纯粹的前向传播，不需要计算任何梯度
30         logits1 = xenc @ W1 + b1
31         h = torch.tanh(logits1)
32         logits2 = h @ W2 + b2
33
34         counts = torch.exp(logits2)
35         probs = counts / torch.sum(counts, dim=1, keepdims=True)
36
37         # 按照模型给出的概率进行多项式抽样
38         ix = torch.multinomial(probs, num_samples=1, replacement=True, generator=g_gen).item()
39
40         # 抽到 '.' 说明名字结束了
41         if ix == s_to_i['.']:
42             break
43
44         out.append(i_to_s[ix])
45
46     print("".join(out))

```

这样generate.py 就可以使用model.pt 中的参数进行生成名字。

值得注意的是：

- **为什么不直接用 `torch.argmax` 而是用 `torch.multinomial` ？** 如果使用 `argmax`，模型每次遇到字符 `a`，都只会输出它认为概率最高的那一个固定字符。这样不仅导致每次生成的名字完全一样，还很容易陷入死循环（比如 `a -> n -> a -> n` 陷入 `ananan...`）。而 `multinomial` 会严格按照模型输出的概率去抽样，概率高的字符被抽到的几率大，概率低的也有小几率被抽到，从而保证了生成文本的灵活性。
- **参数状态：**生成预测时，代码中的 `w1`，`b1`，`w2`，`b2` 已经是大循环迭代 100 个 epoch 之后更新好的最终权重。
- **输入维度：**在训练时你的输入 `xenc` 形状是 `(batchsize, 27)`，而在生成预测时，因为是逐字生成，所以输入形状是 `(1, 27)`。

13.3. 生成！

希望你的终端窗口输出了如下内容：

```

1  The 10names are as follows:
2
3  on
4  mmyonanteyn
5  trahn
6  sh
7  saynle
8  n
9  lon
10 qumaharshikaheco
11 jarinienused
12 chysth
```

当你看到这些奇怪的名字，请别害怕，这并非我们在前边搞错了什么，而是因为二元字符模型本身太弱。

相当于我只根据前一个字符预测下一个字符，这种只看眼前一个字符的预测模式十分脆弱。

举个例子：

给定一个字符“水”的条件下，让你去预测下一个字符，你可能会说“水流”、“水果”、“水滴”.....

但我给你一段很长的字符“我喜欢吃水”，你大概立刻就知道，下一个字符很可能是“果”，因为“我喜欢吃水果”是一句很常用的话，它在生活语境的概率中远远高于“我喜欢吃水流”等其他语句。

至于如何构建多元字符预测模型，我们会在后边继续探索！

14. 关于BigramMLP的补充

从最开始的导入数据，到最后生成一个个名字；从冷冰冰的数学公式到一张张损失图。回头看这条路，不知道你现在作何感受。

我们通过一个个数学公式，将整个反向传播构建出来，你有没有感觉到，整个世界的运转都掌握在你的手中。

这大概就是理解与创造的快乐。

如果你能在此中感受到了一丝丝幸福，这篇文档也便多了一丝意义。

在结束BigramMLP并开启下一段冒险之前，让我们对刚刚走过的路做一次总结。

当然，你可以尝试自己写一写这个过程你犯过了那些错，或觉得有意思的地方，抑或有了什么思考。我想这是十分有意思的一件事。

14.1. 性能优化

我们从硬件与代码上优化一下速度。

14.1.1. 设备优化

Nvidia GPU的cuda核心天然适合进行大量并行运算，尤其对张量运算做了优化，如果你的电脑支持GPU运算，我建议你吧张量运算放在GPU上。

只需在train.py上补充如下代码：

```
1  import...
2  ...
3  # 设置默认设备，之后所有新创建的张量都会在GPU上
4  device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5  torch.set_default_device(device)
6  print(f"默认设备设置为: {device}")
7
8  # 性能优化设置
9  if device.type == 'cuda':
10     # 添加非阻塞数据传输
11     torch.backends.cudnn.benchmark = True # 自动寻找最优算法
12     torch.backends.cuda.matmul.allow_tf32 = True # 允许TF32加速（Ampere及以上架构）
13     ...
14     # 参数设置
15     g = torch.Generator(device=device).manual_seed(123987)
16     ...
```

generate.py全部代码：

```

1  import torch
2  import torch.nn.functional as F
3
4  # 设置默认设备，之后所有张量自动在此设备上
5  device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
6  torch.set_default_device(device)
7  print(f"device: {device}")
8
9  # 直接从硬盘读取训练好的参数和映射表
10 print("loading...")
11 checkpoint = torch.load('model.pt', map_location=device, weights_only=False)
12
13 W1 = checkpoint['W1']
14 b1 = checkpoint['b1']
15 W2 = checkpoint['W2']
16 b2 = checkpoint['b2']
17 s_to_i = checkpoint['s_to_i']
18 i_to_s = checkpoint['i_to_s']
19
20 print(f"The model had been loaded to: {W1.device}")
21 print("Succeeded\n")
22
23 # 采样生成名字
24 g_gen = torch.Generator(device=device).manual_seed(187) # Generator也要指定设备
25 num_names = 10
26
27 print(f"The {num_names} names are as follows:\n")
28 for _ in range(num_names):
29     out = []
30     ix = s_to_i['.'] # 从起始符 '.' 开始
31
32     while True:
33         # 逐字预测：tensor自动在GPU上创建
34         xenc = F.one_hot(torch.tensor([ix]), num_classes=27).float()
35
36         # 前向传播
37         logits1 = xenc @ W1 + b1
38         h = torch.tanh(logits1)
39         logits2 = h @ W2 + b2
40
41         counts = torch.exp(logits2)
42         probs = counts / torch.sum(counts, dim=1, keepdims=True)
43
44         # 多项式抽样
45         ix = torch.multinomial(probs, num_samples=1, replacement=True, generator=g_gen).item()
46
47         # 遇到结束符则停止
48         if ix == s_to_i['.']:
49             break
50
51     out.append(i_to_s[ix])

```

```
52
53     print("".join(out))
```

14.1.2. 代码优化

如果已经换到了GPU，运算时明明显卡占用爆表，可是速度没有提上去，那可能是CPU在计算时拖后腿了。

因为我们为了向你全面、清晰地展示整个代码构建过程，使用了大量中间变量，其中有些变量是交给CPU运算的，我们这里可以将代码精简一下：除掉多余无关中间变量、将for循环内的重复计算移出。

整段代码如下：

```
1 import torch
2 import torch.nn.functional as F
3 torch.set_printoptions(linewidth=300)
4
5 # 设置默认设备，之后所有新创建的张量都会在GPU上
6 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
7 torch.set_default_device(device) # PyTorch 2.0+ 推荐方式
8
9 # 性能优化设置
10 if device.type == 'cuda':
11     # 添加非阻塞数据传输
12     torch.backends.cudnn.benchmark = True # 自动寻找最优算法
13     torch.backends.cuda.matmul.allow_tf32 = True # 允许TF32加速（Ampere及以上架构）
14
15 # 导入与处理数据
16 file_path = "names.txt"
17 with open(file_path, "r", encoding="utf-8") as f:
18     data = list(".".join(f.read().splitlines())) # 消除中间变量
19
20 # 映射
21 s_to_i = {ch: i for i, ch in enumerate(sorted(set(data)))} # 消除中间变量
22 i_to_s = {i: ch for ch, i in s_to_i.items()}
23
24 # 参数设置
25 g = torch.Generator(device=device).manual_seed(123987)
26 W1 = torch.randn((27, 27), generator=g, dtype=torch.float32)
27 b1 = torch.randn((1, 27), generator=g, dtype=torch.float32)
28 W2 = torch.randn((27, 27), generator=g, dtype=torch.float32)
29 b2 = torch.randn((1, 27), generator=g, dtype=torch.float32)
30
31 # 大循环
32
33 # 在进入大循环之前（大循环外部）一次性处理完毕
34 A_data = ["."] + data
35 data_B = data + ["."]
36 xs = [s_to_i[ch] for ch in A_data]
37 ys = [s_to_i[ch] for ch in data_B]
38 X_all = torch.tensor(xs, dtype=torch.long)
39 Y_all = torch.tensor(ys, dtype=torch.long)
40
41 # 整个数据集的 one_hot 直接在外面做完（如果显存允许）
42 X_enc_all = F.one_hot(X_all, num_classes=27).float()
43 Y_enc_all = F.one_hot(Y_all, num_classes=27).float()
44
45 # 循环内参数
46 epochs = 100
47 batchsize = 512 # step参数；设置为设置为 2 的幂次方，适配GPU的流多处理器调度线程，且符合黄金法则（样本数是特征数的10-100倍）。
48 Total_len = len(A_data) # stop参数
49 lr = 0.1
50
```

```

51 for epoch in range(epochs):
52     # 小循环
53     for start in range(0, Total_len, batchsize):
54         end = start + batchsize
55         # 内部直接切片! 没有任何 Python 字符串操作, 速度提升百倍
56         xenc = X_enc_all[start:end]
57         yenc = Y_enc_all[start:end]
58
59         # 前向传播
60         logits1 = xenc @ W1 + b1
61         h = torch.tanh(logits1)
62         logits2 = h @ W2 + b2
63         counts = torch.exp(logits2)
64         counts_sum = torch.sum(counts, dim=1, keepdims=True)
65         counts_sum_inv = (counts_sum + 1e-8) ** -1 # 在counts_sum中加一个
极小的常量 (如 1e-8), 避免NaN/inf出现
66         probs = counts * counts_sum_inv
67         # probs = exp_logits2 / torch.sum(exp_logits2, dim=1, keepdims=True)
e) # 可以写为一行, 减少中间变量, 提高运行速度, 避免NaN/inf出现
68         log_probs = -1 * yenc * torch.log(probs)
69         sum_loss = torch.sum(log_probs, dim=1)
70         L = torch.mean(sum_loss)
71
72         # 反向传播
73         dL = 1.0
74         dsum_loss = dL / xenc.shape[0]
75         dlog_probs = dsum_loss * torch.ones_like(log_probs)
76         dprobs = dlog_probs * -1 * yenc * (probs) ** -1
77         dcounts = dprobs * (counts_sum_inv * torch.ones_like(counts)) # 相
当于一次手动广播
78         dcounts_sum_inv = torch.sum(dprobs * counts, dim=1, keepdim=True)
79         dcounts_sum = dcounts_sum_inv * -1 * ((counts_sum + 1e-8) ** -2)
80         dcounts += dcounts_sum * torch.ones_like(counts) # 相当于一次手动广
播
81         dlogits2 = dcounts * torch.exp(logits2)
82         # dlogits2 = (probs - yenc) / batch_size # 将 Softmax 和 Cross-Entropy
结合在一起从数学公式上化简, 你会发现一个极其优雅的结果
83         dW2 = h.T @ dlogits2
84         dh = dlogits2 @ W2.T
85         db2 = torch.sum(dlogits2, dim=0, keepdim=True)
86         dlogits1 = dh * (1 - h**2)
87         dW1 = xenc.T @ dlogits1
88         db1 = torch.sum(dlogits1, dim=0, keepdim=True)
89
90         # 梯度更新
91         W2 = W2 - lr * dW2
92         W1 = W1 - lr * dW1
93         b2 = b2 - lr * db2
94         b1 = b1 - lr * db1
95
96     print(f"Finshed! loss: {L.item():.4f}")
97
98     # 保存模型参数到硬盘
99     checkpoint = {

```

```

100     'w1': w1,
101     'b1': b1,
102     'w2': w2,
103     'b2': b2,
104     's_to_i': s_to_i,  # 顺便把映射表也存进去，防止以后字符顺序变了导致索引对
    不上
105     'i_to_s': i_to_s
106 }
107 torch.save(checkpoint, 'model.pt')
108 print("The results have been successfully exported as the file 'model.p
    t'")

```

14.2. 为什么显存越大越好？

关于这个问题，还要从这两行代码开始：

```

1 print(f"loss:{L.item()}")
2 print(f"loss:{L}")

```

14.2.1. 深度学习框架（PyTorch）底层计算图与内存管理机制

- 叶子节点 vs 非叶子节点

内存中存在两类截然不同的变量，它们的命运完全不同：

叶子节点 (Leaf Tensor)： 由人工手动创建的变量（如模型参数 W_1, W_2, b_1, b_2 ），它们是计算图的起点。特点：占据**固定内存**。对于设置了 `requires_grad=True` 的参数，反向传播后不仅自身的值会被保留，其计算出的梯度也会被永久保存在 `.grad` 属性中（我们是手动反向传播，所以并未设置 `requires_grad=True` 参数，后边我们会用到）。

非叶子节点 (Non-Leaf Tensor / 中间变量)： 由叶子节点经过数学运算（如加减乘除、激活函数等）产生的所有中间变量（如 `out1`, `prob`, `loss`）。特点：占据**临时内存**。它们是计算图的组成部分，一旦反向传播完成，它们的值和梯度会被**立即销毁**。

- 动态生命周期：脚手架的搭建与拆除

我们可以用 "盖楼" 来完美还原一个 Training Step 的底层过程：

1. **前向传播 (搭脚手架)：** 数据 X 和参数 W 进行运算。为了后续能利用链式法则求导，框架会在后台搭建一个 "临时计算图 (脚手架)"，并将海量的**中间变量 (非叶子节点)** 像砖头水泥一样暂存在脚手架上。
2. **反向传播 (质检员下楼)：** `loss.backward()` 触发。程序顺着脚手架往回推导，读取中间变量，算出每个参数的梯度并贴在叶子节点上（赋值给 `W.grad`）。
3. **图的销毁 (拆除脚手架)：** 反向传播结束的瞬间，临时计算图连同上面所有的中间变量（包括 `loss` 本身）被瞬间释放，交还显存。
4. **权重更新 (原地挪砖)：** `optimizer.step()` 仅读取固定的 W 和 `W.grad` 进行原地加减法 $W \leftarrow W - lr \times W.grad$ 。
5. **梯度清零 (撕掉标签)：** `optimizer.zero_grad()` 将 `W.grad` 清零，准备下一轮循环。

14.2.2. 为什么需要给loss加一个.item()?

- 危险区：Python 引用与内存泄漏 (OOM)

虽然框架会自动拆除脚手架，但如果触发了 Python 的**引用保留**，垃圾回收机制 (GC) 就会失效，导致内存爆炸。

什么是引用？ 将一个带有计算图的中间变量（如 `loss` 或 `out`）赋值或累加给**循环外部的长效变量**（如全局列表、全局累加器）。

这相当于用绳子把临时变量死死拴住，导致整个庞大的计算图无法被销毁。

正确解法（金蝉脱壳）：使用 `.item()` 或 `.detach()`。例如 `total_loss += loss.item()`，只剥离出纯数值 (Float)，切断与计算图的联系，让原图安心销毁。

14.2.3. 为什么我们需要大显存？

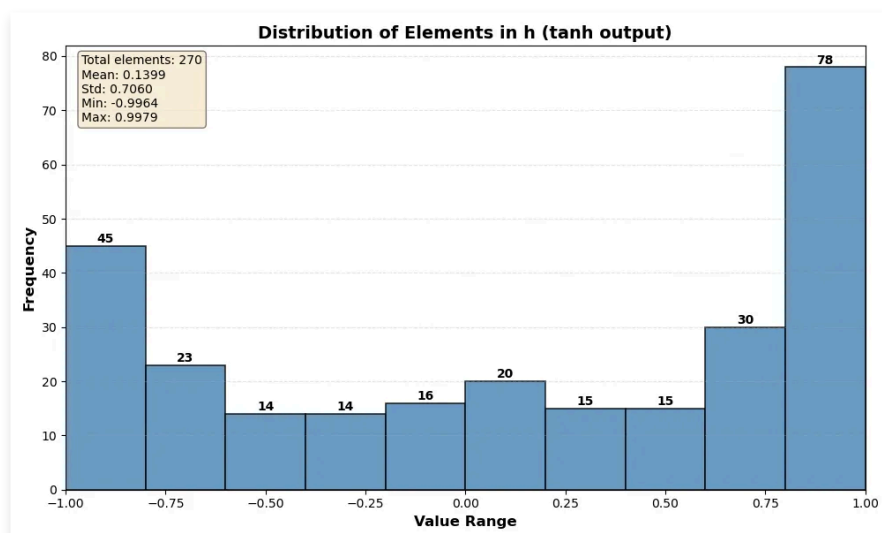
模型参数 W 本身（固定内存）占用的显存其实很小。

真正吃显存的“内存杀手”，是前向传播时保存在计算图中的海量中间变量（临时内存）。

显存越大，意味着你能搭建的“脚手架”越大。这就是为什么更大的显存可以支持**更大的模型参数量、更深的网络层数以及更大的 Batch Size**。

14.3. Xavier 初始化

还记得我们在前向传播中发现的有趣的统计规律吗？



梯度消失危机

当时我们讲到这里，只告诉你这样的数据分布在深层次的网络中会导致梯度消失，但我并未告诉你解决办法。

这里，我们就用概率论武器来破解这个难题！

14.3.1. 高斯采样: .randn()

在开始之前,我想先向你解释一下关于数据与权重初始化的采样方式。

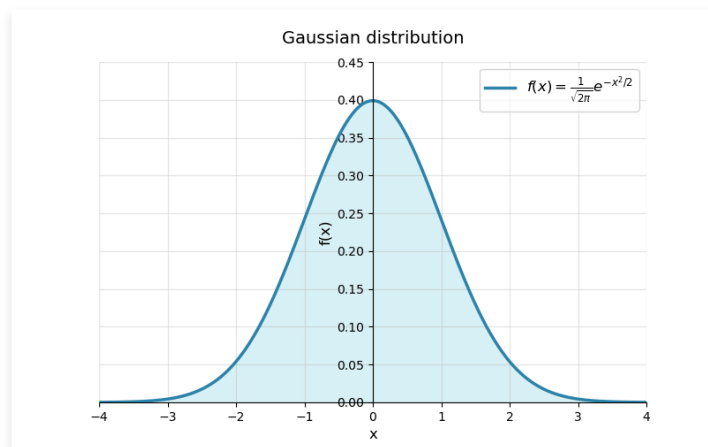
想必你在初始化 $W1$ 的时候,已经注意到`torch.randn()`了。

`torch.randn()` 是基于标准正态分布(高斯分布)进行采样的。

$$\text{正态分布: } f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

当 $\mu = E[X] = 0$; $\sigma^2 = \text{Var}(X) = E[(X - \mu)^2] = 1$ 时为标准正态分布。

$$\text{标准正态分布及图像: } f(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$$



高斯分布采样的特点是:

- **y轴(概率密度)越高** → 对应的x值**越容易被采到**;
- **y轴(概率密度)越低** → 对应的x值**越难被采到**;
- 某区间对应曲线下面积为该区间采样出现的概率。

14.3.2. 概率论中“大量独立同分布变量之和”的方差

我们在“概率论前置基础约定”中已经见过期望和方差了。

方差描述了高斯分布的“胖瘦”。方差越小,高斯分布越尖锐/高瘦,反之则平滑/矮胖。

假设我们有两组**相互独立**的随机变量 $x_1, x_2, x_3, \dots, x_n$ 和 $w_1, w_2, w_3, \dots, w_n$ 。

所有 x_i 独立同分布,服从某高斯分布;所有 w_i 独立同分布,服从某高斯分布,且 x_i 与 w_i 相互独立。

那 $h = \sum_{i=1}^n x_i w_i$ (逐元素乘积之和) 会服从什么样的分布?

根据概率论中独立变量乘积的方差公式:

$$\text{Var}(x_i w_i) = \mathbb{E}[w_i]^2 \text{Var}(x_i) + \mathbb{E}[x_i]^2 \text{Var}(w_i) + \text{Var}(w_i) \text{Var}(x_i)$$

因为 $x_1, x_2, x_3, \dots, x_n$ 和 $w_1, w_2, w_3, \dots, w_n$ 的均值 $\mathbb{E}[x_i]$ 和 $\mathbb{E}[w_i]$ 都是 0, 所以前两项为 0, 公式简化为:

$$\text{Var}(x_i w_i) = \text{Var}(w_i) \cdot \text{Var}(x_i)$$

因为 h 是 n 个这样的项相加, 且各项独立, 所以两侧同时求和, h 的独立性令其穿越求和符号:

$$\sum_{i=1}^n \text{Var}(x_i w_i) = \text{Var} \sum_{i=1}^n (x_i w_i) = \text{Var}(h) = \sum_{i=1}^n (\text{Var}(w_i) \cdot \text{Var}(x_i))$$

因为 x_i 和 w_i 独立同分布, $\text{Var}(w_i) \cdot \text{Var}(x_i)$ 对任意 i 来说都相等, 所以:

$$\text{Var}(h) = n \cdot \text{Var}(w_i) \cdot \text{Var}(x_i) \text{ (对任意 } i \text{ 成立)}$$

14.3.3. “大量独立同分布变量之和”的方差在输入层中的体现

基础工具已经搭建到位。

上述 $h = \sum_{i=1}^n x_i w_i$ 形式如同我们在BigramMLP中输入层 “logits1 = xenc @ W1 + b1” 一样。

只不过, 这里的xenc并不服从高斯分布, 但后边在多元字符模型 (ngramMLP) 中我们会通过嵌入向量, 将数据转换为嵌入矩阵, 而嵌入向量的数据是服从高斯分布的。

我们现在就先用.randn()随机生成两个服从高斯分布的矩阵, 来看一下他们的乘积之和的分布特点:

```
1 import torch
2
3 g = torch.Generator().manual_seed(123789)
4 X = torch.randn((64, 27), dtype=float(), generator=g)
5 W = torch.randn((27, 64), dtype=float(), generator=g)
6 h = X @ W # 偏置只会影响分布的均值, 而不影响方差, 这里不在讨论重点中, 所以不加偏置项。
```

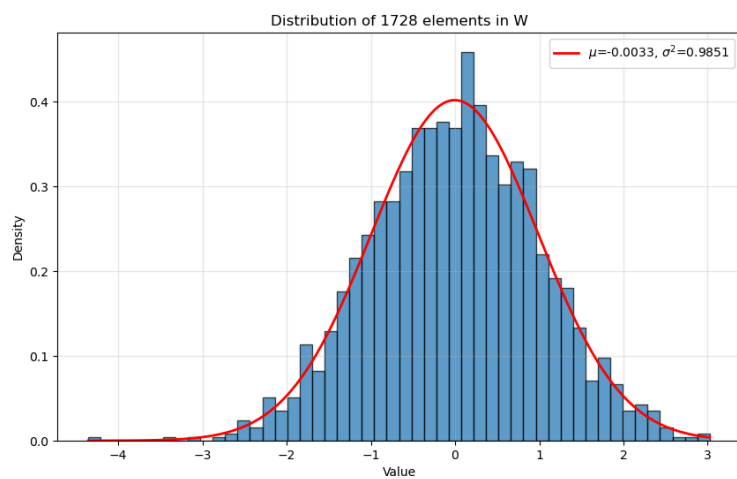
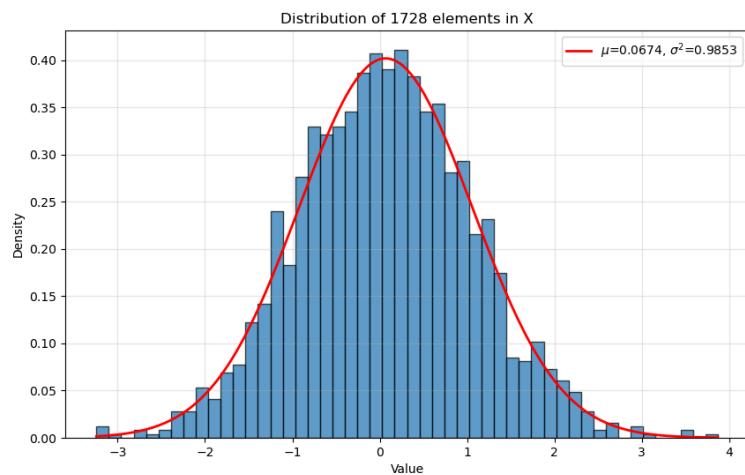
X 中的每一行代表 $x_1, x_2, x_3, \dots, x_n$;

W 中的每一列代表 $w_1, w_2, w_3, \dots, w_n$;

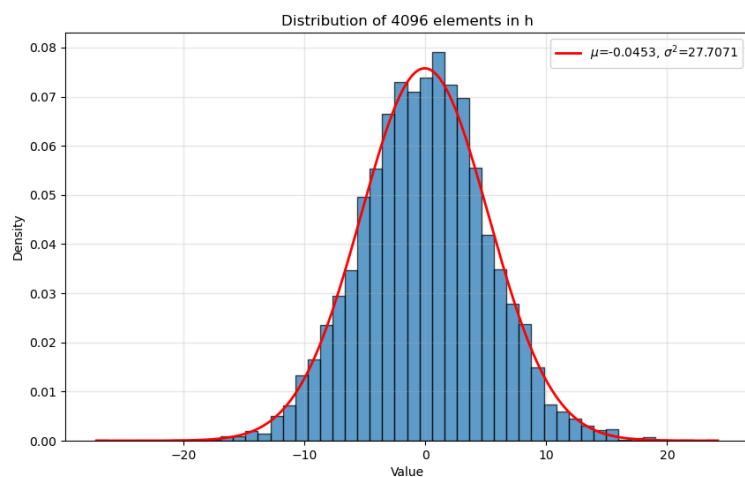
h 中每一个元素代表 X 中的某一行与 W 中的某一列的乘积, 整个 h 中所有元素独立同分布。

我们绘制图像, 分别看一下 X 中的1728个元素、 W 中的1728个元素和 h 中的4096个元素分别服从什么分布。

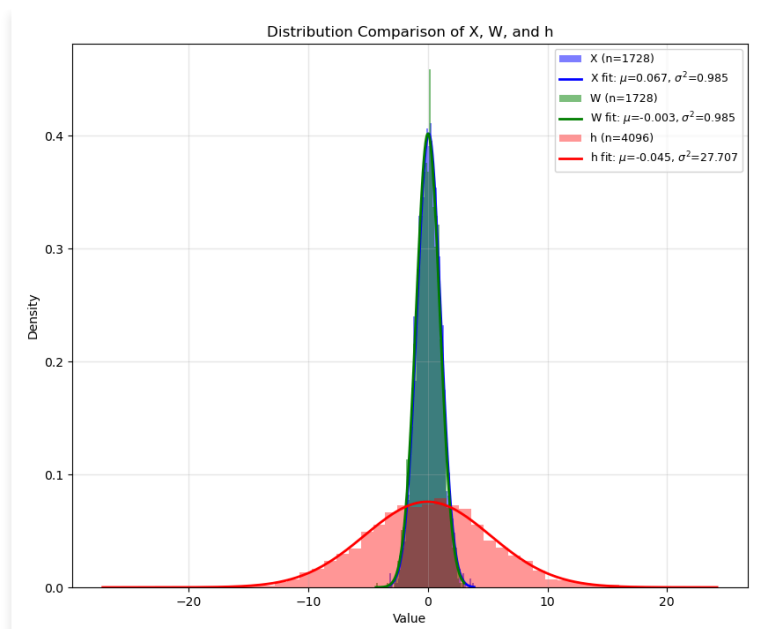
可见 X 、 W 分别服从标准正态分布:



但 h 中的4096个元素方差远大于1, 属于非标准正态分布:



放在一起会更明显:



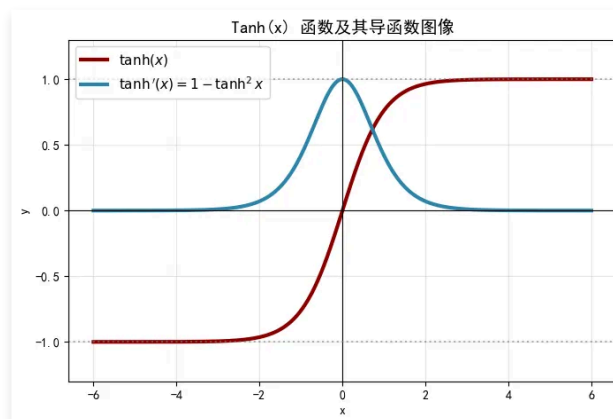
标准正态分布中位于 $[-\infty, -2]$ 和 $[2, \infty]$ 区间下的数值，被采样的概率之和是**4.55%**。

而 h 的非标准正态分布中位于 $[-\infty, -2]$ 和 $[2, \infty]$ 区间下的数值，被采样的概率之和高达**70.40%**，也就是说logits1中70.40%的数据落在 $[-\infty, -2]$ 和 $[2, \infty]$ 区间中。

别忘了，我们的logits1下一步就被激活函数 $\tanh()$ 压缩了。

```
1 logits1 = xenc @ W1 + b1
2 h = torch.tanh(logits1)
```

再次回顾 $\tanh(x)$ 的图像。



当 $x < -2$ 或 $x > 2$ 时， $\tanh(x) < -0.96$ 或 $\tanh(x) > 0.96$ ，这时 $\tanh'(x) < 0.07$ 。

当我们经过两层神经网络反向传播时，分布在 $[-\infty, -2]$ 和 $[2, \infty]$ 这70.40%的元素的梯度就会变为上游梯度的0.0049倍，一旦层数增多，梯度将会呈**指数级缩小**，导致梯度消失甚至内存下溢。

14.3.4. 解决方法

我们的目的是希望 $\text{Var}(h)$ 不要太大，最好是保持与输入变量的方差一致：

因为：

$$\text{Var}(h) = n \cdot \text{Var}(w_i) \cdot \text{Var}(x_i) \text{ (对任意 } i \text{ 成立)}$$

我们想要令 $\text{Var}(h) = \text{Var}(x_i)$ ，于是方程两侧就可以消去 $\text{Var}(h)$ 和 $\text{Var}(x_i)$ ，得到：

$$\text{Var}(w_i) = \frac{1}{n}$$

方差缩放 $\frac{1}{n}$ ，则单个变量缩放 $\frac{1}{\sqrt{n}}$ 。这个 n 在权重中表示神经元权重数，于是初始权重矩阵设置时应当在设置时乘上 $\frac{1}{\sqrt{n}}$ 。

在代码中：

```
1 W1 = torch.randn((27, 27), generator=g) * (1/(27 ** 0.5))
2 W2 = torch.randn((27, 27), generator=g) * (1/(27 ** 0.5))
```

同理，在反向传播时，梯度是从后往前传。梯度的计算公式和前向传播是对称的，只不过此时输入变成了“上一层传回来的梯度”，而参与相乘的权重数量变成了**当前层输出神经元的数量** n_{out} 。为了保证反向传播时梯度不爆炸不消失，用完全相同的推导逻辑，我们会得到：

$$\text{Var}(w) = \frac{1}{n_{out}}$$

现在矛盾来了：

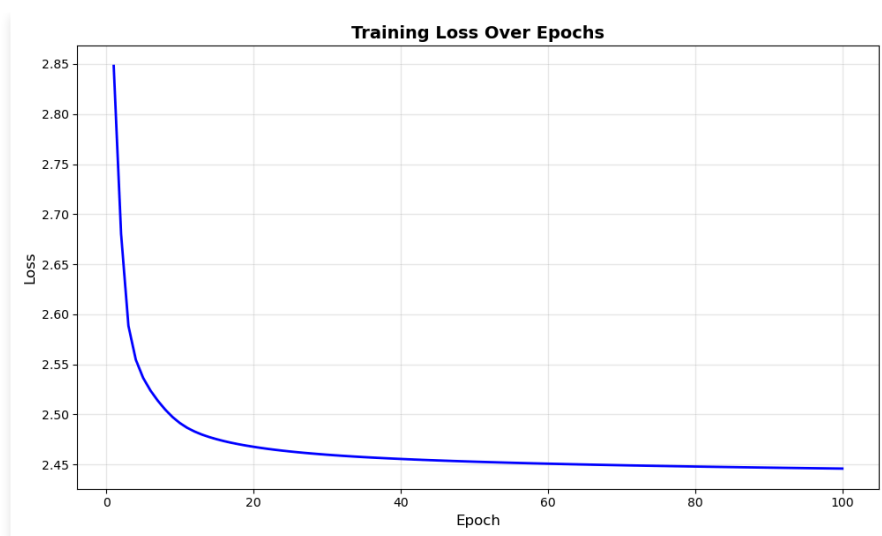
- 为了前向传播稳定，要求 $\text{Var}(w) = 1/n$
- 为了反向传播稳定，要求 $\text{Var}(w) = 1/n_{out}$ 除非输入维度和输出维度相等 $n = n_{out}$ ，否则这两个条件无法同时满足。

Xavier Glorot 和 Yoshua Bengio 在 2010 年的论文中提出了一个折中方案：**取两者的调和平均数（或者说把分母加起来求平均）**。为了同时照顾前向和反向传播，他们将方差设定为：

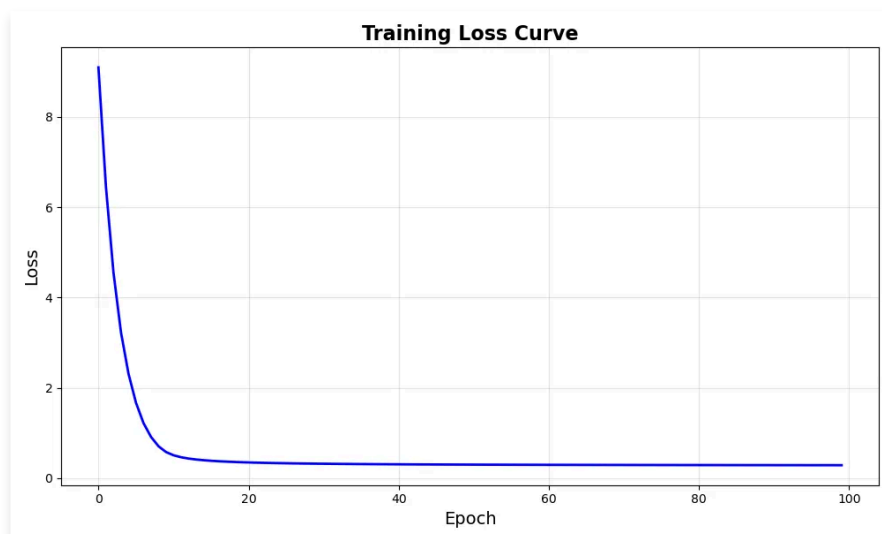
$$2/(n + n_{out})$$

值得注意的是，我们的xenc并非来自高斯分布，我们只需要在后边乘一个小的数就好了，比如**0.1**。

我们再次运行代码，发现loss下降曲线有了新的变化：



方差修正W1、W2的损失曲线



未经方差修正W1、W2的损失曲线

修正后，损失从2.8左右开始下降，而非一开始的9.5左右。在大规模训练中，可以帮助我们减少前十几轮循环中算力的浪费！

而且最终loss也变为了2.45，优于之前的2.52！

14.4. 学习率优化器

Karpathy一开始并未提到学习率的优化器，本节是我在闲暇时想到的一点，觉得很有趣，分享给各位。后来才知道有大名鼎鼎的Adam优化器，不过，作为启发式的学习案例，还是比较有意思的~

为什么要设计一款学习率优化器？

在我们设计的代码中，学习率是一个固定死的数值，每次我们觉得训练效果不好的时候，就要去修改后重新训练，这对训练算力和时间的消耗是巨大的；

如果学习率被固定，相当于将每次的步子都限制了，当loss下降顺利时，我们希望学习率大一点，当loss下降慢时，我们希望它小一点，慢慢接近，这天然需要我们动态调整学习率；

14.4.1. 损失的变化率

loss的变化可以用变化率来描述，我们先定义损失的变化率。

令第 n 次迭代的损失为 L_n ; 则损失的变化率可以表示为: $loss_rate = \frac{L_{n+1} - L_n}{L_n}$, 且 $L \geq 0$ 。

想一下, 损失的变化率取值可能什么样, 以及对应的情况是什么:

- 当 $L_{n+1} > L_n$, 意味着loss不降反升, 很可能是我们的步子迈地太大了(学习率太大), 也有可能是新的一个epoch开始了, 我们这时候将学习率收一收总是没问题的。
- 当 $L_{n+1} < L_n$, 意味着loss正常下降, 很可能是在正常进行, 当然也有能是我们的步子迈地太小了(学习率太小), 不管因为什么, 我们这时候大胆将学习率放大一点, 这总归是没问题的。
- 当 $L_{n+1} = L_n$, 意味着loss基本平稳, 我们不需要调整学习率。

14.4.2. 寻找合适的函数

对于以上三种情况, 我们需要分别做这几件事:

- 当 $L_{n+1} > L_n$, 需要 $lr \times factor$, 这个 $factor$ 应该是个小于1的数, 来遏制学习率。
- 当 $L_{n+1} < L_n$, 需要 $lr \times factor$, 这个 $factor$ 应该是个大于1的数, 来鼓励学习率放大。
- 当 $L_{n+1} = L_n$, 需要 $lr \times factor$, 这个 $factor$ 应该是个等于1的数, 希望学习率不做变化。

那接下来的关键就是, 如何将 $loss_rate = \frac{L_{n+1} - L_n}{L_n}$ 和这个 $factor$ 捆绑在一起。

我们想到了tanh函数, 它可以将任意实数压缩到 $(-1, 1)$ 。

对于 $loss_rate = \frac{L_{n+1} - L_n}{L_n}$ 来说:

- $L_{n+1} > L_n$, $loss_rate > 0$, $\tanh(loss_rate) > 0$, $1 - \tanh(loss_rate) < 1$;
- $L_{n+1} < L_n$, $loss_rate < 0$, $\tanh(loss_rate) < 0$, $1 - \tanh(loss_rate) > 1$;
- $L_{n+1} = L_n$, $loss_rate = 0$, $\tanh(loss_rate) = 0$, $1 - \tanh(loss_rate) = 1$;

这个函数就这样被我们找到了, 我们令 $factor = 1 - \tanh(loss_rate)$ 。

有趣的是, 这个函数最大可以将学习率翻一倍(当 $loss_rate = -2.64$, $factor = 1.99$), 但最小却可以缩放到原来的成百上千分之一(当 $loss_rate = 2.64$, $factor = 0.01$)。

也就是说, 这个函数鼓励学习率放大的时候也是谨慎的, 但对于loss不降反升这种情况的惩罚则会明显地多。

如果你觉得放大一倍是不可接受地, 希望函数更平滑地改变学习率, 我们可以令 $\tanh(loss_rate)/2$, 这样就更加平滑了。

最终, 学习率加装优化器后的最终版本是:

$$lr_{new} = lr_{old} \times (1 - \frac{\tanh(\frac{L_{n+1} - L_n}{L_n})}{2}), n > 0$$

14.4.3. 代码处理

尽管数学上的处理很简单, 但是在代码中我们需要考虑这样几个很现实的问题:

- 虽然损失值为0的概率很小，但如何保证 $loss_rate = \frac{L_{n+1} - L_n}{L_n}$ 在运算中不会因为偶然事件导致分母为0？
- 仅考虑相邻两次的损失变化率，极其容易受到噪声的影响，如何更加平滑地排除噪声影响？
- 虽然 $n > 0$ ，但是学习率地更新需要至少2次loss数据，能使用优化后地学习率时，起码要第3轮迭代，如何将优雅地将loss存储进一个列表，使得这个列表不会因不断增长而吃掉内存？

第一个问题我们已经有相关经验可以解决了，给分母加一个极小值，比如 10^{-8} ，保证了分母不会为

0: $loss_rate = \frac{L_{n+1} - L_n}{L_n + 10^{-8}}$ 。

第二个问题，这里我们选择求均值的方法（平滑滤波）。

我们不再只关注两次loss之差，而是获得20次迭代的loss，选取前10次，求一个均值，作为 L_n ，选取后10次，求一个均值，作为 L_{n+1} 。

为了避免极大值极小值的影响，我们升级平滑滤波为截尾均值。求10个loss中最大最小值剔除后的均值。

第三个问题，我们使用 `loss_list.clear()` 来清除每次计算后的列表，避免内存消耗。

增加代码如下：

```

1  import math
2  ...
3  # 大循环
4  # 循环内参数
5  epochs = 100
6  batchsize = 512
7  Total_len = len(A_data)
8  loss_list = [] # 增加列表，用于学习率优化
9  lr = 0.1
10 ...
11 for epoch in range(epochs):
12     # 小循环
13     for start in range(0, Total_len, batchsize):
14         ...
15         # 学习率优化器
16         losses = 10
17         loss_list.append(L.item()) # 将loss值剥离并添加到列表，防止计算图引用
18         if len(loss_list)<(losses * 2): # 小于20个数据时沿用原来的lr
19             # 梯度更新
20             W2 = W2 - lr * dW2
21             W1 = W1 - lr * dW1
22             b2 = b2 - lr * db2
23             b1 = b1 - lr * db1
24         else: # 列表中有20个数据时则更新学习率列表
25             new = loss_list[losses:]
26             old = loss_list[:losses]
27             L_new = sum(sorted(new)[1:-1]) / (len(new) - 2)
28             L_old = sum(sorted(old)[1:-1]) / (len(old) - 2)
29             loss_rate = (L_new - L_old)/(L_old + 1e-8)
30             lr = max(lr * (1 - math.tanh(loss_rate) / 2), 1e-5)
31             # 梯度更新
32             W2 = W2 - lr * dW2
33             W1 = W1 - lr * dW1
34             b2 = b2 - lr * db2
35             b1 = b1 - lr * db1
36
37             # 删除最开始的10个old loss，重新开始存入数据，保证每10个loss一计算
38             del loss_list[:losses]
39
40         print(f"learning rate: {lr}\n")
41     print(f"Finshed! loss: {L.item():.4f}")

```

当你的终端最终显示：

▼ Plain Text

```

1  learning rate: 0.05957401059129616
2  Finshed! loss: 2.5062

```

说明我们的优化器确实起作用了，相比于最开始的lr = 0.1，最终调整为0.05957401059129616。

你可以试着调整losses的值，试一下不同数量下，学习率对训练的影响。

14.5. 代码中的数据类型

15. L2正则化 (L2 Regularization)

正则化是一个庞大的概念家族，包括L1 正则化、L2 正则化、Dropout 正则化等。

我们在本章仅了解一下L2正则化。

15.1. L2正则化起源与过程

从贝叶斯到高斯。从理论到实践。

15.1.1. 起源

- 我先问你个问题：袋子里有红色和蓝色且质地相同的小球，数量巨大，每次抽取后记录颜色，再放回去摇匀继续抽取，在抽取10次后，其中有6次红色球，4次蓝色球，请你分析第11次抽取时，抽到袋子中红色球的概率是多少？

这个问题实际上可以换个问法：抽到红色球这件事，最可能的概率是多少？

因为每次抽取相互独立，且非红即蓝，我们令每次抽到红球的概率 $P(red) = x$ ，则抽到蓝球的概率时 $P(blue) = 1 - x$ 。

则在抽取10次后，其中有6次红色球，4次蓝色球的概率为 $P(D|x) = \binom{10}{6} x^6 (1-x)^4$ 。

我们求 x 的概率，就是在问， x 的概率是多少时最合理（最可能的概率）。在上述公式中 x 最合理时即 $P(sample|x)$ 处在极值处。

原式求导太复杂，两侧同时取对数不改变单调性，得到：

$$\log P(D|x) = \log \binom{10}{6} + 6 \log x + 4 \log(1-x)。$$

再两侧求导得到： $(\log P(D|x))' = 6/x - 4/(1-x) = 0$ ，解的 $x = 0.6$ 。

- 贝叶斯公式

未知参数： x = 单次抽到红球的概率， $x \in (0, 1)$

观测数据： $D = 10$ 次有放回抽样，结果为**6 红 4 蓝**

$\pi(x)$ ：**先验分布**（抽样前对 x 的认知）

$P(D|x)$ ：**似然**（给定参数 x ，出现数据 D 的概率）

$P(x|D)$ ：**后验分布**（看到数据后，参数 x 的分布）， $P(x|D) \sim \text{Beta}(7, 5)$

$$P(x | D) = \frac{P(D | x) \cdot \pi(x)}{P(D)}, \text{ 代入计算可得 } x = 0.6。$$

- 高斯均值分布

上述例子讲的是Beta分布。

但对于权重采样的标准正态分布而言，先验分布（高斯先验）为均值，也就是0。

这意味着什么？在没有数据的支撑下，我们天然认为权重取值大概率为0（取0是最合理的）。

只有数据提供了足够强的“证据”（梯度），权重才会偏离0。而且偏离得越多，需要的数据证据就越强。

这就是**奥卡姆剃刀**的数学化：如无必要，勿增实体。没证据就让权重靠近0，有强烈证据才允许它长大。

所以我们的权重在更新过程中，需要一股力量将其拉回均值（0）。

- L2正则化 = 高斯先验

如果先验 $P(W)$ 是一个**均值为0的高斯分布**：
$$P(W) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{W^2}{2\sigma^2}}$$

取对数：
$$\log P(W) = -\frac{W^2}{2\sigma^2} + \text{常数}$$

最大化 $\log P(W | D)$ 等价于最小化：
$$-\log P(D|W) + \frac{1}{2\sigma^2} W^2$$

令 $\lambda = \frac{1}{\sigma^2}$ ，这就是：原始损失 + λW^2

这正是L2正则化。

15.1.2. 过程

我们在原本的损失函数（Loss）后面，加上惩罚项 $\frac{1}{2} \lambda W_i^2$ （ λ 是一个很小的常数； i 是权重矩阵的计数）：

$$L_{new} = L_{old} + \sum_i \left(\frac{1}{2} \lambda W_i^2 \right)$$

因为在 L 到 W^2 之间的所有中间变量都不涉及对 W^2 的求导，所以梯度层层忽略，直到反向传播到 W^2 时， dW^2 的梯度需要补充一个 λW_i ：

$$\frac{\partial L}{\partial \mathbf{W}^2} = \mathbf{h}^T \cdot \frac{\partial L}{\partial \text{logits}^2} + \lambda \mathbf{W}^2$$

同理：

$$\frac{\partial L}{\partial \mathbf{W}^1} = \mathbf{x} \text{enc}^T \cdot \frac{\partial L}{\partial \text{logits}^1} + \lambda \mathbf{W}^1$$

于是在梯度更新时：

$$\mathbf{W2} \leftarrow \mathbf{W2} - lr \times (\mathbf{h}^T \cdot \frac{\partial L}{\partial \text{logits2}} + \lambda \mathbf{W2}) = \mathbf{W2} - lr \times \mathbf{h}^T \cdot \frac{\partial L}{\partial \text{logits2}} - lr \times \lambda \mathbf{W2} = (1 - lr \times \lambda) \mathbf{W2} - lr \times \mathbf{h}^T \cdot \frac{\partial L}{\partial \text{logits2}}$$

因为 $1 - lr \times \lambda$ 是个略小于1的数字，所以梯度更新时，权重矩阵大小会按照 $1 - lr \times \lambda$ 比例缩小，在数值上的表现就是每个数字都往0靠拢。绝对值大的收缩快，绝对值小的收缩慢

上述过程被称为**权重衰减 (Weight Decay)**。

在代码中：

```
1  lambda = 0.01
2  ...
3  dw2 = h.T @ dlogits2 + lambda * W2
4  ...
5  dw1 = xenc.T @ dlogits1 + lambda * W1
6  ...
7  W2 = W2 - lr * dw2
8  W1 = W1 - lr * dw1
9  # 在实际网络中，我们并不手动构建反向传播，所以更好的做法是把lambda写在梯度更新里：
10 lambda = 0.01
11 ...
12 dw2 = h.T @ dlogits2
13 ...
14 dw1 = xenc.T @ dlogits1
15 ...
16 W2 = ( 1 - lr * lambda ) * W2 - lr * dw2
17 W1 = ( 1 - lr * lambda ) * W1 - lr * dw1
```

15.2. 为什么需要正则化？

正则化的功能是防止过拟合，提高泛化能力。

如果权重 W 的绝对值非常大，意味着模型对输入的微小变化极其敏感（输入稍微变一点，乘以巨大的权重后，输出就会剧烈波动）。这种模型往往是在“死记硬背”训练集里的噪声和异常值，导致泛化能力极差（过拟合）。通过权重衰减强行压制 W 的大小，可以迫使模型学习到一条**平滑、简单**的决策边界，从而在没见过的新数据上表现更好。

回顾权重更新公式（权重衰减形式）：

$$W \leftarrow W + \underbrace{(-\eta\lambda)W}_{\text{衰减力}} + \underbrace{-\eta \cdot W.\text{grad_data}}_{\text{数据引力}}$$

更新梯度时，有两股力量博弈：一个是衰减力，负责缩小自己，往0靠拢（基于权重为0的先验分布）；另一部分是数据引力，负责指导权重走多少。

数据引力斩钉截铁地（一开始它的影响很大）说：“你要往前走3步，这样就能使损失减小了”。

衰减力弱弱地（ $lr \times \lambda$ 是个很小的数，话语权不高）说：“你可以往前走，但是我要你先缩小一下你自己，不要把步子迈地太大”。

当随着 W 不断更新, loss 越来越低, W 也就越来越接近自己地目的地 (最终训练完成状态的 W) 。

这时数据引力越来越小, 梯度更新越来越慢, 终于到达一个点, 和衰减力相等。

$$W = (1 - \eta\lambda)W - \eta \cdot W.\text{grad_data}$$

$$\lambda W = -W.\text{grad_data}$$

不加正则化时: 梯度下降会一直走到训练损失的**全局最低点**, 此时模型在训练集上损失最小, 但容易过拟合, 因为它完全 “记住” 了训练数据的噪声。

加了 L2 正则化后: 正则项在损失函数里增加了一个 “惩罚项”, 相当于把原来的损失曲面 “抬高” 了, 它的**真实目标是最小化**: $\text{Loss}(W) + \lambda \|W\|_2^2$, 而不是只最小化数据损失 $\text{Loss}(W)$ 。

所谓 “平衡点”, 其实是**正则化后的总损失的最低点**, 而不是数据损失的最低点。在这个点上, 模型为了满足正则项的约束, 没有把权重调整到完全拟合训练数据噪声的程度, 因此保留了泛化能力。

16. 结语

现在是2026年6月。最近常听人提起 “AI泡沫” 这个词, 我不懂经济学上的泡沫具体意味着什么, 但 “AI技术泡沫” 可能是存在的。我所认为的泡沫在根源上来自于神经网络理论上的不完备性。

太多工程上的缝缝补补, 例如在超参数的选择上, 并没有系统完备的理论指导。试错对于一项风靡全球的技术来说是不可接受。

当然, 或许真如Yann LeCun所言, 大模型无法通向AGI。以目前的技术发展上来看, 尽管AI发展很快, 但是依然是在堆算力优化算法。这与Ilya Sutskever所推崇的规模效应恰恰相反。

至于谁对谁错, 那是神仙打架。我等凡人只能坐看华山论剑。

添加 TeX 公式